



ASHOKA WOMEN'S ENGINEERING COLLEGE (Autonomous)

Accredited by NBA (CSE, CIVIL, ECE, EEE), Accredited by NAAC - A⁺
Approved by AICTE & Affiliated to JNTUA, Ananthapuramu
(NH-44, Dupadu, Kurnool-518218)



Academic Year 2025-26 COURSE PACK



ASHOKA

Course Code & Name	R233PC31 – Artificial Intelligence
Year / Class / Sem	II Year CSE AI – A, B, C – Sem I
Faculty Name	Dr. K. Samunnisa
Designation	Incharge HoD & Associate Professor-CSE



ASHOKA WOMEN'S ENGINEERING COLLEGE (Autonomous)

Accredited by NBA (CSE, CIVIL, ECE, EEE), Accredited by NAAC - A⁺
Approved by AICTE & Affiliated to JNTUA, Ananthapuramu
(NH-44, Dupadu, Kurnool-518218)



Table of Contents

S.No.	Contents
1.	Faculty Profile
2.	Pre-requisite of the course
3.	course Assessment
4.	Syllabus
5.	Course Outcomes
6.	Course Outcome Mapping with Program Outcome
7.	Lesson Plan
8.	Preface – Innovative Teaching Methods
9.	APIC Plan
10.	External References
11.	Classroom Conduct
12.	Real time feedback submission to course
13.	Course Materials

ASHOKA

Faculty Profile



Dr. K. Samunnisa

Incharge HoD & Associate Professor-CSE

Dr. K. Samunnisa is a dedicated academician with 14 years of rich experience in teaching, research, and academic administration. She has a commendable research profile, with 29 publications in reputed international journals, including 4 SCI-indexed, 8 indexed in Scopus, and has also presented 5 papers at renowned IEEE national and international conferences. A strong advocate of lifelong learning, she has completed 6 NPTEL courses and holds 5 Indian patents. Her academic journey is further enriched by her participation in 6 international workshops, enhancing her expertise in innovation and pedagogy. Her unwavering commitment to academic excellence and her active involvement in institutional accreditation reflect her passion for upholding the highest standards in education.

Research Interests:

Artificial Intelligence, Machine Learning, Deep learning & Cloud Computing.

Education:

- Ph.D. in Computer Science and Engineering from JNTUA, Anantapur, AP.
- M.Tech in Computer Science and Engineering from G. Pullaiah College of Engineering and Technology, Kurnool, AP.
- B.Tech in Computer Science and Information Technology from G. Pulla Reddy Engineering College, Kurnool, AP.

Acted as a Mentor for the following Mentees:

1. Kuruba Sumanjali-222T1A0594 was selected for grand finale in Smart India Hackathon (SIH) 2024, in Jammu & Kashmir.
2. K. Malleswaramma-222T1A0597 got a job as Branch Post Master (BPM) in Postal department, in the month of June 2025.
3. K. Aswini-222T1A0576 got a job as GDS in Postal department, in the month of January 2025.
4. A. Indira Priyadarshini- 222T1A0501, C. Varshitha-222T1A0534, K. Raajashree-222T1A0567, P. Durga Bhavani-222T1A05E1, Fraud Alliance team got selected for SAP Hackfest 2025.

Profile Links

Orcid ID: <https://orcid.org/my-orcid?orcid=0000-0002-4216-0745>

Scopus ID: <https://www.scopus.com/authid/detail.uri?authorId=57210730676>

Linkedin: <https://www.linkedin.com/in/dr-k-samunnisa-2196a4308>

Google Scholar ID: <https://scholar.google.com/citations?hl=en&user=IF3zeh8AAAAJ>



Pre-requisite for the Course

Basic Programming Skills

- Proficiency in at least one programming language (preferably Python).
- Understanding of control structures, functions, and data structures.

Mathematics Foundation

- Knowledge of linear algebra, probability, and statistics.
- Familiarity with matrices, vectors, derivatives, and Bayes theorem.

Discrete Mathematics & Logic

- Understanding of propositional and first-order logic, sets, relations, and graph theory.

Data Structures and Algorithms

- Ability to implement and analyze algorithms like search, sort, recursion, and trees/graphs.

Course Assessment

For R23 Regulation

Tool	2-Marks Questions	Descriptive Questions	Assignment (Case Study)
Mid-I	10 marks	30 marks condensed to 15 marks	5 marks
Mid-2	10 marks	30 marks condensed to 15 marks	5 marks
Internal (30)	80% of best mid mark & 20% of other mid mark		
External (70)	End Semester Exam		
Total (100)	Internal (30) + External (70)		



ASHOKA WOMEN'S ENGINEERING COLLEGE (Autonomous)

Accredited by NBA (CSE, CIVIL, ECE, EEE), Accredited by NAAC - A⁺
Approved by AICTE & Affiliated to JNTUA, Ananthapuramu
(NH-44, Dupadu, Kurnool-518218)



II Year B.Tech. CSE (AI) – I Semester

L	T	P	C
3	0	0	3

(R233PC31) ARTIFICIAL INTELLIGENCE

Pre-requisite:

- Knowledge in Computer Programming.
- A course on “Mathematical Foundations of Computer Science”.
- Background in linear algebra, data structures and algorithms, and probability.

Course Objectives:

- The student should be made to study the concepts of Artificial Intelligence.
- The student should be made to learn the methods of solving problems using Artificial Intelligence.
- The student should be made to introduce the concepts of Expert Systems.
- To understand the applications of AI, namely game playing, theorem proving, and machine learning.
- To learn different knowledge representation techniques

Course Outcomes:

- Explain the concepts of AI and Intelligent agents.
- Apply the various search strategies in problem solving.
- Examine the effectiveness of truths by knowledge representation methods in AI.
- Analyze the logical agents to do inference using first order logic.
- Understand the basics of an expert system.

UNIT – I

Introduction: AI problems, foundation of AI and history of AI intelligent agents: Agents and Environments, the concept of rationality, the nature of environments, structure of agents, problem solving agents, problem formulation.

UNIT - II

Searching- Searching for solutions, uniformed search strategies – Breadth first search, depth first Search. Search with partial information (Heuristic search) Hill climbing, A*, AO* Algorithms, Problem reduction, Game Playing-Adversarial search, Games, mini-max algorithm, optimal decisions in multiplayer games, Problem in Game playing, Alpha-Beta pruning, Evaluation functions.

UNIT - III

Representation of Knowledge: Knowledge representation issues, predicate logic-logic programming, semantic nets- frames and inheritance, constraint propagation, representing knowledge using rules, rules based deduction systems. Reasoning under uncertainty, review of probability, Bayes' probabilistic interferences and dempster-shafer theory.

UNIT - IV



ASHOKA WOMEN'S ENGINEERING COLLEGE (Autonomous)

Accredited by NBA (CSE, CIVIL, ECE, EEE), Accredited by NAAC - A⁺
Approved by AICTE & Affiliated to JNTUA, Ananthapuramu
(NH-44, Dupadu, Kurnool-518218)



Logic concepts: First order logic. Inference in first order logic, propositional vs. first order inference, unification & lifts forward chaining, Backward chaining, Resolution, Learning from observation Inductive learning, Decision trees, Explanation based learning, Statistical Learning methods, Reinforcement Learning.

UNIT - V

Expert Systems: Architecture of expert systems, Roles of expert systems – Knowledge Acquisition Meta knowledge Heuristics. Typical expert systems – MYCIN, DART, XCON: Expert systems shell, Hardware based AI-GPU, TPU, NPU.

Textbooks:

1. S. Russel and P. Norvig, “Artificial Intelligence – A Modern Approach”, Second Edition, Pearson Education.
2. Kevin Night and Elaine Rich, Nair B., “Artificial Intelligence (SIE)”, Mc Graw Hill

Reference Books:

1. David Poole, Alan Mackworth, Randy Goebel, “Computational Intelligence: a logical approach”, Oxford University Press.
2. G. Luger, “Artificial Intelligence: Structures and Strategies for complex Problem solving”, Fourth Edition, Pearson Education.
3. J. Nilsson, “Artificial Intelligence: A new Synthesis”, Elsevier Publishers.
4. Artificial Intelligence, Saroj Kaushik, CENGAGE Learning.

Online Learning Resources:

1. <https://ai.google/>
2. https://swayam.gov.in/nd1_noc19_me71/preview
3. https://onlinecourses.nptel.ac.in/noc22_cs56/preview



ASHOKA WOMEN'S ENGINEERING COLLEGE

(Autonomous)

Accredited by NBA (CSE, CIVIL, ECE, EEE), Accredited by NAAC - A⁺

Approved by AICTE & Affiliated to JNTUA, Ananthapuramu
(NH-44, Dupadu, Kurnool-518218)



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AI)

Course Outcomes (COs)

Program Name:	B.TECH(CSE - AI)	AY	2025-26
Course Name:	ARTIFICIAL INTELLIGENCE	Class / Sem	II/I
Faculty Name:	Dr. T. Murali Krishna / Dr. K. Samunnisa	Regulation	R23

Course Outcomes

After completing this course, the student will be able to:

CO Number	CO Statement	Taxonomy ^{\$}
CO1	Explain the concepts of AI and Intelligent agents	Understand
CO2	Apply the various search strategies in problem solving	Apply
CO3	Summarize the effectiveness of truths by knowledge representation methods in AI	Understand
CO4	Analyze the logical agents to do inference using first order logic	Analyze
CO5	Understand the basics of an expert system	Understand

^{\$}: Remember, Understand, Apply, Analyze, Evaluate, Create

Signature of the Faculty



ASHOKA WOMEN'S ENGINEERING COLLEGE

(Autonomous)

Accredited by NBA (CSE, CIVIL, ECE, EEE), Accredited by NAAC - A⁺

Approved by AICTE & Affiliated to JNTUA, Ananthapuramu
(NH-44, Dupadu, Kurnool-518218)



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AI)

COURSE OUTCOME MAPPING WITH POS

Program Name:	B.TECH (CSE - AI)	AY	2025-26
Course Name:	ARTIFICIAL INTELLIGENCE	Class / Sem	II/I
Faculty Name:	Dr. K. Samunnisa	Regulation	R23

COURSE OUTCOMES (COs):

Upon completion of the course, students will be able to:

CO#	Course Outcomes
CO1	Explain the concepts of AI and Intelligent agents
CO2	Apply the various search strategies in problem solving
CO3	Summarize the effectiveness of truths by knowledge representation methods in AI
CO4	Analyze the logical agents to do inference using first order logic
CO5	Understand the basics of an expert system

CO-PO/PSO MATRIX: (Level of Mapping- 3: High; 2: Moderate; 1-Low; -: Not mapped)

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2
CO1	2	1	1									3	3	1
CO2	1	1	3									3	3	2
CO3	3	3	3									2	3	1
CO4	3	3	3									2	3	1
CO5	3	1	1									2	3	1
Course	2.4	1.8	2.2									2.4	3	1.2

PO1	Engineering Knowledge	PO7	Environment & Sustainability
PO2	Problem Analysis	PO8	Ethics
PO3	Design / Development of Solutions	PO9	Individual & Team Work
PO4	Conduct Investigations of complex problems	PO10	Communication Skills
PO5	Modern Tool usage	PO11	Project Management & Finance
PO6	Engineer & Society	PO12	Life-long Learning



ASHOKA WOMEN'S ENGINEERING COLLEGE (Autonomous)

Accredited by NBA (CSE, CIVIL, ECE, EEE), Accredited by NAAC - A⁺
Approved by AS/P (PPT)E & Affiliated to JNTUA, Ananthapuramu
(NH-44, Dupadu, Kurnool-518218)



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AI)

LESSON PLAN

Program Name:	B.TECH (CSE - AI)	AY	2025-26
Course Name:	ARTIFICIAL INTELLIGENCE	Class / Sem	II/I
Faculty Name:	Dr. K. Samunnisa	Regulation	R23

L. No	Topic/ Sub Topic	Reference	Teaching method ^{##}
1.	Unit – I: (Introduction to AI)	T1(1-4)	BS
2.	AI Problem	T1(5-7)	S/P (PPT)
3.	Foundations of AI	T1(8-15)	S/P (PPT)
4.	History of AI	T1(16-27)	S/P (PPT)
5.	Intelligent Agents	T1(28-30)	ITM
6.	Agents and Environments	T1(34-35)	
7.	The concept of rationality	T1(36-39)	ITM
8.	The nature of Environments	T1(40-45)	S/P (PPT)
9.	The structure of Agents	T1(46-59)	S/P (PPT)
10.	Problem solving agents	T1(64-68)	ITM
11.	Problem formulation	T1(69-74)	ITM
12.	Unit-II (Searching)		
13.	Searching for solutions	T1(75-80)	S/P (PPT)
14.	Uninformed search strategies	T1(81-91)	ITM
15.	BFS , DFS		
16.	Heuristic Search (Search with partial information)	T1(103-108)	GD
17.	A* Algorithm	T2(73-77)	
18.	AO* Algorithm	T2(82-87)	CPS
19.	Problem Reduction		
20.	Game Playing – Adversial Search	T1(189-192)	ITM
21.	Mini-Max Algorithm	T1(193-194) T2(310-318)	
22.	Optimal decisions in multi-player games		S/P (PPT)
23.	Problem in Game Playing		Seminar
24.	Alpha-Beta Pruning	T1(195-201)	ITM
25.	Evaluation Functions		
26.	UNIT-III (Representation of Knowledge)		
27.	Knowledge representation issues	T2 (105-114)	ITM
28.	predicate logic- logic programming	T2(131-137)	ITM
29.	semantic nets- frames and inheritance	T2(251-257)	S/P (PPT)
30.	constraint propagation		CPS
31.	representing knowledge using rules	T2(171-172)	ASG
32.	Rules based deduction systems	T2(195-199)	Chart
33.	Reasoning under uncertainty	T2 (231-246)	S/P (PPT)
34.	Review of probability		S/P (PPT)
35.	Bayes’ probabilistic interferences		ITM
36.	Dempster-Shafer theory		
37.	UNIT-IV (Logic Concepts)		
38.	First-order logic	T2(131-141)	ITM



ASHOKA WOMEN'S ENGINEERING COLLEGE (Autonomous)

Accredited by NBA (CSE, CIVIL, ECE, EEE), Accredited by NAAC - A⁺
Approved by AS/P (PPT)E & Affiliated to JNTUA, Ananthapuramu
(NH-44, Dupadu, Kurnool-518218)



39.	Inference in first order logic		ITM
40.	Propositional Vs. First order inference		S/P (PPT)
41.	Unification & lifts		ASG
42.	Forward chaining	T2(143-163) T2(177-182)	ITM
43.	Backward chaining		ITM
44.	Resolution		ITM
45.	Learning from observation	T1(677-690)	ITM
46.	Inductive learning		ITM
47.	Decision trees & Explanation based learning	T1(718-720)	S/P (PPT)
48.	Statistical Learning methods	T1(740-751) T1(764-768)	S/P (PPT)
49.	Reinforcement Learning	T1(791-800)	S/P (PPT)
50.	Unit – V: Expert Systems		
51.	Architecture of expert systems	T2(547-552)	ITM
52.	Roles of expert systems		S/P (PPT)
53.	Knowledge Acquisition & Meta knowledge	T2(553-557)	ITM
54.	Heuristics in expert systems	T2 (555-564)	ITM
55.	Typical expert systems		S/P (PPT)
56.	MYCIN, DART, XCON: Expert systems shell		GD
57.	Hardware based AI-GPU, TPU, NPU.		S/P (PPT)

Teaching Methods:

C&T:-Chalk & Talk; S/P:-Slides/PPT; Videos; SEM: Seminar; Demo; CHART; ET/GL: Expert Talk/Guest Lecture; QUIZ; CPS: Class room problem solving; GD:-Group discussion; RTCS: Real time case studies; JAR:-Journal article review; PD:-Poster design; OL:-Online lecture/Google class room; Industrial Visit (IV), Assignment (ASG), Quiz/Puzzle (Q), Brain storming (BS), Think-Pair-Share (TPS), Certification(CERT), SIM: Simulation, P/G: Pledge/Greeting, Q/R: Quotes, references, LS: Literature Survey, RW: Report Writing, MM: Model making, PED: Professional/ethical dilemma, Coding, Activity/Event, FV: Filed Visit etc.

Textbooks:

1. S. Russel and P. Norvig, "Artificial Intelligence – A Modern Approach", Second Edition, Pearson Education.
2. Kevin Night and Elaine Rich, Nair B., "Artificial Intelligence (SIE)", Mc Graw Hill

Reference Books:

1. David Poole, Alan Mackworth, Randy Goebel, "Computational Intelligence: a logical approach", Oxford University Press.
2. G. Luger, "Artificial Intelligence: Structures and Strategies for complex problem solving", Fourth Edition, Pearson Education.
3. J. Nilsson, "Artificial Intelligence: A new Synthesis", Elsevier Publishers.
4. Artificial Intelligence, Saroj Kaushik, CENGAGE Learning.

Online Learning Resources:

1. <https://ai.google/>
2. https://swayam.gov.in/nd1_noc19_me71/preview
3. https://onlinecourses.nptel.ac.in/noc22_cs56/preview



ASHOKA WOMEN'S ENGINEERING COLLEGE (Autonomous)

Accredited by NBA (CSE, CIVIL, ECE, EEE), Accredited by NAAC - A⁺
Approved by AICTE & Affiliated to JNTUA, Ananthapuramu
(NH-44, Dupadu, Kurnool-518218)



Teaching Methods & Description

S.No	Teaching Method	Description
1	Problem-Based Learning (PBL)	Students work in groups to solve open-ended, real-world engineering problems, developing problem-solving, teamwork, and critical thinking skills under the guidance of a facilitator.
2	Project-Based Learning	Students design, plan, and execute engineering projects over an extended time, integrating theory with practical application and fostering skills in project management and innovation.
3	Inquiry-Based Learning	Students are guided to ask questions, explore, and investigate engineering challenges, encouraging curiosity, independent learning, and deeper conceptual understanding.
4	Case-Based Learning	Real or hypothetical engineering case studies are analyzed, promoting the application of engineering concepts, ethical reasoning, and decision-making skills.
5	Flipped Classroom	Traditional lectures are replaced with pre-class study materials (videos, readings), and classroom time is dedicated to discussion, problem-solving, and hands-on activities.
6	Peer Learning	Students discuss conceptual questions with their peers during class, enhancing engagement, conceptual understanding, and communication skills.
7	Experiential Learning	Learning by doing through labs, internships, industrial visits, or fieldwork, linking theoretical knowledge with real-world experience.
8	Collaborative Learning	Students work in teams to achieve common learning objectives, fostering communication, leadership, and team-working abilities.
9	Cooperative Learning	Structured group work where each student is accountable for their role, encouraging mutual support and positive interdependence.
10	Gamification	Using game elements (points, leaderboards, challenges) to increase student motivation, engagement, and active participation in engineering education.
11	Simulation-Based Learning	Computer simulations or virtual labs are used to model complex engineering systems, allowing students to experiment and observe outcomes safely and efficiently.

12	Design Thinking	An iterative, user-centered approach where students empathize with users, define problems, ideate, prototype, and test solutions creatively.
13	Role Play	Students act out roles relevant to engineering projects or professional practice, strengthening interpersonal skills, negotiation, and stakeholder management.
14	Concept Mapping	Students create diagrams showing relationships among engineering concepts, supporting knowledge organization and critical thinking.
15	Mind Mapping	Visual tool where students branch out ideas from a central concept, promoting brainstorming and synthesis of complex engineering topics.
16	Blended Learning	A hybrid approach combining face-to-face teaching with online components to maximize flexibility and learner engagement.
17	Service Learning	Students apply engineering skills to community projects, combining civic responsibility with technical knowledge.
18	Think-Pair-Share	Students think individually about a question, discuss with a partner, and then share with the class, encouraging participation and critical reflection.
19	Socratic Questioning	Teachers use guided, open-ended questions to lead students to discover answers and challenge their assumptions, building analytical skills.
20	Team-Based Learning	Carefully structured teams tackle significant engineering tasks, promoting deep learning, accountability, and peer support.
21	Just-in-Time Teaching (JiTT)	Pre-class online quizzes inform instructors about student misconceptions, allowing targeted, adaptive in-class teaching.
22	Learning-by-Teaching	Students prepare and teach a concept to their peers, reinforcing their own mastery and communication skills.
23	Analogical Reasoning	Students draw analogies between familiar concepts and engineering problems, aiding in creative problem-solving and transfer of knowledge.
24	Challenge-Based Learning	Students work on meaningful societal or industrial challenges, developing entrepreneurial thinking and design skills.
25	Heutagogy	Self-determined learning where students plan their own learning pathways, fostering lifelong learning and adaptability.
26	Immersive Field Experiences	Real-life experiences in industry, construction sites, or research centers, providing context-rich, authentic learning opportunities.
27	Reflection-Based Learning	Students regularly reflect on their experiences, progress, and mistakes, deepening their metacognitive skills and self-awareness.
28	Rapid Prototyping	Students quickly build and test physical or digital prototypes, encouraging iterative design and resilience.
29	Discussion-Based Learning	Class discussions replace one-way lectures, building a deeper conceptual framework through shared dialogue.

30	MOOC-Integrated Learning	Integrating massive open online courses into the engineering curriculum to provide access to cutting-edge knowledge and skills.
31	Cross-Disciplinary Learning	Integrating other disciplines (e.g., business, humanities) with engineering, broadening students' problem-solving and design perspectives.
32	Scaffolded Learning	Breaking complex tasks into manageable steps with guidance and gradually reducing support as competence increases.
33	Industry Mentorship (Invited Guest Talks)	Professionals mentor students, providing career-relevant skills, motivation, and practical advice beyond academic curriculum.
34	Digital Storytelling	Students use multimedia tools to narrate engineering processes or projects, enhancing creativity and technical communication.





ASHOKA WOMEN'S ENGINEERING COLLEGE

(Autonomous)

Accredited by NBA (CSE, CIVIL, ECE, EEE), Accredited by NAAC - A⁺
Approved by AICTE & Affiliated to JNTUA, Ananthapuramu
(NH-44, Dupadu, Kurnool-518218)



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AI)

APIC-INNOVATIVE TEACHING METHODOLOGIES

Program Name:	B.Tech - CSE (AI)	AY	2025-2026
Course Name:	Artificial Intelligence	Class / Sem	II/I
Faculty Name:	Dr. K. Samunnisa	Regulation	R23

S.No.	Topic	Innovative Teaching Method	Real Time Application [Min:2]
1	Introduction to AI problems	Lecture + Brainstorming	Google Maps / GPS Navigation (A*) Virtual Assistants (e.g., Siri, Alexa) Robotics (Warehouse Automation, Drones)→ Path Planning
2	Intelligent agents and environments	Role Play	Robotic Vacuum Cleaners Self-Driving Cars AI in Medical Diagnosis
3	Concept of rationality	Think-Pair-Share with Ethical Dilemmas	Google Maps / GPS Navigation Systems Rationality in Action: (Suggests alternative routes based on traffic data, time of day, and user preferences)
4	Problem-solving agents	Flipped Classroom	Game Playing (Chess, Go...)
5	Problem formulation	Quiz-based Learning using Kahoot	Medical Diagnosis AI
6	Uninformed search strategies – BFS, DFS	Flipped Classroom Model	Social Network Friend Recommendations (BFS) Use: Finding people within 2 or 3 degrees of connection (e.g., “friends of friends”). Example: Facebook suggests mutual friends by traversing your social graph level-by-level.



ASHOKA WOMEN'S ENGINEERING COLLEGE (Autonomous)

Accredited by NBA (CSE, CIVIL, ECE, EEE), Accredited by NAAC - A⁺
Approved by AICTE & Affiliated to JNTUA, Ananthapuramu
(NH-44, Dupadu, Kurnool-518218)



7	Heuristic search – Hill Climbing, A*, AO*	Group Discussion	Hill Climbing: Navigating mountain terrain without a map. (CASE BASED LEARNING)
8	Game playing – Minimax algorithm	Game Play	Tic-Tac-Toe Tournament with Minimax Roles
9	Optimal decisions in multiplayer games	LECTURE + Think-Pair-Share	Chess Game Playing Context: AI vs human or AI vs AI games.
10	Alpha-beta pruning	Role Play	Game-Playing AI (Chess, Checkers, Tic-Tac-Toe) Use: Optimize decision-making in turn-based, two-player games. Example: Chess engines like <i>Stockfish</i> or <i>GNU Chess</i> . How: Prunes unnecessary moves while searching the game tree using Minimax
11	Knowledge representation issues	Live Coding with Prolog	Search Engines (e.g., Google Knowledge Graph) For eg: APPLE (a fruit or company)
12	Predicate logic & logic programming	Flipped Classroom	Expert Systems (e.g., MYCIN, DENDRAL)
13	Constraint propagation	Gamified Constraint Propagation Race / Class Room Problem solving	Solving crypt-arithmetic problem, Sudoku, map colouring.
14	Bayes' probabilistic inference & Dempster-Shafer theory	Quiz based learning using Kahoot or Mentimeter	Medical Diagnosis Use: Estimating the probability of a disease given symptoms and test results.
15	First-order logic concepts	Role-Playing with Quantifiers	Representing knowledge and rules in AI medical expert systems. Semantic parsing and question answering in NLP
16	Inference in first-order logic	Logic Detective Game (Method: Present a mystery or crime scenario with clues expressed in FOL.)	Business Rules Engines (Use: Automating workflows and decisions in CRMs or ERP systems.)



ASHOKA WOMEN'S ENGINEERING COLLEGE (Autonomous)

Accredited by NBA (CSE, CIVIL, ECE, EEE), Accredited by NAAC - A⁺
Approved by AICTE & Affiliated to JNTUA, Ananthapuramu
(NH-44, Dupadu, Kurnool-518218)



17	Forward & backward chaining	Inference Flowcharts (Method: Giving students a set of rules and facts, and ask them to draw) Forward chaining: arrows from facts to conclusions. Backward chaining: arrows from goal back to required facts.	Medical Diagnosis Systems
18	Resolution	Resolution Relay (Collaborative Learning) Method: Teams of students work in a relay: • First student picks two clauses to resolve. • Second student performs the resolution step. • Third writes the resulting clause. • Repeat until empty clause (goal) is reached.	Automated Theorem Proving
19	Inductive learning and decision trees	Think-Pair-Split: Think: Students analyze data independently. Pair: Discuss attribute selection and gain with a partner. Split: Decide on the first split in a tree and justify it.	E-commerce & Retail (Use: Recommend products or predict customer behavior.)
20	Architecture of expert systems	Gamified Learning (Design a quiz-based game (e.g., Kahoot or Quizizz))	Medical Diagnosis Systems (eg. MYCIN, IBM Watson Health)
21	Knowledge acquisition and meta-knowledge	Jigsaw Teaching Strategy Method: Divide the class into groups, each learning a specific aspect (e.g., tacit knowledge, meta-knowledge, knowledge validation). Activity: Re-group students to teach each other what they've learned.	Fraud Detection in Banking Chatbots and Virtual Assistants (e.g., Alexa, Siri)
22	Heuristics in expert systems	Role-Play (Method: Assign students real-life scenarios (e.g., diagnosing a disease, recommending a car). Activity: One student plays the user , others act as the expert system applying heuristic rules (e.g., "If fever and rash, then likely measles").	Aviation Systems and Pilot Assistance Heuristic Use: "If altitude drops quickly and airspeed is low, initiate stall recovery."



ASHOKA WOMEN'S ENGINEERING COLLEGE (Autonomous)

Accredited by NBA (CSE, CIVIL, ECE, EEE), Accredited by NAAC - A⁺
Approved by AICTE & Affiliated to JNTUA, Ananthapuramu
(NH-44, Dupadu, Kurnool-518218)



*** 75% of the topics should be delivered by innovative teaching methods**

CASE STUDY ON REAL WORLD PROBLEM RELATED TO THE SUBJECT

✚ Case Study 1 (Assignment 1): Rescue the Robot in MazeWorld”

✚ **Scenario:** A robot has accidentally entered a complex underground maze known as **MazeWorld**. It must **find the safest and shortest path** to the exit while avoiding blocked paths and minimizing energy usage.

✚ Design the robot’s navigation system using **search techniques**.

✚ Case Study 2 (Assignment 2): "AI Detective: Solving the Museum Heist with Logic"

Scenario: Design an **AI detective system** to help solve a **museum theft**. The system must reason through clues, witness statements, and object locations using **logical inference** to find out **who stole the ancient artifact**. (given Knowledge Base and Rules)

*** Split Class into 10 student groups**

MINI-PROJECT

Sudoku Solver using Constraint Satisfaction (Implement a Sudoku solver using backtracking and forward-checking algorithms)

***To be completed before the end of the semester**

INDUSTRIAL VISITS [If Any]

External References

1. <https://www.coursera.org/learn/ai-for-everyone>
2. <https://ocw.mit.edu/courses/6-034-artificial-intelligence-fall-2010/>
3. <https://www.edx.org/course/artificial-intelligence-ai>
4. <https://www.udacity.com/course/intro-to-artificial-intelligence--cs271>
5. <https://aima.cs.berkeley.edu/>
6. <https://nptel.ac.in/courses/106105079>
7. <https://www.youtube.com/watch?v=2ePf9rue1Ao>
8. <https://www.youtube.com/watch?v=JMUxmLyrhSk>



Classroom Conduct

- 🕒 Maintain silence and discipline to create a focused learning environment.
- 🕒 Be punctual — arrive on time and be prepared with required materials.
- 🗣️ ♀️ Participate actively in discussions, activities, and raise doubts respectfully.
- 📱 Use electronic devices (phones, laptops) only for academic purposes.
- 🤝 Respect faculty and peers by listening attentively and avoiding interruptions.
- Attendance percentage should be above 85% to appear for end semester exam.
- Mid exams absenteeism should be avoided and no retest will be conducted.



ASHOKA

Any grievances related to teaching, teaching methods and violation from lesson plan shall be reported to the mail

principal@ashokacollege.in, apic@ashokacollege.in

(or)

Share your real time feedback on the course through this link

<https://forms.gle/P11Sia4fSRQTFRxXA>

UNIT 1

INTRODUCTION TO ARTIFICIAL INTELLIGENCE

Introduction: AI problems, foundation of AI and history of AI intelligent agents: Agents and Environments, the concept of rationality, the nature of environments, structure of agents, problem solving agents, problem formulation.

INTRODUCTION

INTELLIGENCE	ARTIFICIAL INTELLIGENCE
It is a natural process.	It is programmed by humans.
It is actually hereditary.	It is not hereditary.
Knowledge is required for intelligence.	KB and electricity are required to generate output.
No human is an expert. We may get better solutions from other humans.	Expert systems are made which aggregate many person's experience and ideas.

1.1 DEFINITION

The study of how to make computers do things at which at the moment, people are better.
“Artificial Intelligence is the ability of a computer to act like a human being”.

- Systems that think like humans
- Systems that act like humans
- Systems that think rationally. Systems that act rationally.

“The exciting new effort to make computers think ... <i>machines with minds</i>, in the full and literal sense” (Haugeland, 1985) “[The automation of] activities that we associate with human thinking, activities such as decision-making, problem solving, learning ...” (Bellman, 1978)	“The study of mental faculties through the use of computational models” (Charniak and McDermott, 1985) “The study of the computations that make it possible to perceive, reason, and act” (Winston, 1992)
“The art of creating machines that perform functions that require intelligence when performed by people” (Kurzweil, 1990) “The study of how to make computers do things at which, at the moment, people are better” (Rich and Knight, 1991)	“A field of study that seeks to explain and emulate intelligent behavior in terms of computational processes” (Schalkoff, 1990) “The branch of computer science that is concerned with the automation of intelligent behavior” (Luger and Stubblefield, 1993)

Figure 1.1 Some definitions of artificial intelligence, organized into four categories

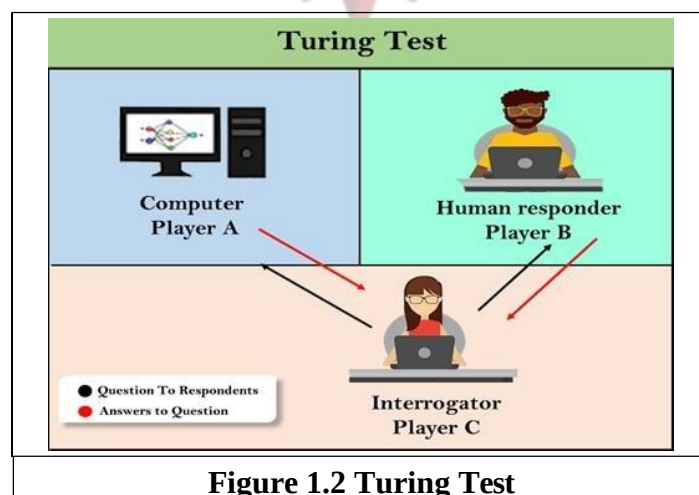
- (a) **Intelligence** - Ability to apply knowledge in order to perform better in an environment.
- (b) **Artificial Intelligence** - Study and construction of agent programs that perform well in a given environment, for a given agent architecture.
- (c) **Agent** - An entity that takes action in response to precepts from an environment.
- (d) **Rationality** - property of a system which does the “right thing” given what it knows.
- (e) **Logical Reasoning** - A process of deriving new sentences from old, such that the new sentences are necessarily true if the old ones are true.

Four Approaches of Artificial Intelligence:

- Acting humanly: The Turing test approach.
- Thinking humanly: The cognitive modelling approach.
- Thinking rationally: The laws of thought approach.
- Acting rationally: The rational agent approach.

1.2 ACTING HUMANLY: THE TURING TEST APPROACH

The Turing Test, proposed by Alan Turing (1950), was designed to provide a satisfactory operational definition of intelligence. A computer passes the test if a human interrogator, after posing some written questions, cannot tell whether the written responses come from a person or from a computer.



- **natural language processing** to enable it to communicate successfully in English;
- **knowledge representation** to store what it knows or hears;
- **automated reasoning** to use the stored information to answer questions and to draw new conclusions
- **machine learning** to adapt to new circumstances and to detect and extrapolate patterns.

Total Turing Test includes a video signal so that the interrogator can test the subject's perceptual abilities, as well as the opportunity for the interrogator to pass physical objects "through the hatch." To pass the total Turing Test, the computer will need

- **computer vision** to perceive objects, and **robotics** to manipulate objects and move about.

Thinking humanly: The cognitive modelling approach

Analyse how a given program thinks like a human, we must have some way of determining how humans think. The interdisciplinary field of **cognitive science** brings together computer models from AI and experimental techniques from psychology to try to construct precise and testable theories of the workings of the human mind.

Although cognitive science is a fascinating field in itself, we are not going to be discussing it all that much in this book. We will occasionally comment on similarities or differences between AI techniques and human cognition. Real cognitive science, however, is necessarily based on experimental investigation of actual humans or animals, and we assume that the reader only has access to a computer for experimentation. We will simply note that AI and cognitive science continue to fertilize each other, especially in the areas of vision, natural language, and learning.

Thinking rationally: The "laws of thought" approach

The Greek philosopher Aristotle was one of the first to attempt to codify "right thinking," that is, irrefutable reasoning processes. His famous syllogisms provided patterns for argument structures that always gave correct conclusions given correct premises.

For example, "Socrates is a man; all men are mortal; therefore Socrates is mortal."

These laws of thought were supposed to govern the operation of the mind, and initiated the field of **logic**.

Acting rationally: The rational agent approach

Acting rationally means acting so as to achieve one's goals, given one's beliefs. An agent is just something that perceives and acts.

The right thing: that which is expected to maximize goal achievement, given the available information

Does not necessary involve thinking.

For Example - blinking reflex- but should be in the service of rational action.

1.3 FUTURE OF ARTIFICIAL INTELLIGENCE

- **Transportation:** Although it could take a decade or more to perfect them, autonomous cars will one day ferry us from place to place.
- **Manufacturing:** AI powered robots work alongside humans to perform a limited range of tasks like assembly and stacking, and predictive analysis sensors keep equipment running smoothly.
- **Healthcare:** In the comparatively AI-nascent field of healthcare, diseases are more quickly and accurately diagnosed, drug discovery is sped up and streamlined, virtual nursing assistants monitor patients and big data analysis helps to create a more personalized patient experience.
- **Education:** Textbooks are digitized with the help of AI, early-stage virtual tutors assist human instructors and facial analysis gauges the emotions of students to help determine who's struggling or bored and better tailor the experience to their individual needs.
- **Media:** Journalism is harnessing AI, too, and will continue to benefit from it. Bloomberg uses Cyborg technology to help make quick sense of complex financial reports. The Associated Press employs the natural language abilities of Automated Insights to produce 3,700 earning reports stories per year — nearly four times more than in the recent past
- **Customer Service:** Last but hardly least, Google is working on an AI assistant that can place human-like calls to make appointments at, say, your neighborhood hair salon. In addition to words, the system understands context and nuance.

1.4 CHARACTERISTICS OF INTELLIGENT AGENTS

Situatedness

The agent receives some form of sensory input from its environment, and it performs some action that changes its environment in some way.

Examples of environments: the physical world and the Internet.

- Autonomy

The agent can act without direct intervention by humans or other agents and that it has control over its own actions and internal state.

- Adaptivity

The agent is capable of

- (1) reacting flexibly to changes in its environment;
- (2) taking goal-directed initiative (i.e., is pro-active), when appropriate; and
- (3) Learning from its own experience, its environment, and interactions with others.

- Sociability

The agent is capable of interacting in a peer-to-peer manner with other agents or humans

1.5 AGENTS AND ITS TYPES

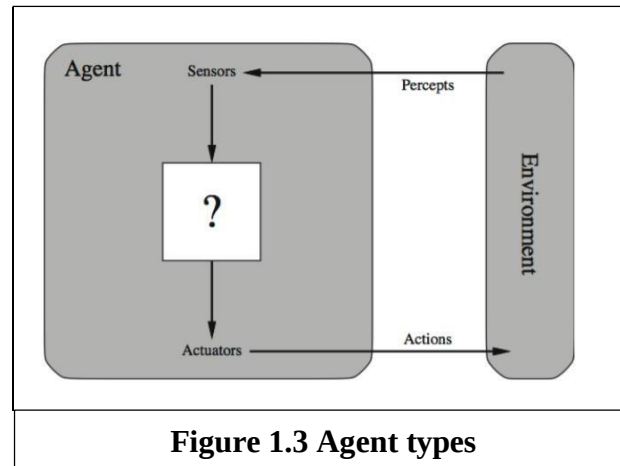


Figure 1.3 Agent types

An agent is anything that can be viewed as perceiving its environment through sensors and acting upon that environment through actuators.

- Human Sensors:
- Eyes, ears, and other organs for sensors.
- Human Actuators:
- Hands, legs, mouth, and other body parts.
- Robotic Sensors:
- Mic, cameras and infrared range finders for sensors
- Robotic Actuators:
- Motors, Display, speakers etc An agent can be:

Human-Agent: A human agent has eyes, ears, and other organs which work for sensors and hand, legs, vocal tract work for actuators.

Robotic Agent: A robotic agent can have cameras, infrared range finder, NLP for sensors and various motors for actuators.

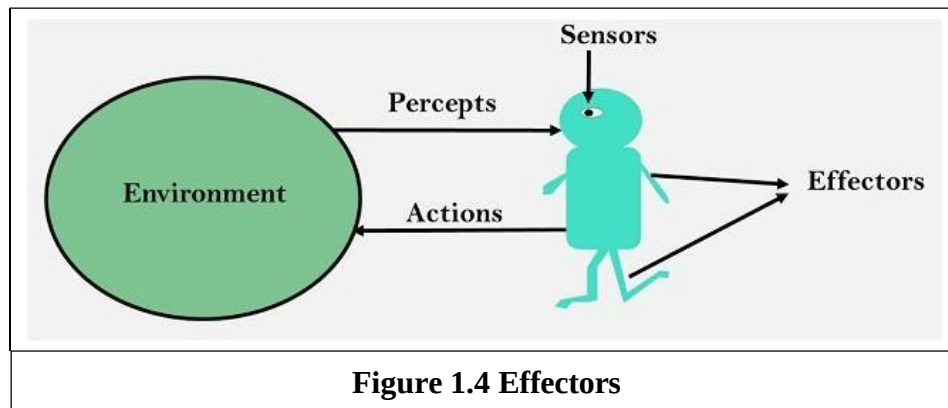
Software Agent: Software agent can have keystrokes, file contents as sensory input and act on those inputs and display output on the screen.

Hence the world around us is full of agents such as thermostat, cell phone, camera, and even we are also agents. Before moving forward, we should first know about sensors, effectors, and actuators.

Sensor: Sensor is a device which detects the change in the environment and sends the information to other electronic devices. An agent observes its environment through sensors.

Actuators: Actuators are the component of machines that converts energy into motion. The actuators are only responsible for moving and controlling a system. An actuator can be an electric motor, gears, rails, etc.

Effectors: Effectors are the devices which affect the environment. Effectors can be legs, wheels, arms, fingers, wings, fins, and display screen.



1.6 PROPERTIES OF ENVIRONMENT

An **environment** is everything in the world which surrounds the agent, but it is not a part of an agent itself. An environment can be described as a situation in which an agent is present.

The environment is where agent lives, operate and provide the agent with something to sense and act upon it.

Fully observable vs Partially Observable:

If an agent sensor can sense or access the complete state of an environment at each point of time then it is a **fully observable** environment, else it is **partially observable**.

A fully observable environment is easy as there is no need to maintain the internal state to keep track history of the world.

An agent with no sensors in all environments then such an environment is called as unobservable.

Example: chess – the board is fully observable, as are opponent's moves. Driving – what is around the next bend is not observable and hence partially observable.

1. Deterministic vs Stochastic

- If an agent's current state and selected action can completely determine the next state of the environment, then such environment is called a deterministic environment.

- A stochastic environment is random in nature and cannot be determined completely by an agent.
- In a deterministic, fully observable environment, agent does not need to worry about uncertainty.

2 **Episodic vs Sequential**

- In an episodic environment, there is a series of one-shot actions, and only the current percept is required for the action.
- However, in Sequential environment, an agent requires memory of past actions to determine the next best actions.

3 **Single-agent vs Multi-agent**

- If only one agent is involved in an environment, and operating by itself then such an environment is called single agent environment.
- However, if multiple agents are operating in an environment, then such an environment is called a multi-agent environment.
- The agent design problems in the multi-agent environment are different from single agent environment.

4 **Static vs Dynamic**

- If the environment can change itself while an agent is deliberating then such environment is called a dynamic environment else it is called a static environment.
- Static environments are easy to deal because an agent does not need to continue looking at the world while deciding for an action.
- However for dynamic environment, agents need to keep looking at the world at each action.
- Taxi driving is an example of a dynamic environment whereas Crossword puzzles are an example of a static environment.

5 **Discrete vs Continuous**

- If in an environment there are a finite number of precepts and actions that can be performed within it, then such an environment is called a discrete environment else it is called continuous environment.
- A chess game comes under discrete environment as there is a finite number of moves that can be performed.
- A self-driving car is an example of a continuous environment.

6. Known vs Unknown

- Known and unknown are not actually a feature of an environment, but it is an agent's state of knowledge to perform an action.
- In a known environment, the results for all actions are known to the agent. While in unknown environment, agent needs to learn how it works in order to perform an action.
- It is quite possible that a known environment to be partially observable and an Unknown environment to be fully observable.

7. Accessible vs. Inaccessible

- If an agent can obtain complete and accurate information about the state's environment, then such an environment is called an Accessible environment else it is called inaccessible.
- An empty room whose state can be defined by its temperature is an example of an accessible environment.
- Information about an event on earth is an example of Inaccessible environment.

Task environments, which are essentially the "problems" to which rational agents are the "solutions."

PEAS: Performance Measure, Environment, Actuators, Sensors

Performance

The output which we get from the agent. All the necessary results that an agent gives after processing comes under its performance.

Environment

All the surrounding things and conditions of an agent fall in this section. It basically consists of all the things under which the agents work.

Actuators

The devices, hardware or software through which the agent performs any actions or processes any information to produce a result are the actuators of the agent.

Sensors

The devices through which the agent observes and perceives its environment are the sensors of the agent.

Agent Type	Performance Measure	Environment	Actuators	Sensors
Medical diagnosis system	Healthy patient, reduced costs	Patient, hospital, staff	Display of questions, tests, diagnoses, treatments, referrals	Keyboard entry of symptoms, findings, patient's answers
Satellite image analysis system	Correct image categorization	Downlink from orbiting satellite	Display of scene categorization	Color pixel arrays
Part-picking robot	Percentage of parts in correct bins	Conveyor belt with parts; bins	Jointed arm and hand	Camera, joint angle sensors
Refinery controller	Purity, yield, safety	Refinery, operators	Valves, pumps, heaters, displays	Temperature, pressure, chemical sensors
Interactive English tutor	Student's score on test	Set of students, testing agency	Display of exercises, suggestions, corrections	Keyboard entry

Figure 1.5 Examples of agent types and their PEAS descriptions

Rational Agent - A system is rational if it does the “right thing”. Given what it knows.

Characteristic of Rational Agent

- The agent's prior knowledge of the environment.
- The performance measure that defines the criterion of success.
- The actions that the agent can perform.
- The agent's percept sequence to date.

For every possible percept sequence, a rational agent should select an action that is expected to maximize its performance measure, given the evidence provided by the percept sequence and whatever built-in knowledge the agent has.

- An **omniscient agent** knows the actual outcome of its actions and can act accordingly; but omniscience is impossible in reality.
- **Ideal Rational Agent** precepts and does things. It has a greater performance measure. Eg. Crossing road. Here first perception occurs on both sides and then only action. No perception occurs in **Degenerate Agent**.
Eg. Clock. It does not view the surroundings. No matter what happens outside. The clock works based on inbuilt program.
- **Ideal Agent** describes by ideal mappings. “Specifying which action an agent ought to take in response to any given percept sequence provides a design for ideal agent”.

- **Eg.** SQRT function calculation in calculator.
- Doing actions in order to modify future precepts-sometimes called **information gathering**- is an important part of rationality.
- A rational agent should be **autonomous**-it should learn from its own prior knowledge (experience).

The Structure of Intelligent Agents

$$\text{Agent} = \text{Architecture} + \text{Agent Program}$$

Architecture = the machinery that an agent executes on. (Hardware)

Agent Program = an implementation of an agent function. (Algorithm, Logic – Software)

1.7 TYPES OF AGENTS

Agents can be grouped into four classes based on their degree of perceived intelligence and capability :

- Simple Reflex Agents
- Model-Based Reflex Agents
- Goal-Based Agents
- Utility-Based Agents
- Learning Agent



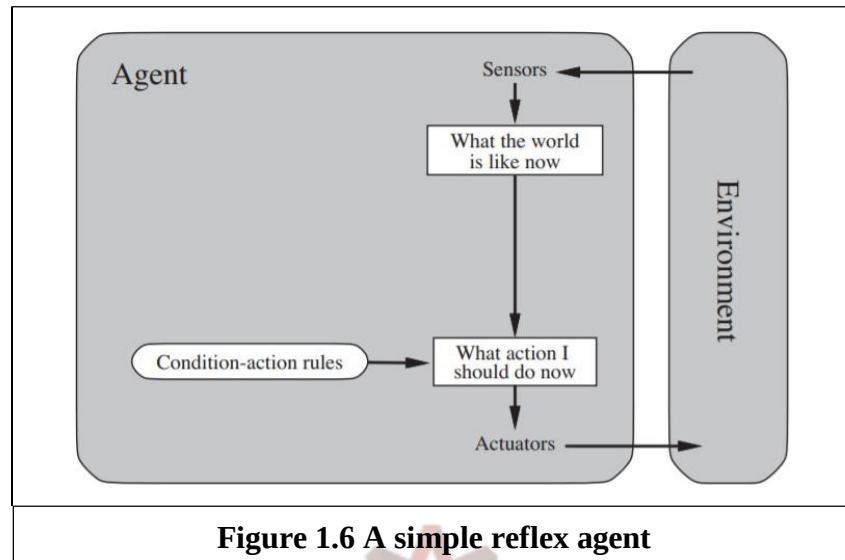
The Simple reflex agents

- The Simple reflex agents are the simplest agents. These agents take decisions on the basis of the current percepts and ignore the rest of the percept history (**past State**).
- These agents only succeed in the fully observable environment.
- The Simple reflex agent does not consider any part of percepts history during their decision and action process.
- The Simple reflex agent works on Condition-action rule, which means it maps the current state to action. Such as a Room Cleaner agent, it works only if there is dirt in the room.
- Problems for the simple reflex agent design approach:
 - o They have very limited intelligence
 - o They do not have knowledge of non-perceptual parts of the current state

- o Mostly too big to generate and to store.
- o Not adaptive to changes in the environment.

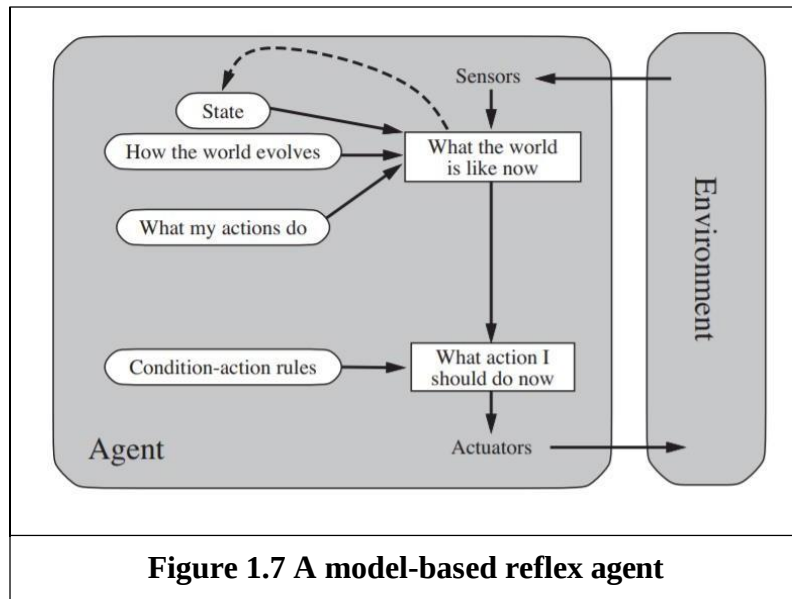
Condition-Action Rule – It is a rule that maps a state (condition) to an action.

Ex: if car-in-front-is-braking then initiate- braking.



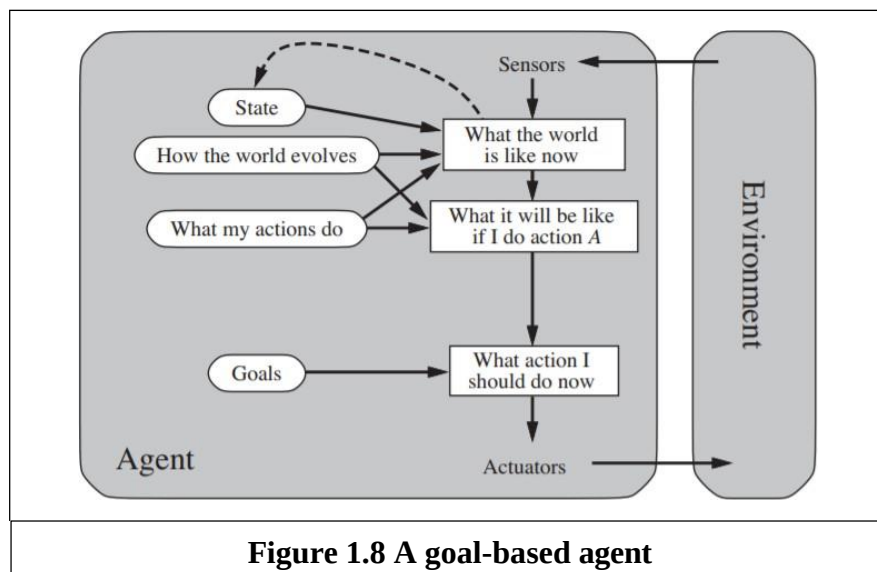
Model Based Reflex Agents

- The Model-based agent can work in a partially observable environment, and track the situation.
- A model-based agent has two important factors:
 - o **Model:** It is knowledge about "how things happen in the world," so it is called a Model-based agent.
 - o **Internal State:** It is a representation of the current state based on percept history.
- These agents have the model, "which is knowledge of the world" and based on the model they perform actions.
- Updating the agent state requires information about:
 - o How the world evolves
 - o How the agent's action affects the world.



Goal Based Agents

- The knowledge of the current state environment is not always sufficient to decide for an agent to what to do.
- The agent needs to know its goal which describes desirable situations.
- Goal-based agents expand the capabilities of the model-based agent by having the "goal" information.
- They choose an action, so that they can achieve the goal.
- These agents may have to consider a long sequence of possible actions before deciding whether the goal is achieved or not. Such considerations of different scenario are called searching and planning, which makes an agent proactive.



Utility Based Agents

- These agents are similar to the goal-based agent but provide an extra component of utility measurement (“**Level of Happiness**”) which makes them different by providing a measure of success at a given state.
- Utility-based agent act based not only goals but also the best way to achieve the goal.
- The Utility-based agent is useful when there are multiple possible alternatives, and an agent has to choose in order to perform the best action.
- The utility function maps each state to a real number to check how efficiently each action achieves the goals.

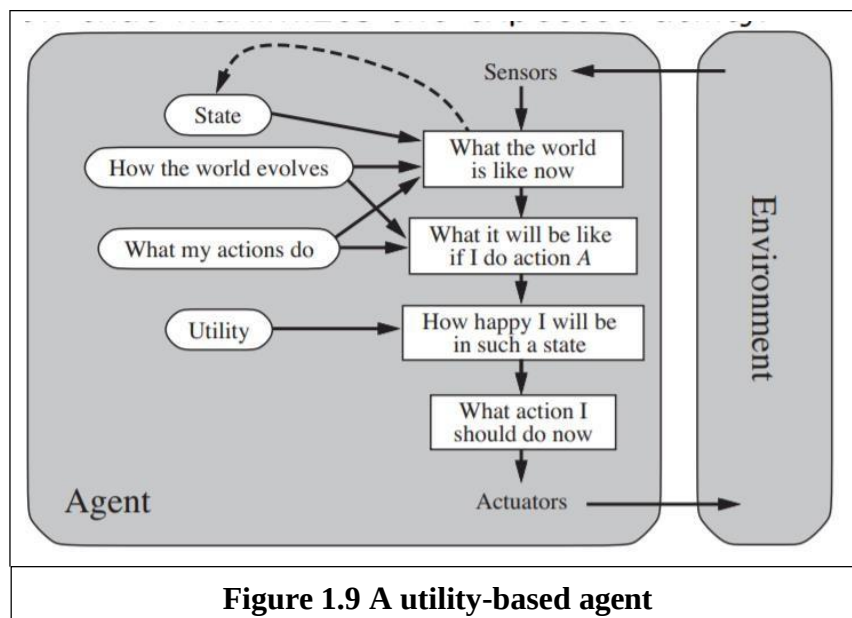
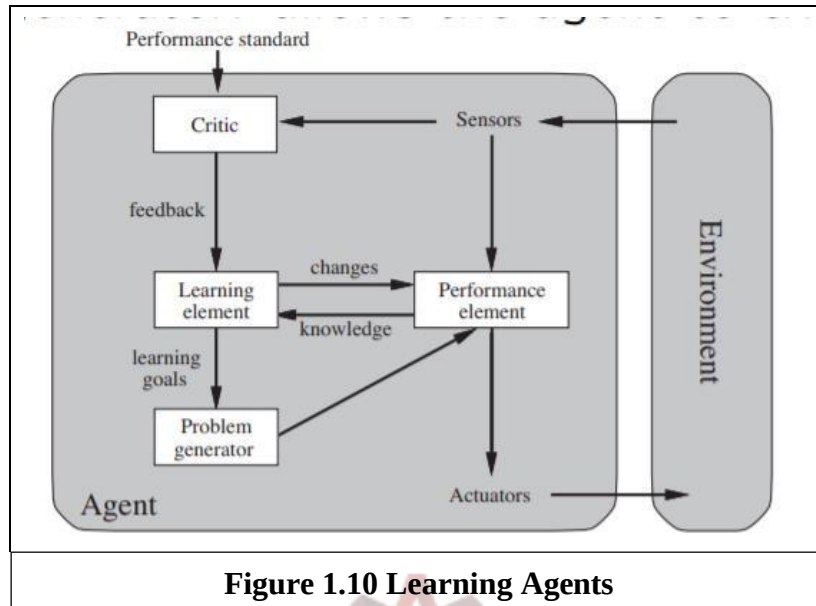


Figure 1.9 A utility-based agent

Learning Agents

- A learning agent in AI is the type of agent which can learn from its past experiences, or it has learning capabilities.
- It starts to act with basic knowledge and then able to act and adapt automatically through learning.
- A learning agent has mainly four conceptual components, which are:
 - a. **Learning element:** It is responsible for making improvements by learning from environment
 - b. **Critic:** Learning element takes feedback from critic which describes that how well the agent is doing with respect to a fixed performance standard.
 - c. **Performance element:** It is responsible for selecting external action

- d. **Problem generator:** This component is responsible for suggesting actions that will lead to new and informative experiences.
- Hence, learning agents are able to learn, analyze performance, and look for new ways to improve the performance.



1.8 PROBLEM SOLVING APPROACH TO TYPICAL AI PROBLEMS

Problem-solving agents

In Artificial Intelligence, Search techniques are universal problem-solving methods. **Rational agents** or **Problem-solving agents** in AI mostly used these search strategies or algorithms to solve a specific problem and provide the best result. Problem-solving agents are the goal-based agents and use atomic representation. In this topic, we will learn various problem-solving search algorithms.

- Some of the most popularly used problem solving with the help of artificial intelligence are:
 1. Chess.
 2. Travelling Salesman Problem.
 3. Tower of Hanoi Problem.
 4. Water-Jug Problem.
 5. N-Queen Problem.

Problem Searching

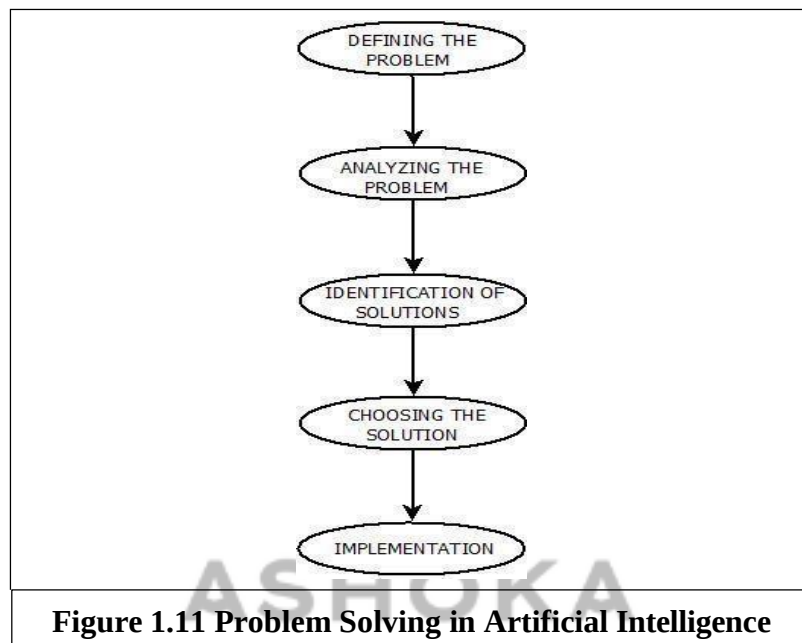
- In general, searching refers to as finding information one needs.

- Searching is the most commonly used technique of problem solving in artificial intelligence.
- The searching algorithm helps us to search for solution of particular problem.

Problem: Problems are the issues which comes across any system. A solution is needed to solve that particular problem.

Steps : Solve Problem Using Artificial Intelligence

- The process of solving a problem consists of five steps. These are:



Defining The Problem: The definition of the problem must be included precisely. It should contain the possible initial as well as final situations which should result in acceptable solution.

1. **Analyzing The Problem:** Analyzing the problem and its requirement must be done as few features can have immense impact on the resulting solution.
2. **Identification Of Solutions:** This phase generates reasonable amount of solutions to the given problem in a particular range.
3. **Choosing a Solution:** From all the identified solutions, the best solution is chosen basis on the results produced by respective solutions.
4. **Implementation:** After choosing the best solution, its implementation is done.

Measuring problem-solving performance

We can evaluate an algorithm's performance in four ways:

Completeness: Is the algorithm guaranteed to find a solution when there is one?

Optimality: Does the strategy find the optimal solution?

Time complexity: How long does it take to find a solution?

Space complexity: How much memory is needed to perform the search?

Search Algorithm Terminologies

- Search: Searching is a step by step procedure to solve a search-problem in a given search space. A search problem can have three main factors:
 1. Search Space: Search space represents a set of possible solutions, which a system may have.
 2. Start State: It is a state from where agent begins the search.
 3. Goal test: It is a function which observe the current state and returns whether the goal state is achieved or not.
- Search tree: A tree representation of search problem is called Search tree. The root of the search tree is the root node which is corresponding to the initial state.
- Actions: It gives the description of all the available actions to the agent.
- Transition model: A description of what each action do, can be represented as a transition model.
- Path Cost: It is a function which assigns a numeric cost to each path.
- Solution: It is an action sequence which leads from the start node to the goal node.
Optimal Solution: If a solution has the lowest cost among all solutions.

Example Problems

A **Toy Problem** is intended to illustrate or exercise various problem-solving methods. **Areal- world problem** is one whose solutions people actually care about.

Toy Problems

Vacuum World

States: The state is determined by both the agent location and the dirt locations. The agent is in one of the 2 locations, each of which might or might not contain dirt. Thus there are $2 \times 2^2 = 8$ possible world states.

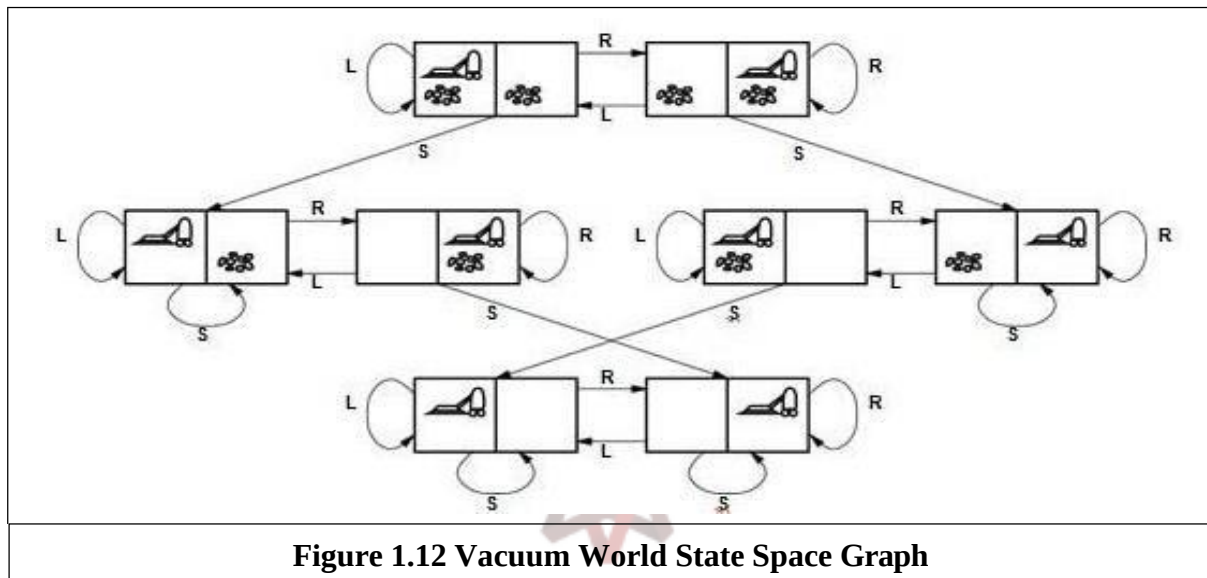
Initial state: Any state can be designated as the initial state.

Actions: In this simple environment, each state has just three actions: *Left*, *Right*, and *Suck*. Larger environments might also include *Up* and *Down*.

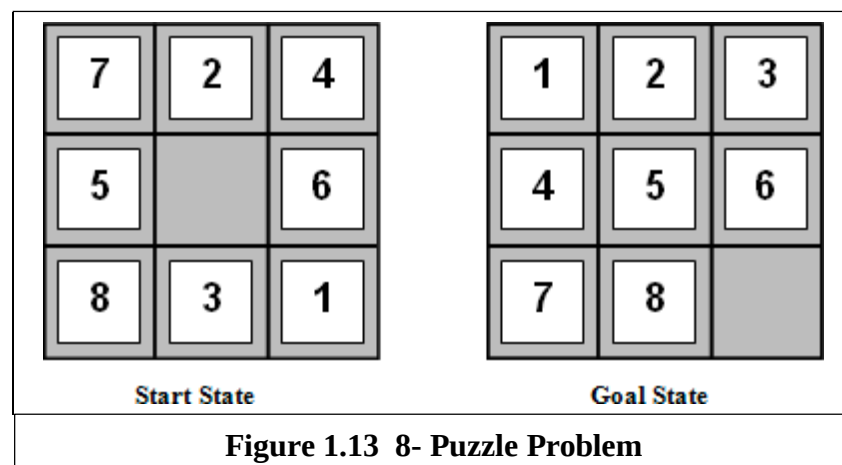
Transition model: The actions have their expected effects, except that moving *Left* in the leftmost square, moving *Right* in the rightmost square, and *Sucking* in a clean square have no effect. The complete state space is shown in Figure.

Goal test: This checks whether all the squares are clean.

Path cost: Each step costs 1, so the path cost is the number of steps in the path.



1) 8- Puzzle Problem



States: A state description specifies the location of each of the eight tiles and the blank in one of the nine squares.

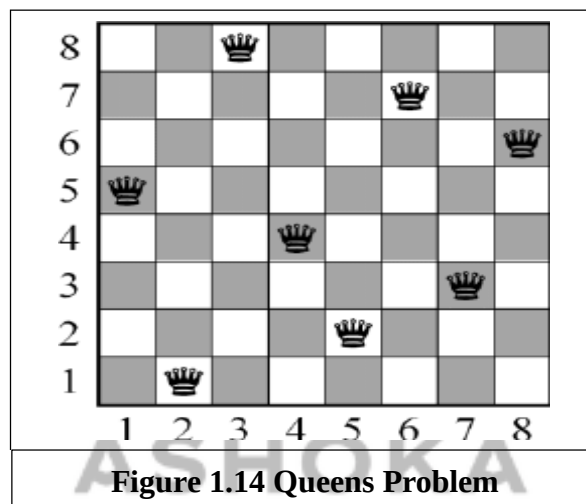
Initial state: Any state can be designated as the initial state. Note that any given goal can be reached from exactly half of the possible initial states.

The simplest formulation defines the actions as movements of the blank space *Left*, *Right*, *Up*, or *Down*. Different subsets of these are possible depending on where the blank is.

Transition model: Given a state and action, this returns the resulting state; for example, if we apply *Left* to the start state in Figure 3.4, the resulting state has the 5 and the blank switched.

Goal test: This checks whether the state matches the goal configuration shown in Figure. **Path cost:** Each step costs 1, so the path cost is the number of steps in the path.

Queens Problem



- **States:** Any arrangement of 0 to 8 queens on the board is a state.
- **Initial state:** No queens on the board.
- **Actions:** Add a queen to any empty square.
- **Transition model:** Returns the board with a queen added to the specified square.
- **Goal test:** 8 queens are on the board, none attacked.

Consider the given problem. Describe the operator involved in it. Consider the water jug problem: You are given two jugs, a 4-gallon one and 3-gallon one. Neither has any measuring marker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallon of water from the 4-gallon jug ?

Explicit Assumptions: A jug can be filled from the pump, water can be poured out of a jug on to the ground, water can be poured from one jug to another and that there are no other measuring devices available.

Here the initial state is (0, 0). The goal state is (2, n) for any value of n.

State Space Representation: we will represent a state of the problem as a tuple (x, y) where x represents the amount of water in the 4-gallon jug and y represents the amount of water in the 3-gallon jug. Note that $0 \leq x \leq 4$, and $0 \leq y \leq 3$.

To solve this we have to make some assumptions not mentioned in the problem. They are:

- We can fill a jug from the pump.
- We can pour water out of a jug to the ground.
- We can pour water from one jug to another.
- There is no measuring device available.

Operators - we must define a set of operators that will take us from one state to another.

Table 1.1

Sr.	Current State	Next State	Descriptions
1	(x,y) if $x < 4$	$(4,y)$	Fill the 4 gallon jug
2	(x,y) if $y < 3$	$(x,3)$	Fill the 3 gallon jug
3	(x,y) if $x > 0$	$(x - d, y)$	Pour some water out of the 4 gallon jug
4	(x,y) if $y > 0$	$(x, y - d)$	Pour some water out of the 3 gallon jug
5	(x,y) if $x > 0$	$(0, y)$	Empty the 4 gallon jug
6	(x,y) if $y > 0$	$(x, 0)$	Empty the 3 gallon jug on the ground
7	(x,y) if $x + y \geq 4$ and $y > 0$	$(4, y - (4 - x))$	Pour water from the 3 gallon jug into the 4 gallon jug until the 4 gallon jug is full
8	(x,y) if $x + y \geq 3$ and $x > 0$	$(x - (3 - x), 3)$	Pour water from the 4 gallon jug into the 3 gallon jug until the 3 gallon jug is full
9	(x,y) if $x + y \leq 4$ and $y > 0$	$(x + y, 0)$	Pour all the water from the 3 gallon jug into the 4 gallon jug
10	(x,y) if $x + y \leq 3$ and $x > 0$	$(0, x + y)$	Pour all the water from the 4 gallon jug into the 3 gallon jug
11	$(0, 2)$	$(2, 0)$	Pour the 2 gallons from 3 gallon jug into the 4 gallon jug
12	$(2, y)$	$(0, y)$	Empty the 2 gallons in the 4 gallon jug on the ground

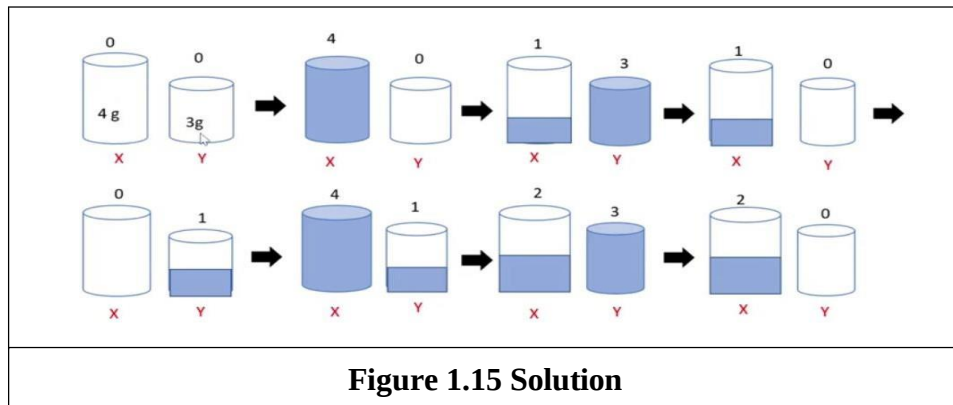
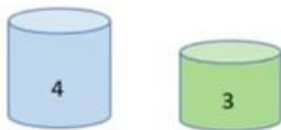


Table 1.2

Solution

S.No.	Gallons in 4-gel jug(x)	Gallons in 3-gel jug (y)	Rule Applied
1.	0	0	Initial state
2..	4	0	1. Fill 4
3	1	3	6. Pour 4 into 3 to fill
4.	1	0	4. Empty 3
5.	0	1	8. Pour all of 4 into 3
6.	4	1	1. Fill 4
7.	2	3	6. Pour 4 into 3

- 4-gallon one and a 3-gallon Jug



- No measuring mark on the jug.
- There is a pump to fill the jugs with water.
- How can you get exactly 2 gallon of water into the 4-gallon jug?

UNIT-II

Searching- Searching for solutions, uniformed search strategies – Breadth first search, depth first Search. Search with partial information (Heuristic search) Hill climbing, A* ,AO* Algorithms, Problem reduction, Game Playing-Adversarial search, Games, mini-max algorithm, optimal decisions in multiplayer games, Problem in Game playing, Alpha-Beta pruning, Evaluation functions.

2.1 PROBLEM SOLVING BY SEARCH

An important aspect of intelligence is **goal-based** problem solving.

The **solution** of many **problems** can be described by finding a **sequence of actions** that lead to a desirable **goal**. Each action changes the **state** and the aim is to find the sequence of actions and states that lead from the initial (start) state to a final (goal) state.

A well-defined problem can be described by:

□ Initial state

- **Operator or successor function** - for any state x returns $s(x)$, the set of states reachable from x with one action
- **State space** - all states reachable from initial by any sequence of actions
- **Path** - sequence through state space
- **Path cost** - function that assigns a cost to a path. Cost of a path is the sum of costs of individual actions along the path
- **Goal test** - test to determine if at goal state

What is Search?

Search is the systematic examination of **states** to find path from the **start/root state** to the **goal state**.

The set of possible states, together with *operators* defining their connectivity constitute the *search space*.

The output of a search algorithm is a solution, that is, a path from the initial state to a state that satisfies the goal test.

Problem-solving agents

A Problem solving agent is a **goal-based** agent. It decide what to do by finding sequence of actions that lead to desirable states. The agent can adopt a goal and aim at satisfying it.

To illustrate the agent's behavior, let us take an example where our agent is in the city of Arad, which is in Romania. The agent has to adopt a **goal** of getting to Bucharest.

Goal formulation, based on the current situation and the agent's performance measure, is the first step in problem solving.

The agent's task is to find out which sequence of actions will get to a goal state.

Problem formulation is the process of deciding what actions and states to consider given a goal.

Example: Route finding problem Referring to figure On holiday in Romania : currently in Arad. Flight leaves tomorrow from Bucharest Formulate goal: be in Bucharest Formulate problem: states: various cities actions: drive between cities Find solution: sequence of cities, e.g., Arad, Sibiu, Fagaras, Bucharest
Problem formulation
A problem is defined by four items: initial state e.g., "at Arad" successor function $S(x)$ = set of action-state pairs e.g., $S(\text{Arad}) = \{[\text{Arad} - > \text{Zerind}; \text{Zerind}], \dots\}$ goal test, can be explicit, e.g., $x = \text{at Bucharest}$" implicit, e.g., $\text{NoDirt}(x)$ path cost (additive) e.g., sum of distances, number of actions executed, etc. $c(x; a; y)$ is the step cost, assumed to be ≥ 0 A solution is a sequence of actions leading from the initial state to a goal state.
Goal formulation and problem formulation

2.2 EXAMPLE PROBLEMS

The problem solving approach has been applied to a vast array of task environments. Some best known problems are summarized below. They are distinguished as toy or real-world problems

A **toy problem** is intended to illustrate various problem solving methods. It can be easily used by different researchers to compare the performance of algorithms.

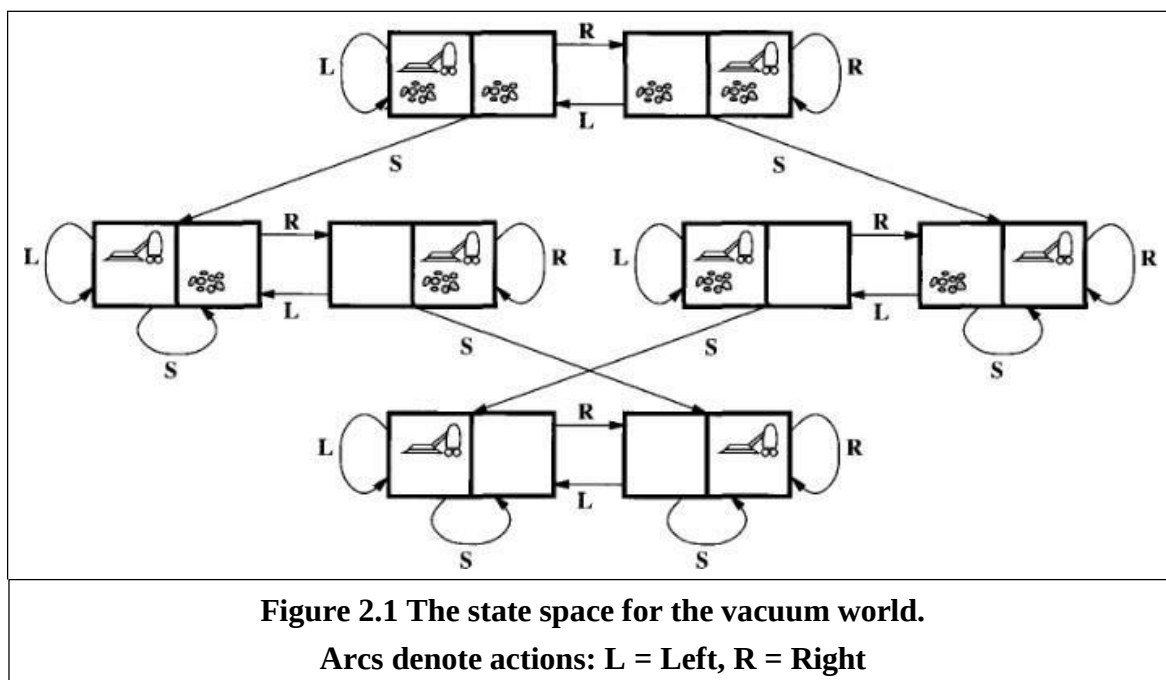
A **real world problem** is one whose solutions people actually care about.

2.3 TOY PROBLEMS

Vacuum World Example

- **States:** The agent is in one of two locations, each of which might or might not contain dirt. Thus there are $2 \times 2^2 = 8$ possible world states.
- **Initial state:** Any state can be designated as initial state.
- **Successor function:** This generates the legal states that results from trying the three actions (left, right, suck). The complete state space is shown in figure
- **Goal Test:** This tests whether all the squares are clean.
- **Path test:** Each step costs one, so that the path cost is the number of steps in the path.

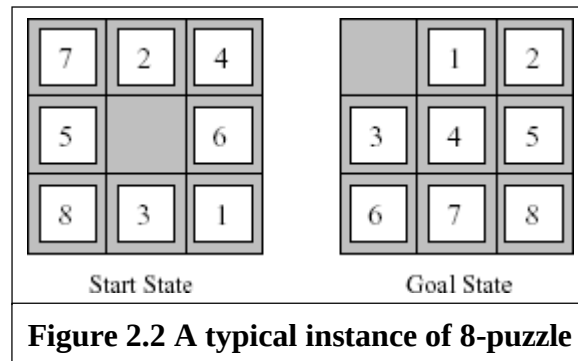
Vacuum World State Space



The 8-puzzle

An 8-puzzle consists of a 3x3 board with eight numbered tiles and a blank space. A tile adjacent to the blank space can slide into the space. The object is to reach the goal state, as shown in Figure 2.4

Example: The 8-puzzle



The problem formulation is as follows:

- **States** : A state description specifies the location of each of the eight tiles and the blank in one of the nine squares.
- **Initial state** : Any state can be designated as the initial state. It can be noted that any given goal can be reached from exactly half of the possible initial states.
- **Successor function** : This generates the legal states that result from trying the four actions(blank moves Left, Right, Up or down).
- **Goal Test** : This checks whether the state matches the goal configuration shown in Figure(Other goal configurations are possible)
- **Path cost** : Each step costs 1,so the path cost is the number of steps in the path.

The 8-puzzle belongs to the family of **sliding-block puzzles**, which are often used as test problems for new search algorithms in AI. This general class is known as NP-complete. The **8-puzzle** has $9!/2 = 181,440$ reachable states and is easily solved.

The **15 puzzle** (4 x 4 board) has around 1.3 trillion states, an the random instances can be solved optimally in few milli seconds by the best search algorithms.

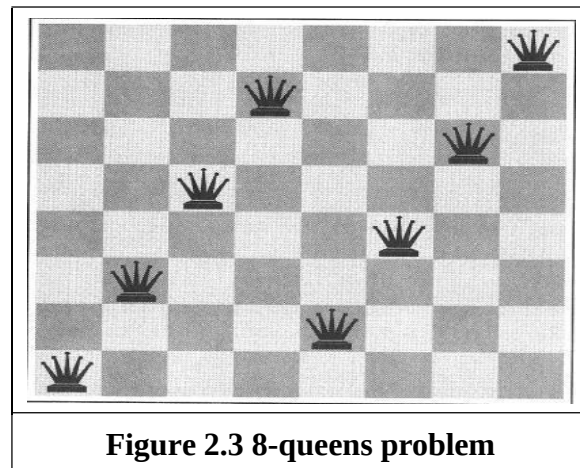
The **24-puzzle** (on a 5 x 5 board) has around 10^{25} states and random instances are still quite difficult to solve optimally with current machines and algorithms.

8-Queens problem

The goal of 8-queens problem is to place 8 queens on the chessboard such that no queen attacks any other.(A queen attacks any piece in the same row, column or diagonal).

Figure 2.3 shows an attempted solution that fails: the queen in the right most column is attacked by the queen at the top left.

An **Incremental formulation** involves operators that augments the state description, starting with an empty state. For 8-queens problem, this means each action adds a queen to the state. A **complete-state formulation** starts with all 8 queens on the board and move them around. In either case the path cost is of no interest because only the final state counts.



The first incremental formulation one might try is the following:

- **States:** Any arrangement of 0 to 8 queens on board is a state.
- **Initial state:** No queen on the board.
- **Successor function:** Add a queen to any empty square.
- **Goal Test:** 8 queens are on the board, none attacked.

In this formulation, we have $64 \cdot 63 \cdot \dots \cdot 57 = 3 \times 10^{14}$ possible sequences to investigate.

A better formulation would prohibit placing a queen in any square that is already attacked.

- **States :** Arrangements of n queens ($0 \leq n \leq 8$), one per column in the left most columns, with no queen attacking another are states.
- **Successor function :** Add a queen to any square in the left most empty column such that it is not attacked by any other queen.

This formulation reduces the 8-queen state space from 3×10^{14} to just 2057, and solutions are easy to find.

For the 100 queens the initial formulation has roughly 10^{400} states whereas the improved formulation has about 10^{52} states. This is a huge reduction, but the improved state space is still too big for the algorithms to handle.

2.4 REAL-WORLD PROBLEMS

ROUTE-FINDING PROBLEM

Route-finding problem is defined in terms of specified locations and transitions along links between them. Route-finding algorithms are used in a variety of applications, such as routing in computer networks, military operations planning, and airline travel planning systems.

2.5 AIRLINE TRAVEL PROBLEM

The **airline travel problem** is specified as follows:

- **States:** Each is represented by a location (e.g., an airport) and the current time.
- **Initial state:** This is specified by the problem.
- **Successor function:** This returns the states resulting from taking any scheduled flight (further specified by seat class and location), leaving later than the current time plus the within-airport transit time, from the current airport to another.
- **Goal Test:** Are we at the destination by some prespecified time?
- **Path cost:** This depends upon the monetary cost, waiting time, flight time, customs and immigration procedures, seat quality, time of date, type of air plane, frequent-flyer mileage awards, and so on.

2.6 TOURING PROBLEMS

Touring problems are closely related to route-finding problems, but with an important difference. Consider for example, the problem, “Visit every city at least once” as shown in Romania map.

As with route-finding the actions correspond to trips between adjacent cities. The state space, however, is quite different.

The initial state would be “In Bucharest; visited{Bucharest}”.

A typical intermediate state would be “In Vaslui; visited {Bucharest, Urziceni, Vaslui}”.

The goal test would check whether the agent is in Bucharest and all 20 cities have been visited.

2.7 THE TRAVELLING SALESPERSON PROBLEM(TSP)

Is a touring problem in which each city must be visited exactly once. The aim is to find the shortest tour. The problem is known to be **NP-hard**. Enormous efforts have been expended to improve the capabilities of TSP algorithms. These algorithms are also used in tasks such as planning movements of **automatic circuit-board drills** and of **stocking machines** on shop floors.

VLSI layout

A **VLSI layout** problem requires positioning millions of components and connections on a chip to minimize area, minimize circuit delays, minimize stray capacitances, and maximize manufacturing yield. The layout problem is split into two parts: **cell layout** and **channel routing**.

ROBOT navigation

ROBOT navigation is a generalization of the route-finding problem. Rather than a discrete set of routes, a robot can move in a continuous space with an infinite set of possible actions and states. For a circular Robot moving on a flat surface, the space is essentially two-dimensional. When the robot has arms and legs or wheels that also must be controlled, the search space becomes multi-dimensional. Advanced techniques are required to make the search space finite.

2.8 AUTOMATIC ASSEMBLY SEQUENCING

The example includes assembly of intricate objects such as electric motors. The aim in assembly problems is to find the order in which to assemble the parts of some objects. If the wrong order is chosen, there will be no way to add some part later without undoing some work already done. Another important assembly problem is protein design, in which the goal is to find a sequence of Amino acids that will be fold into a three-dimensional protein with the right properties to cure some disease.

2.9 INTERNET SEARCHING

In recent years there has been increased demand for software robots that perform Internet searching, looking for answers to questions, for related information, or for shopping deals. The searching techniques consider internet as a graph of nodes(pages) connected by links.

Different Search Algorithm

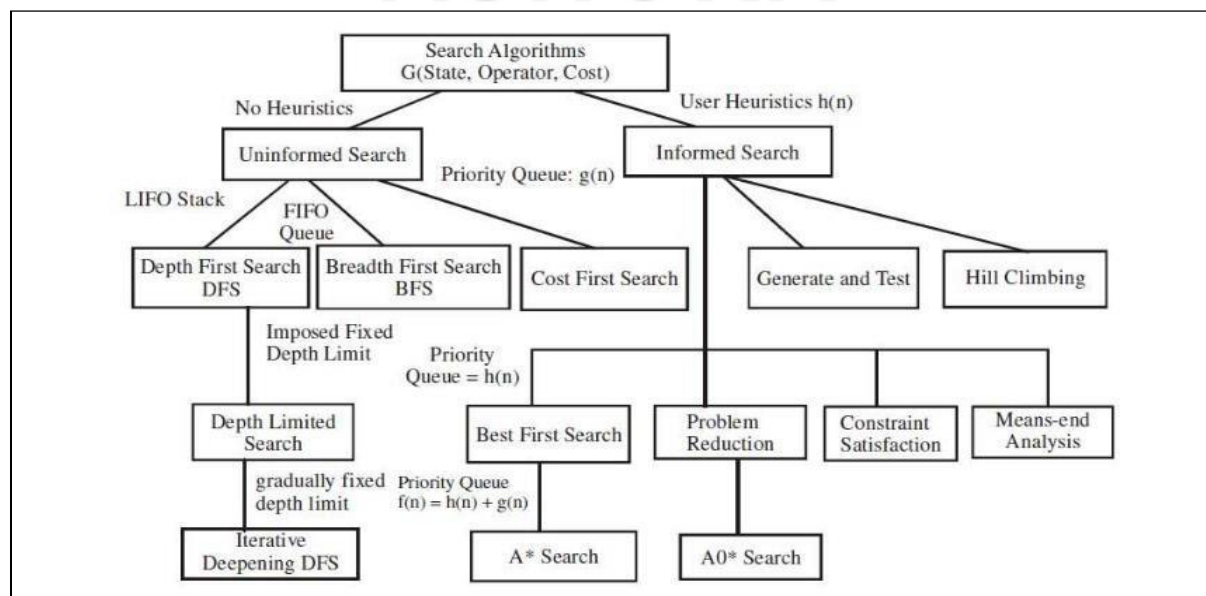


Figure 2.4 Different Search Algorithms

2.10 UNINFORMED SEARCH STRATEGIES

Uninformed Search Strategies have no additional information about states beyond that provided in the **problem definition**.

Strategies that know whether one non goal state is “more promising” than another are called

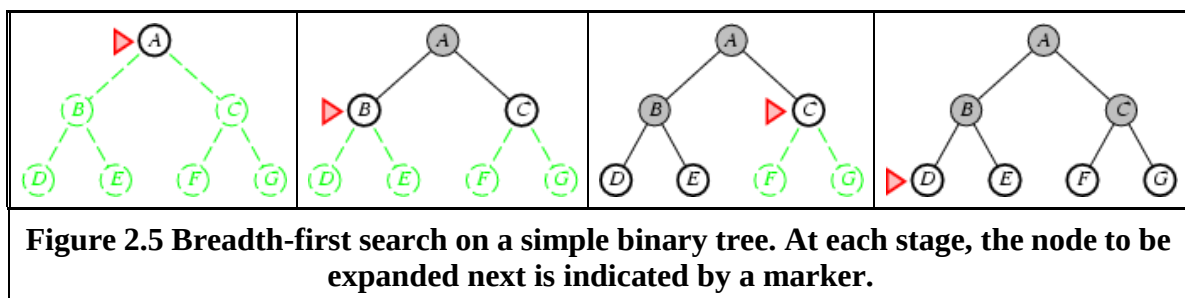
Informed search or heuristic search strategies.

There are five uninformed search strategies as given below.

- Breadth-first search
- Uniform-cost search
- Depth-first search
- Depth-limited search
- Iterative deepening search

Breadth-first search

- Breadth-first search is a simple strategy in which the root node is expanded first, then all successors of the root node are expanded next, then their successors, and so on. In general, all the nodes are expanded at a given depth in the search tree before any nodes at the next level are expanded.
- Breadth-first-search is implemented by calling TREE-SEARCH with an empty fringe that is a first-in-first-out (FIFO) queue, assuring that the nodes that are visited first will be expanded first. In otherwords, calling TREE-SEARCH (problem, FIFO-QUEUE()) results in breadth-first-search. The FIFO queue puts all newly generated successors at the end of the queue, which means that Shallow nodes are expanded before deeper nodes.



Properties of breadth-first-search

Complete?? Yes (if b is finite)

Time?? $1 + b + b^2 + b^3 + \dots + b^d + b(b^d - 1) = O(b^{d+1})$, i.e., exp. in d

Space?? $O(b^{d+1})$ (keeps every node in memory)

Optimal?? No, unless step costs are constant

Space is the big problem; can easily generate nodes at 100MB/sec
so 24hrs = 8640GB.

Figure 2.6 Breadth-first-search properties

Time complexity for BFS

Assume every state has b successors. The root of the search tree generates b nodes at the first level, each of which generates b more nodes, for a total of b^2 at the second level. Each of these generates b more nodes, yielding b^3 nodes at the third level, and so on. Now suppose, that the solution is at depth d . In the worst case, we would expand all but the last node at level d , generating $b^{d+1} - b$ nodes at level $d+1$.

Then the total number of nodes generated is $b + b^2 + b^3 + \dots + b^d + (b^{d+1} - b) = O(b^{d+1})$.

Every node that is generated must remain in memory, because it is either part of the fringe or is an ancestor of a fringe node. The space complexity is, therefore, the same as the time complexity

2.11 UNIFORM-COST SEARCH

Instead of expanding the shallowest node, **uniform-cost search** expands the node n with the lowest path cost. Uniform-cost search does not care about the number of steps a path has, but only about their total cost.

Expand least-cost unexpanded node

Implementation:

fringe = queue ordered by path cost, lowest first

Equivalent to breadth-first if step costs all equal

Complete?? Yes, if step cost $\geq \epsilon$

Time?? # of nodes with $g \leq$ cost of optimal solution, $O(b^{\lceil C^*/\epsilon \rceil})$
where C^* is the cost of the optimal solution

Space?? # of nodes with $g \leq$ cost of optimal solution, $O(b^{\lceil C^*/\epsilon \rceil})$

Optimal?? Yes—nodes expanded in increasing order of $g(n)$

2.12 DEPTH-FIRST-SEARCH

Depth-first-search always expands the deepest node in the current fringe of the search tree. The progress of the search is illustrated in Figure 1.31. The search proceeds immediately to the deepest level of the search tree, where the nodes have no successors. As those nodes are expanded, they are dropped from the fringe, so then the search “backs up” to the next shallowest node that still has unexplored successors.

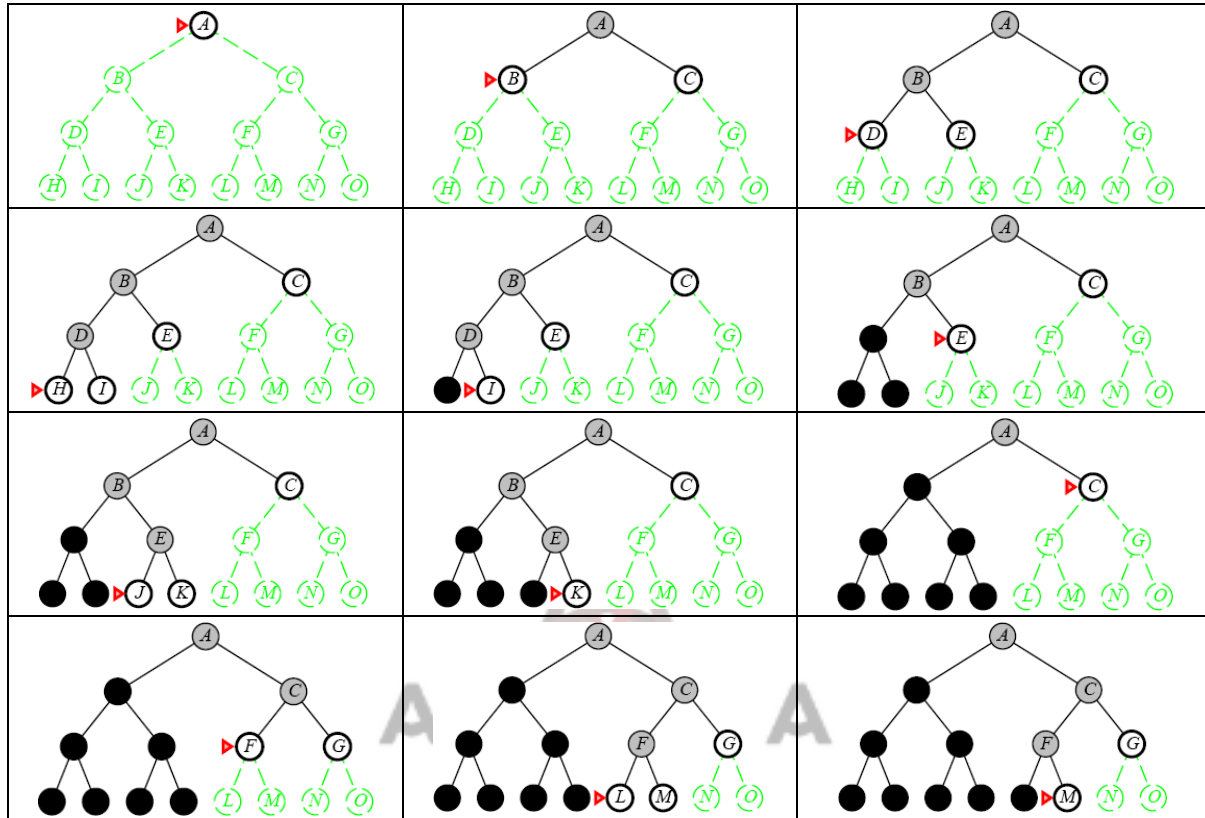


Figure 2.7 Depth-first-search on a binary tree. Nodes that have been expanded and have node scendants in the fringe can be removed from the memory; these are shown in black. Nodes at depth 3 are assumed to have no successors and M is the only goal node.

This strategy can be implemented by TREE-SEARCH with a last-in-first-out (LIFO) queue, also known as a stack.

Depth-first-search has very modest memory requirements. It needs to store only a single path from the root to a leaf node, along with the remaining unexpanded sibling nodes for each node on the path. Once the node has been expanded, it can be removed from the memory, as soon as its descendants have been fully explored (Refer Figure 2.7).

For a state space with a branching factor b and maximum depth m , depth-first-search requires storage of only $bm + 1$ nodes.

Using the same assumptions as Figure, and assuming that nodes at the same depth as the goal node have no successors, we find the depth-first-search would require 118 kilobytes instead of 10 petabytes, a factor of 10 billion times less space.

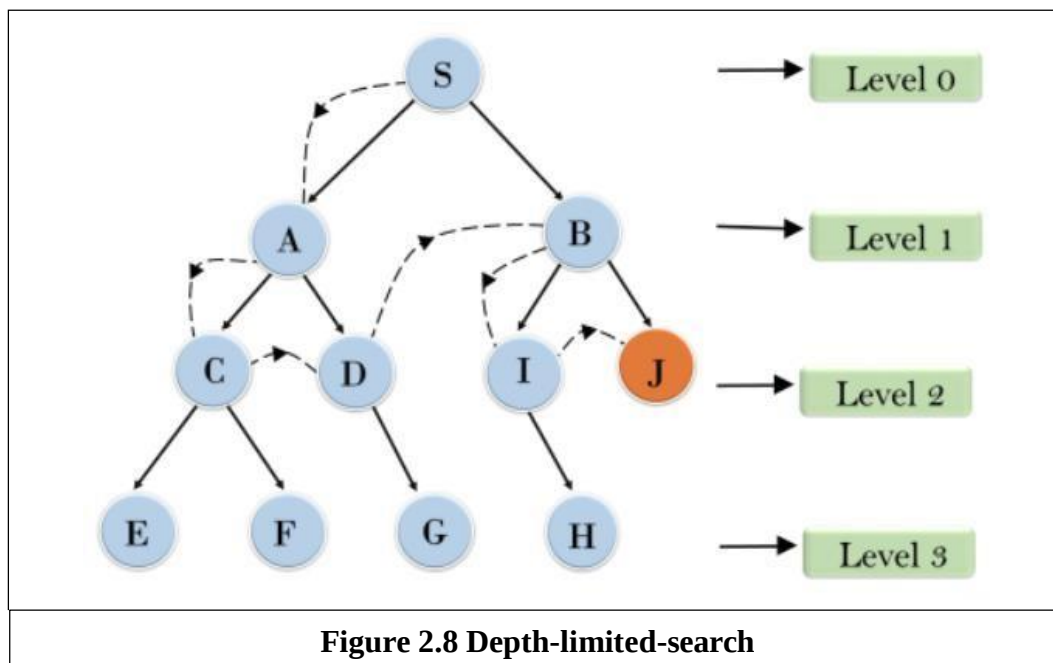
Drawback of Depth-first-search

The drawback of depth-first-search is that it can make a wrong choice and get stuck going down very long(or even infinite) path when a different choice would lead to solution near the root of the search tree. For example, depth-first-search will explore the entire left subtree even if node C is a goal node.

2.12 BACKTRACKING SEARCH

A variant of depth-first search called backtracking search uses less memory and only one successor is generated at a time rather than all successors.; Only $O(m)$ memory is needed rather than $O(bm)$

DEPTH-LIMITED-SEARCH



The problem of unbounded trees can be alleviated by supplying depth-first-search with a pre- determined depth limit l . That is, nodes at depth l are treated as if they have no successors. This approach is called **depth-limited-search**. The depth limit solves the infinite path problem.

Depth limited search will be nonoptimal if we choose $l > d$. Its time complexity is $O(b^l)$ and its space complete is $O(bl)$. Depth-first-search can be viewed as a special case of depth-limited search with $l = \infty$ Sometimes, depth limits can be based on knowledge of the problem. For, example, on the map of Romania there are 20 cities. Therefore, we know that if there is a solution, it must be of length 19 at the longest, So $l = 10$ is a possible choice. However, it can be shown that any city can be reached from any other city in at most 9 steps. This number known as the **diameter** of the state space, gives us a better depth limit.

Depth-limited-search can be implemented as a simple modification to the general tree-search algorithm or to the recursive depth-first-search algorithm. The pseudocode for recursive depth-limited-search is shown in Figure.

It can be noted that the above algorithm can terminate with two kinds of failure : the standard *failure* value indicates no solution; the *cutoff* value indicates no solution within the depth limit. Depth-limited search = depth-first search with depth limit l, returns cut off if any path is cut off by depth limit

```
function Depth-Limited-Search( problem, limit) returns a solution/fail/cutoff return
Recursive-DLS(Make-Node(Initial-State[problem]), problem, limit) function Recursive-
DLS(node, problem, limit) returns solution/fail/cutoff cutoff-occurred?    false
if Goal-Test(problem,State[node]) then return Solution(node)
else if Depth[node] = limit then return cutoff
else for each successor in Expand(node, problem) do result
Recursive-DLS(successor, problem, limit) if result = cutoff then cutoff_occurred?true
else if result not = failure then return result
if cutoff_occurred? then return cutoff else return failure
```

Figure 2.9 Recursive implementation of Depth-limited-search

2.13 ITERATIVE DEEPENING DEPTH-FIRST SEARCH

Iterative deepening search (or iterative-deepening-depth-first-search) is a general strategy often used in combination with depth-first-search, that finds the better depth limit. It does this by gradually increasing the limit – first 0, then 1, then 2, and so on – until a goal is found. This will occur when the depth limit reaches d, the depth of the shallowest goal node. The algorithm is shown in Figure.

Iterative deepening combines the benefits of depth-first and breadth-first-search. Like depth-first-search, its memory requirements are modest; $O(bd)$ to be precise.

Like Breadth-first-search, it is complete when the branching factor is finite and optimal when the path cost is a non decreasing function of the depth of the node.

Figure shows the four iterations of ITERATIVE-DEEPENING_SEARCH on a binary search tree, where the solution is found on the fourth iteration.

```

function ITERATIVE-DEEPENING-SEARCH(problem) returns a solution
  inputs: problem, a problem
  for depth  $\leftarrow$  0 to  $\infty$  do
    result  $\leftarrow$  DEPTH-LIMITED-SEARCH(problem, depth)
    if result  $\neq$  cutoff then return result
  end

```

Figure 2.10 The iterative deepening search algorithm, which repeatedly applies depth-limited- search with increasing limits. It terminates when a solution is found or if the depth limited search returns *failure*, meaning that no solution exists.

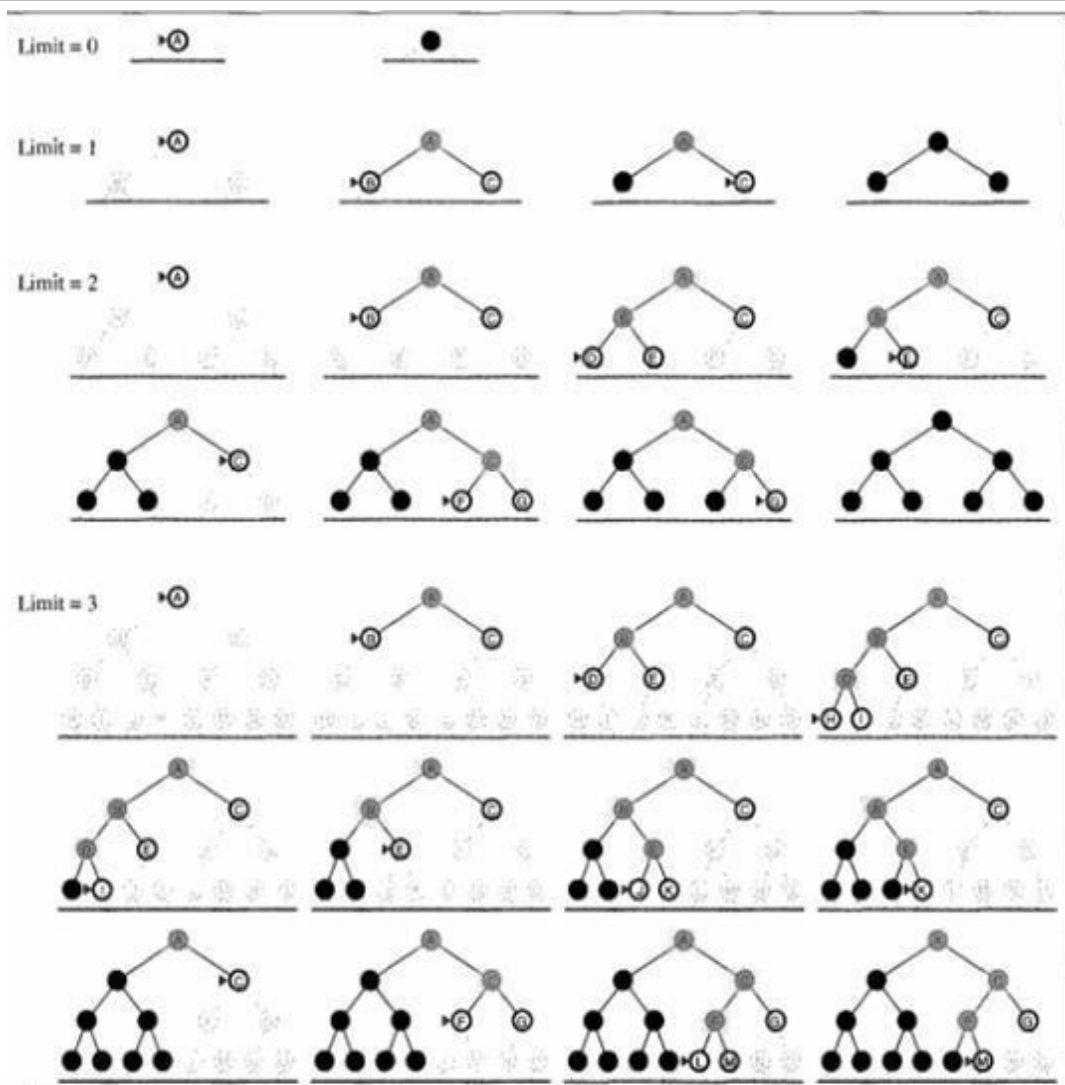


Figure 2.11 Four iterations of iterative deepening search on a binary tree

Iterative search is not as wasteful as it might seem

Iterative deepening

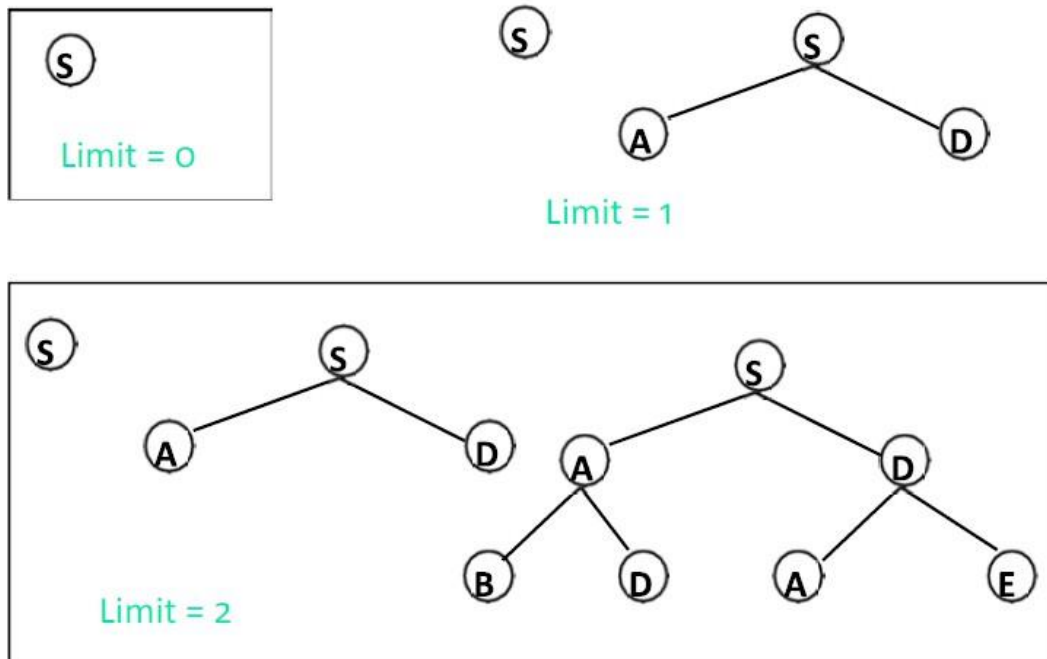


Figure 2.12 Iterative search is not as wasteful as it might seem

Properties of iterative deepening search

Complete?? Yes

Time?? $(d+1)b^0 + db^1 + (d-1)b^2 + \dots + b^d = O(b^d)$

Space?? $O(bd)$

Optimal?? No, unless step costs are constant

Can be modified to explore uniform-cost tree

Numerical comparison for $b = 10$ and $d = 5$, solution at far right leaf:

$$N(\text{IDS}) = 50 + 400 + 3,000 + 20,000 + 100,000 = 123,450$$

$$N(\text{BFS}) = 10 + 100 + 1,000 + 10,000 + 100,000 + 999,990 = 1,111,100$$

IDS does better because other nodes at depth d are not expanded

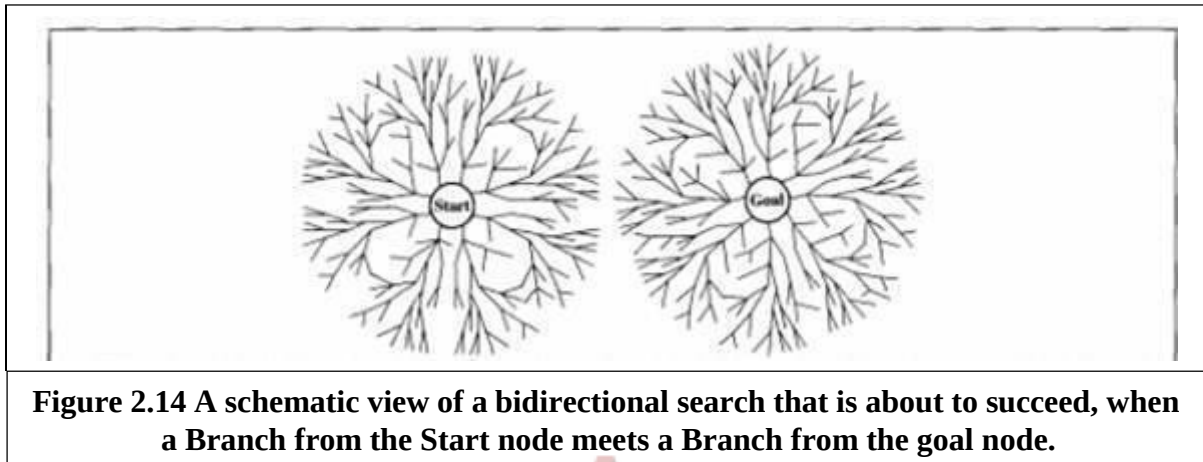
BFS can be modified to apply goal test when a node is generated

Figure 2.13 Properties of iterative deepening search

Bidirectional Search

The idea behind bidirectional search is to run two simultaneous searches – one forward from the initial state and the other backward from the goal, stopping when the two searches meet in the middle

The motivation is that $b^{d/2} + b^{d/2}$ much less than, or in the figure, the area of the two small circles is less than the area of one big circle centered on the start and reaching to the goal.



- Before moving into bidirectional search let's first understand a few terms.
- Forward Search: Looking in-front of the end from start.
- Backward Search: Looking from end to the start back-wards.
- So Bidirectional Search as the name suggests is a combination of forwarding and backward search. Basically, if the average branching factor going out of node / fan-out, if fan-out is less, prefer forward search. Else if the average branching factor is going into a node/fan in is less (i.e. fan-out is more), prefer backward search.
- We must traverse the tree from the start node and the goal node and wherever they meet the path from the start node to the goal through the intersection is the optimal solution. The BS Algorithm is applicable when generating predecessors is easy in both forward and backward directions and there exist only 1 or fewer goal states.

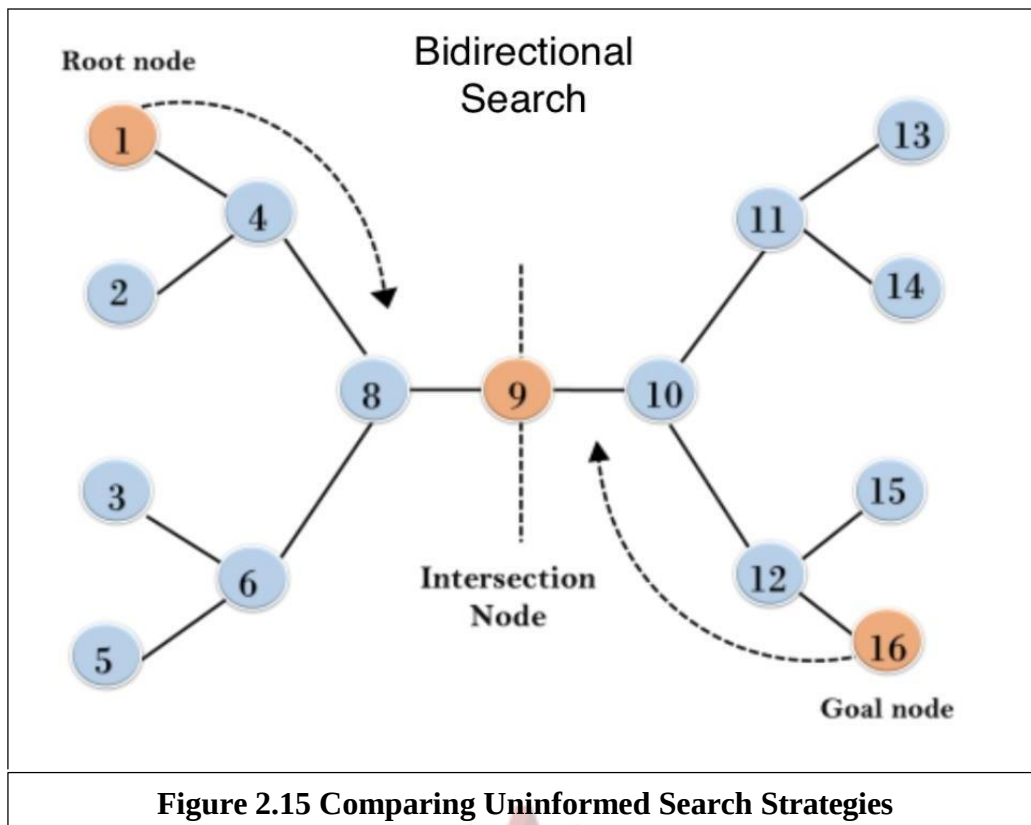


Figure 2.15 Comparing Uninformed Search Strategies

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening	Bidirectional (if applicable)
Complete?	Yes ^a	Yes ^{a,b}	No	No	Yes ^a	Yes ^{a,d}
Time	$O(b^{d+1})$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(b^m)$	$O(b^l)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^{d+1})$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(bm)$	$O(b^l)$	$O(bd)$	$O(b^{d/2})$
Optimal?	Yes ^c	Yes	No	No	Yes ^c	Yes ^{c,d}

Figure 2.16 Evaluation of search strategies, b is the branching factor; d is the depth of the shallowest solution; m is the maximum depth of the search tree; l is the depth limit. Superscript caveats are as follows: ^a complete if b is finite; ^b complete if step costs $\geq E$ for positive E ; ^c optimal if step costs are all identical; ^d if both directions use breadth-first search.

2.14 SEARCHING WITH PARTIAL INFORMATION

- Different types of incompleteness lead to three distinct problem types:
- **Sensorless problems** (conformant): If the agent has no sensors at all
- **Contingency problem**: if the environment is partially observable or if actions are uncertain (adversarial)
- **Exploration problems**: When the states and actions of the environment are unknown.

- No sensor
- Initial State(1,2,3,4,5,6,7,8)
- After action [Right] the state (2,4,6,8)
- After action [Suck] the state (4, 8)
- After action [Left] the state (3,7)
- After action [Suck] the state (8)
- Answer : [Right, Suck, Left, Suck] coerce the world into state 7 without any sensor
- Belief State: Such state that agent belief to be there

Partial knowledge of states and actions:

- *sensorless or conformant problem*
 - Agent may have no idea where it is; solution (if any) is a sequence.
- *contingency problem*
 - Percepts provide *new* information about current state; solution is a tree or policy; often interleave search and execution.
 - If uncertainty is caused by actions of another agent: *adversarial problem*
- *exploration problem*
 - When states and actions of the environment are unknown.

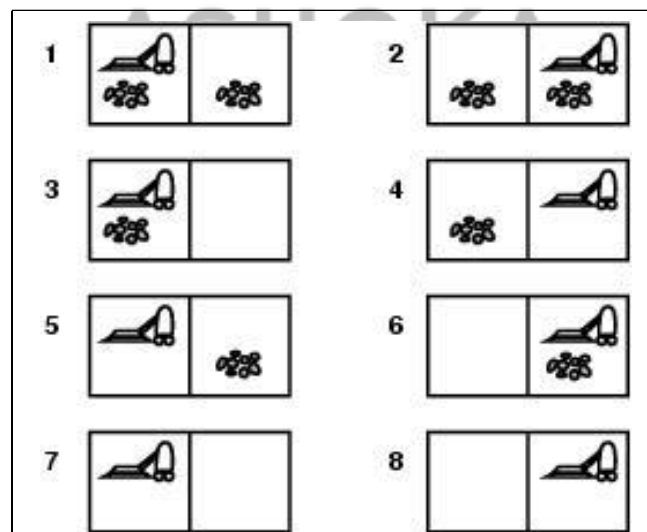


Figure 2.17 states and actions of the environment are unknown

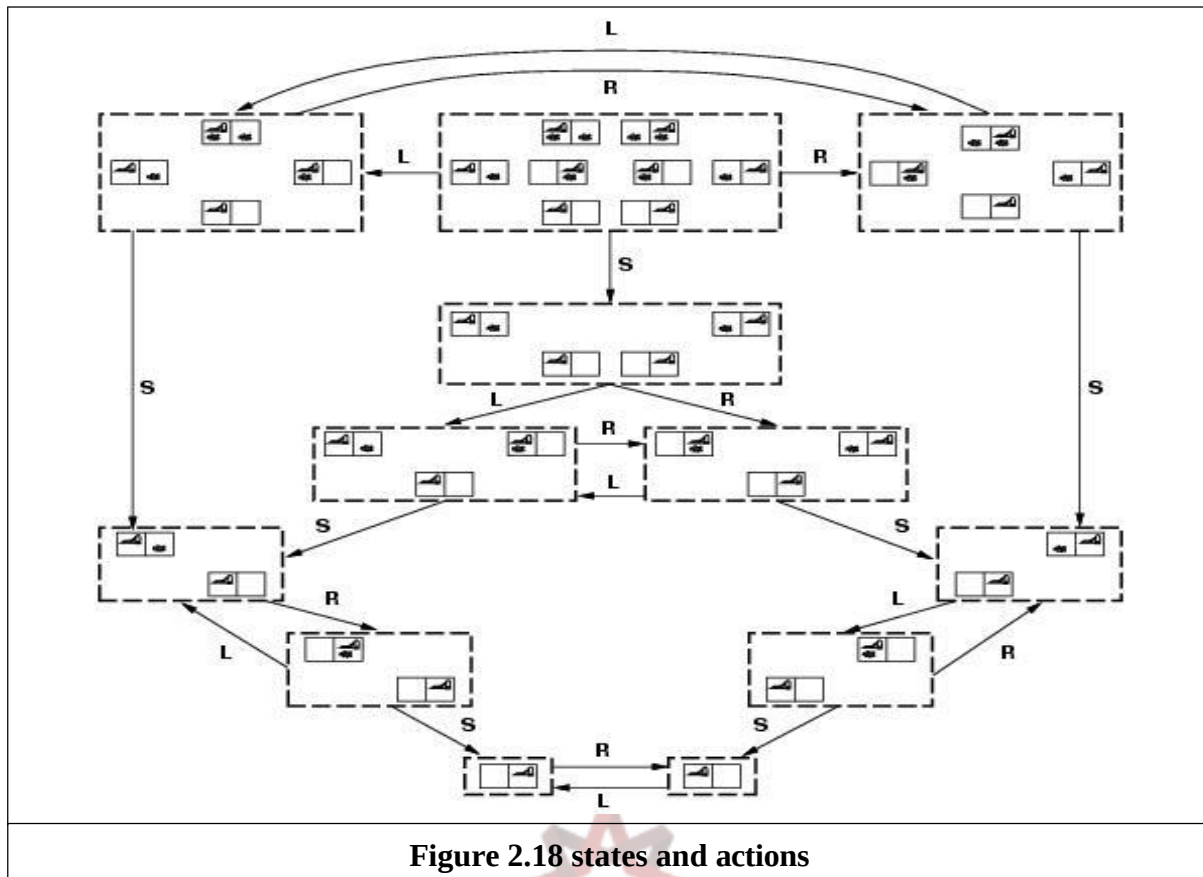


Figure 2.18 states and actions

Contingency, start in {1,3}.

Murphy's law, Suck *can* dirty a clean carpet. Local sensing: dirt, location only.

- Percept = [L,Dirty] = {1,3}
- [Suck] = {5,7}
- [Right] = {6,8}
- [Suck] in {6} = {8} (Success)
- BUT [Suck] in {8} = failure Solution??
- Belief-state: no fixed action sequence guarantees solution

Relax requirement:

- [Suck, Right, if [R,dirty] then Suck]
- Select actions based on contingencies arising during execution.

Time and space complexity are always considered with respect to some measure of the problem difficulty. In theoretical computer science, the typical measure is the size of the state space.

In AI, where the graph is represented implicitly by the initial state and successor function, the complexity is expressed in terms of three quantities:

- b**, the **branching factor** or maximum number of successors of any node;
- d**, the **depth** of the **shallowest goal node**; and

m, the **maximum length** of any path in the state space.

Search-cost - typically depends upon the time complexity but can also include the term for memory usage.

Total-cost – It combines the search-cost and the path cost of the solution found.

2.15 INFORMED SEARCH AND EXPLORATION

Informed (Heuristic) Search Strategies

Informed search strategy is one that uses problem-specific knowledge beyond the definition of the problem itself. It can find solutions more efficiently than uninformed strategy.

Best-first search

Best-first search is an instance of general TREE-SEARCH or GRAPH-SEARCH algorithm in which a node is selected for expansion based on an **evaluation function** $f(n)$. The node with lowest evaluation is selected for expansion, because the evaluation measures the distance to the goal.

This can be implemented using a priority-queue, a data structure that will maintain the fringe in ascending order of f -values.

2.16 HEURISTIC FUNCTIONS

A **heuristic function** or simply a **heuristic** is a function that ranks alternatives in various search algorithms at each branching step basing on an available information in order to make a decision which branch is to be followed during a search.

The key component of Best-first search algorithm is a **heuristic function**, denoted by $h(n)$: $h(n)$ = estimated cost of the **cheapest path** from node n to a **goal node**.

For example, in Romania, one might estimate the cost of the cheapest path from Arad to Bucharest via a **straight-line distance** from Arad to Bucharest (Figure 2.19).

Heuristic function are the most common form in which additional knowledge is imparted to the search algorithm.

Greedy Best-first search

Greedy best-first search tries to expand the node that is closest to the goal, on the grounds that this is likely to a solution quickly.

It evaluates the nodes by using the heuristic function $f(n) = h(n)$.

Taking the example of **Route-finding problems** in Romania, the goal is to reach Bucharest starting from the city Arad. We need to know the straight-line distances to Bucharest from various cities as shown in Figure. For example, the initial state is $In(Arad)$, and the straight line distance heuristic $hSLD(In(Arad))$ is found to be 366.

Using the **straight-line distance** heuristic **hSLD**, the goal state can be reached faster.

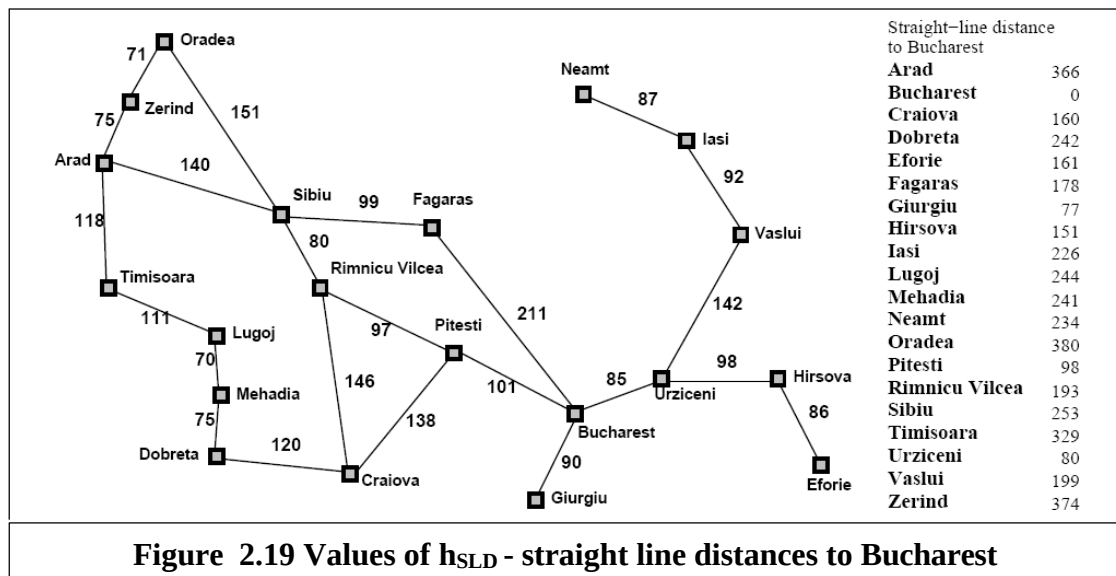


Figure 2.19 Values of h_{SLD} - straight line distances to Bucharest

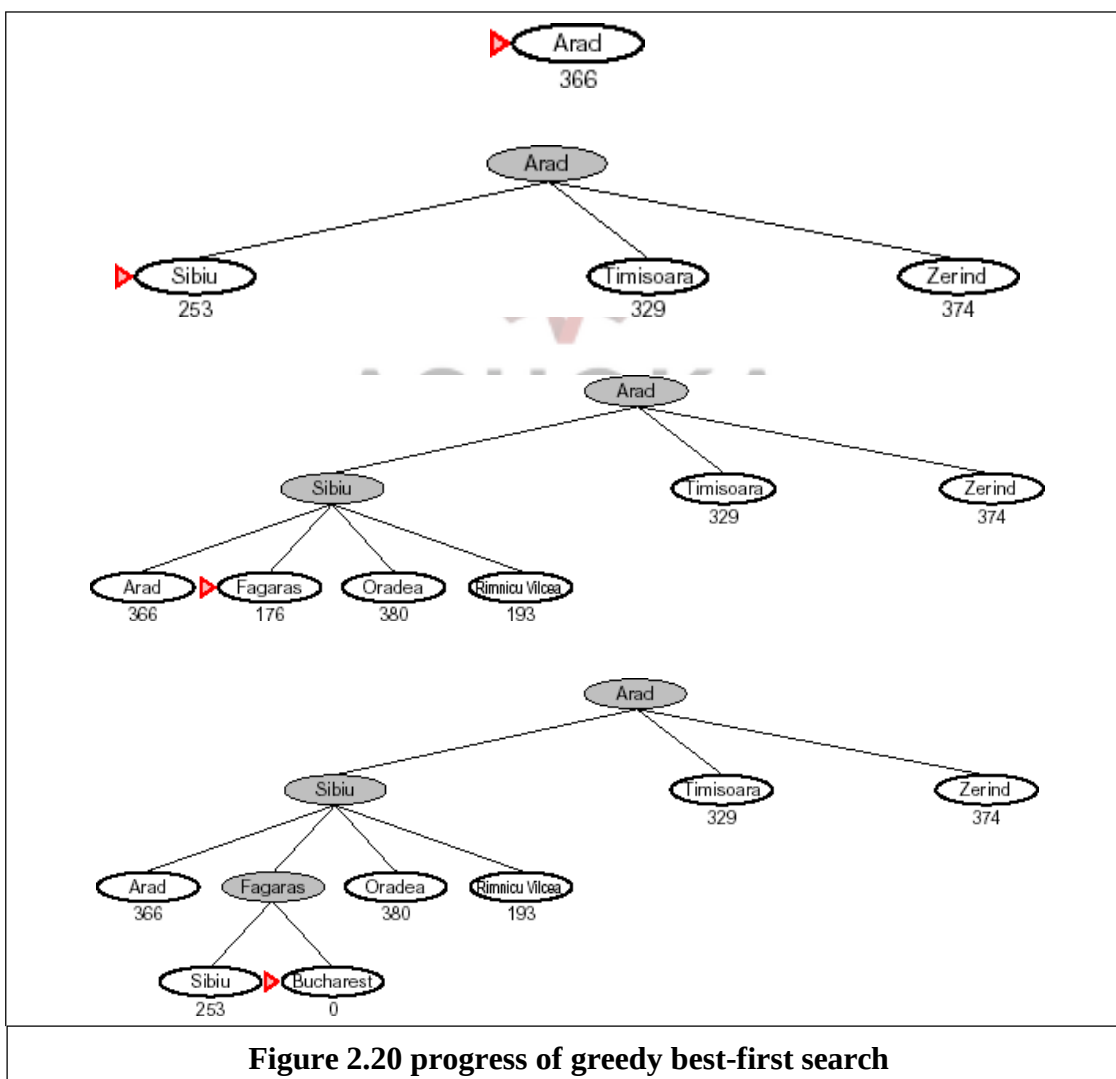


Figure 2.20 progress of greedy best-first search

Figure shows the progress of greedy best-first search using hSLD to find a path from Arad to Bucharest. The first node to be expanded from Arad will be Sibiu, because it is closer to Bucharest than either Zerind or Timisoara. The next node to be expanded will be Fagaras, because it is closest. Fagaras in turn generates Bucharest, which is the goal.

Properties of greedy search

- **Complete:** No—can get stuck in loops, e.g., Iasi !Neamt !Iasi !Neamt !
Complete in finite space with repeated-state checking
- **Time:** $O(bm)$, but a good heuristic can give dramatic improvement
- **Space:** $O(bm)$ - keeps all nodes in memory
- **Optimal:** No

Greedy best-first search is not optimal, and it is incomplete.

The worst-case time and space complexity is $O(b^m)$, where m is the maximum depth of the search space.

A* SEARCH

A* Search is the most widely used form of best-first search. The evaluation function $f(n)$ is obtained by combining

(1) $g(n)$ = the cost to reach the node, and

(2) $h(n)$ = the cost to get from the node to the **goal** :

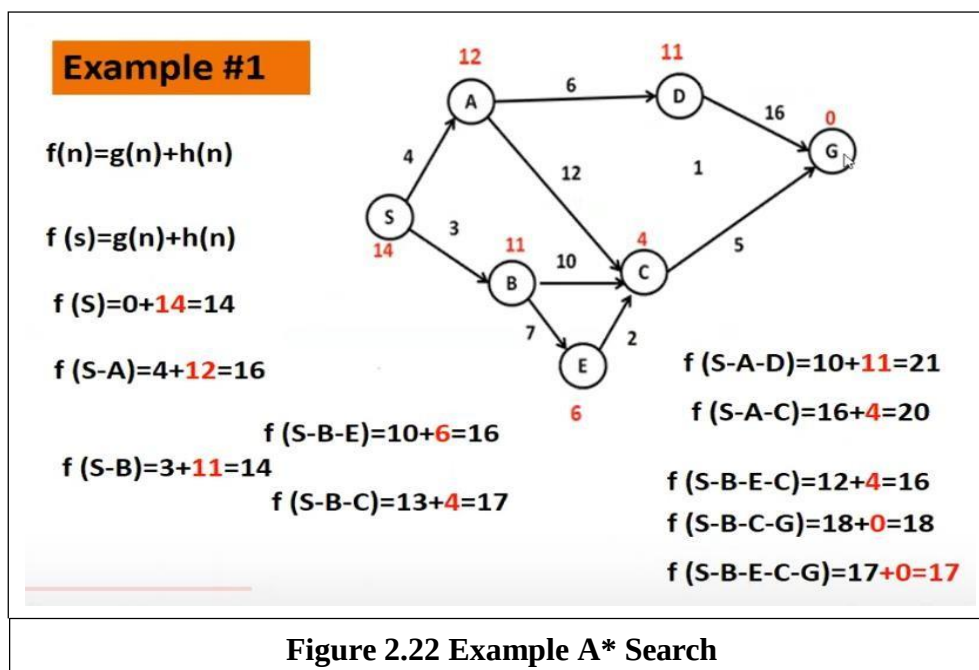
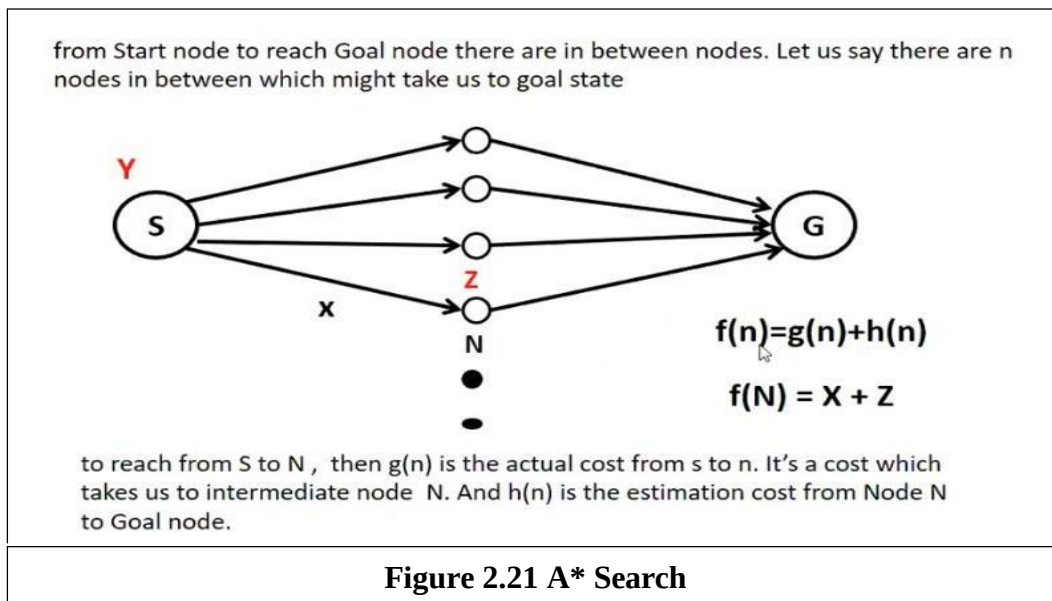
$$f(n) = g(n) + h(n).$$

A* Search is both optimal and complete. A* is optimal if $h(n)$ is an admissible heuristic. The obvious example of admissible heuristic is the straight-line distance hSLD. It cannot be an overestimate.

A* Search is optimal if $h(n)$ is an admissible heuristic – that is, provided that $h(n)$ never overestimates the cost to reach the goal.

An obvious example of an admissible heuristic is the straight-line distance hSLD that we used in getting to Bucharest. The progress of an A* tree search for Bucharest is shown in Figure

The values of 'g' are computed from the step costs shown in the Romania map (figure). Also the values of hSLD are given in Figure



2.17 LOCAL SEARCH ALGORITHMS AND OPTIMIZATION PROBLEMS

- In many optimization problems, the path to the goal is irrelevant; the goal state itself is the solution
- For example, in the 8-queens problem, what matters is the final configuration of queens, not the order in which they are added.
- In such cases, we can **use local search algorithms**. They operate using a **single current state** (rather than multiple paths) and generally move only to neighbors of that state.

- The important applications of these class of problems are (a) integrated-circuit design, (b) Factory-floor layout, (c) job-shop scheduling, (d) automatic programming, (e) telecommunications network optimization, (f) Vehicle routing, and (g) portfolio management.

Key advantages of Local Search Algorithms

- (1) They use very little memory – usually a constant amount; and
- (2) they can often find reasonable solutions in large or infinite(continuous) state spaces for which systematic algorithms are unsuitable.

2.18 OPTIMIZATION PROBLEMS

In addition to finding goals, local search algorithms are useful for solving pure **optimization problems**, in which the aim is to find the **best state** according to an **objective function**.

State Space Landscape

To understand local search, it is better explained using **state space landscape** as shown in Figure.

A landscape has both “**location**” (defined by the state) and “**elevation**” (defined by the value of the heuristic cost function or objective function).

If elevation corresponds to **cost**, then the aim is to find the **lowest valley** – a **global minimum**; if elevation corresponds to an **objective function**, then the aim is to find the **highest peak** – a **global maximum**.

Local search algorithms explore this landscape. A complete local search algorithm always finds a **goal** if one exists; an **optimal** algorithm always finds a **global minimum/maximum**.

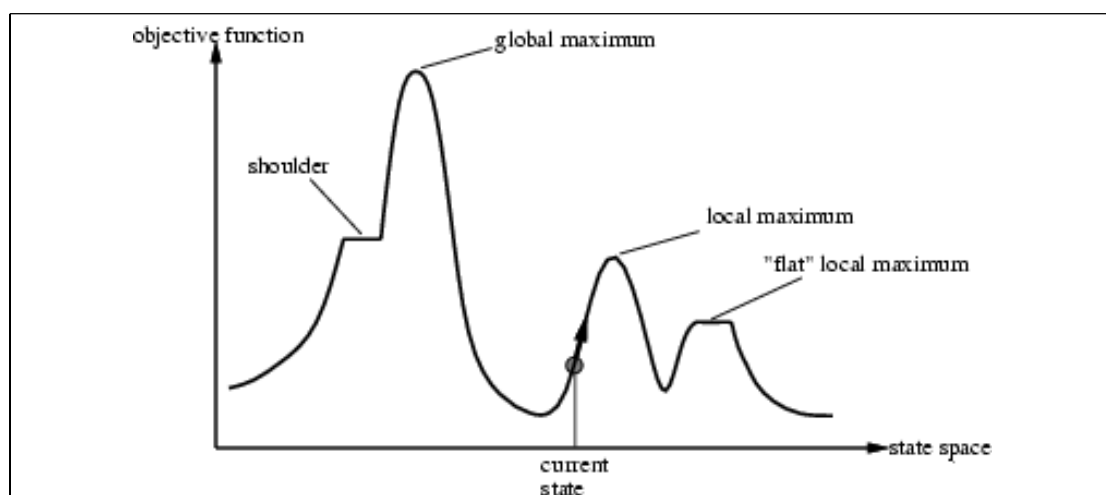


Figure 2.23 A one dimensional state space landscape in which elevation corresponds to the objective function. The aim is to find the global maximum. Hill climbing search modifies the current state to try to improve it, as shown by the arrow. The various topographic features are defined in the text

Hill-climbing search

The **hill-climbing** search algorithm as shown in figure, is simply a loop that continually moves in the direction of increasing value – that is, **uphill**. It terminates when it reaches a “**peak**” where no neighbor has a higher value.

```
function HILL-CLIMBING( problem) return a state that is a local maximum
    input: problem, a problem
    local variables: current, a
                      node.
                      neighbor, a node.

    current ← MAKE-NODE(INITIAL-STATE[problem])
    loop do
        neighbor ← a highest valued successor of current
        if VALUE [neighbor] ≤ VALUE[current] then return STATE[current]
        current ← neighbor
```

Figure 2.24 The hill-climbing search algorithm (steepest ascent version), which is the most basic local search technique. At each step the current node is replaced by the best neighbor; the neighbor with the highest VALUE. If the heuristic cost estimate h is used, we could find the neighbor with the lowest h .

Hill-climbing is sometimes called greedy local search because it grabs a good neighbor state without thinking ahead about where to go next. Greedy algorithms often perform quite well. **Problems with hill-climbing**

Hill-climbing often gets stuck for the following reasons :

- **Local maxima:** a local maximum is a peak that is higher than each of its neighboring states, but lower than the global maximum. Hill-climbing algorithms that reach the vicinity of a local maximum will be drawn upwards towards the peak, but will then be stuck with nowhere else to go
- **Ridges:** A ridge is shown in Figure 2.10. Ridges results in a sequence of local maxima that is very difficult for greedy algorithms to navigate.
- **Plateaux:** A plateau is an area of the state space landscape where the evaluation function is flat. It can be a flat local maximum, from which no uphill exit exists, or a shoulder, from which it is possible to make progress.

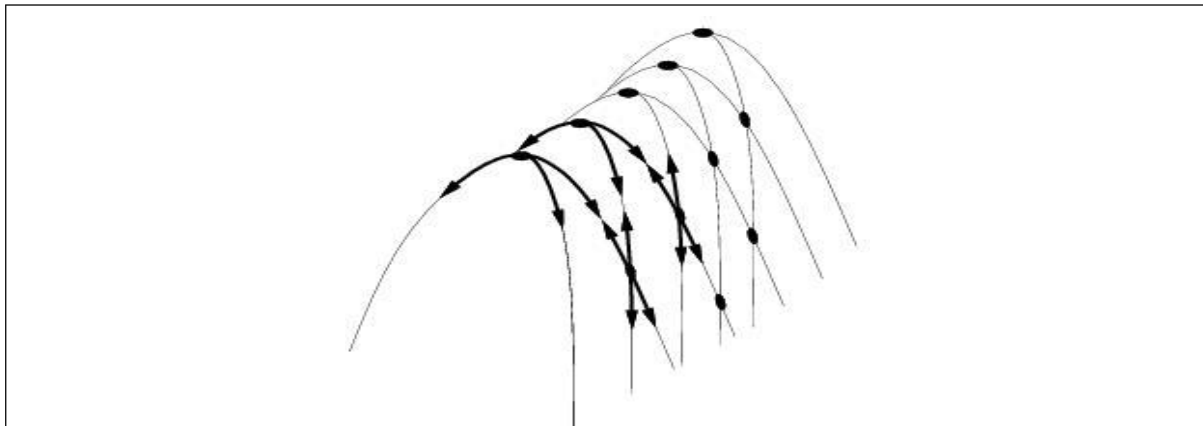


Figure 2.25 Illustration of why ridges cause difficulties for hill-climbing. The grid of states(dark circles) is superimposed on a ridge rising from left to right, creating a sequence of local maxima that are not directly connected to each other. From each local maximum, all the available options point downhill.

Hill-climbing variations

- Stochastic hill-climbing
 - Random selection among the uphill moves.
 - The selection probability can vary with the steepness of the uphill move.
- First-choice hill-climbing
 - cfr. stochastic hill climbing by generating successors randomly until a better one is found.
- Random-restart hill-climbing
 - Tries to avoid getting stuck in local maxima.

Simulated annealing search

A hill-climbing algorithm that never makes “downhill” moves towards states with lower value (or higher cost) is guaranteed to be incomplete, because it can stuck on a local maximum. In contrast, a purely random walk –that is, moving to a successor choosen uniformly at random from the set of successors – is complete, but extremely inefficient.

Simulated annealing is an algorithm that combines hill-climbing with a random walk in someway that yields both efficiency and completeness.

Figure shows simulated annealing algorithm. It is quite similar to hill climbing. Instead of picking the best move, however, it picks the random move. If the move improves the situation, it is always accepted. Otherwise, the algorithm accepts the move with some probability less than 1. The probability decreases exponentially with the “badness” of the move – the amount E by which the evaluation is worsened.

Simulated annealing was first used extensively to solve VLSI layout problems in the early 1980s. It has been applied widely to factory scheduling and other large-scale optimization tasks.

```

function SIMULATED-ANNEALING(problem, schedule) returns a solution state
  inputs: problem, a problem
           schedule, a mapping from time to "temperature"
  local variables: current, a node
                    next, a node
                    T, a "temperature" controlling prob. of downward steps

  current  $\leftarrow$  MAKE-NODE(INITIAL-STATE[problem])
  for t  $\leftarrow$  1 to  $\infty$  do
    T  $\leftarrow$  schedule[t]
    if T = 0 then return current
    next  $\leftarrow$  a randomly selected successor of current
     $\Delta E \leftarrow$  VALUE[next] - VALUE[current]
    if  $\Delta E > 0$  then current  $\leftarrow$  next
    else current  $\leftarrow$  next only with probability  $e^{\Delta E/T}$ 

```

Figure 2.26 The simulated annealing search algorithm, a version of stochastic hill climbing where some downhill moves are allowed.

Genetic algorithms

A Genetic algorithm (or GA) is a variant of stochastic beam search in which successor states are generated by combining two parent states, rather than by modifying a single state

Like beam search, Gas begin with a set of k randomly generated states, called the population. Each state, or individual, is represented as a string over a finite alphabet – most commonly, a string of 0s and 1s. For example, an 8 8-queens state must specify the positions of 8 queens, each in a column of 8 squares, and so requires $8 \times \log_2 8 = 24$ bits.

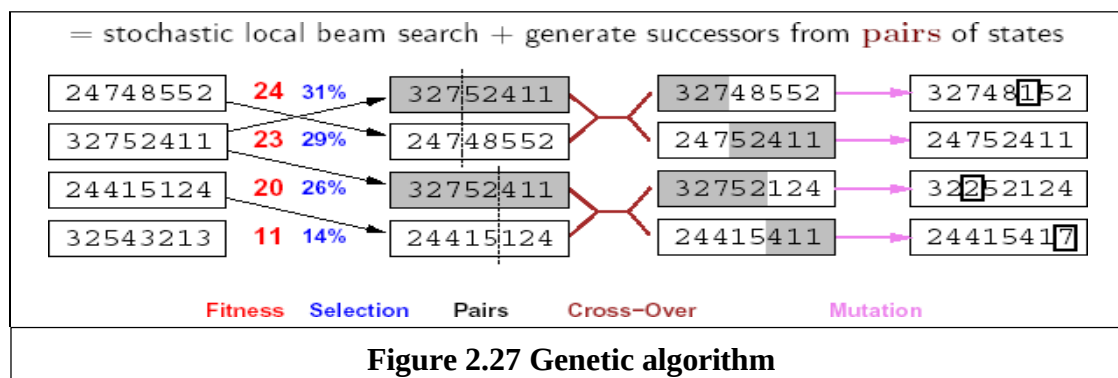


Figure shows a population of four 8-digit strings representing 8-queen states. The production of the next generation of states is shown in Figure

In (b) each state is rated by the evaluation function or the **fitness function**.

In (c), a random choice of two pairs is selected for reproduction, in accordance with the probabilities in (b).

Figure describes the algorithm that implements all these steps.

```

function GENETIC_ALGORITHM( population, FITNESS-FN) return an individual
  input: population, a set of individuals
          FITNESS-FN, a function which determines the quality of the individual
  repeat
    new_population ← empty set
    loop for i from 1 to SIZE(population) do
      x ← RANDOM_SELECTION(population, FITNESS_FN)
      y ← RANDOM_SELECTION(population,
                           FITNESS_FN)
      child ← REPRODUCE(x, y)
      if (small random probability) then child ←
        MUTATE(child) add child to new_population
    population ← new_population
  until some individual is fit enough or enough time has elapsed
  return the best individual
  
```

Figure 2.28 A genetic algorithm.

2.29 CONSTRAINT SATISFACTION PROBLEMS(CSP)

A **Constraint Satisfaction Problem**(or CSP) is defined by a set of **variables**, $X_1, X_2, \dots, X_n$, and a set of constraints $C_1, C_2, \dots, C_m$. Each variable X_i has a nonempty **domain** D_i of possible **values**.

Each constraint C_i involves some subset of variables and specifies the allowable combinations of values for that subset.

A **State** of the problem is defined by an **assignment** of values to some or all of the variables, $\{X_i = v_i, X_j = v_j, \dots\}$. An assignment that does not violate any constraints is called a **consistent** or **legal assignment**. A complete assignment is one in which every variable is mentioned, and a **solution** to a CSP is a complete assignment that satisfies all the constraints.

Some CSPs also require a solution that maximizes an **objective function**.

Example for Constraint Satisfaction Problem

Figure shows the map of Australia showing each of its states and territories. We are given the task of coloring each region either red, green, or blue in such a way that the neighboring regions have the same color. To formulate this as CSP, we define the variable to be the regions

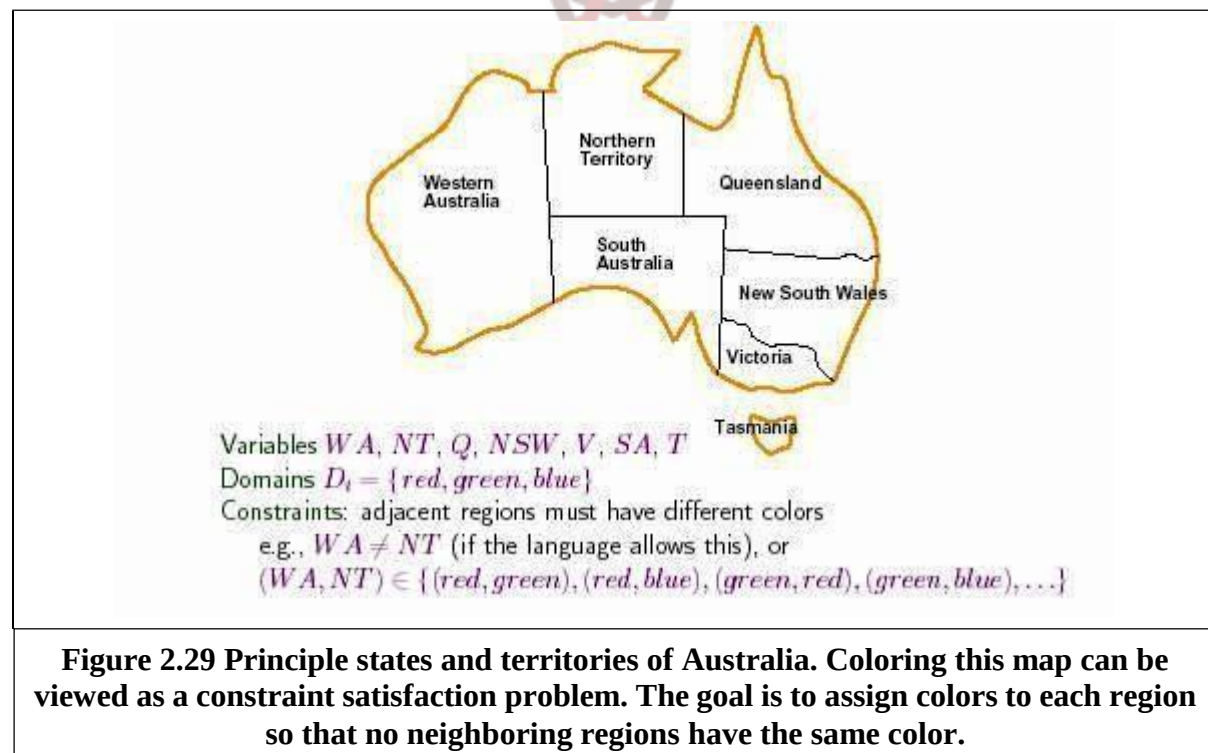
:WA,NT,Q,NSW,V,SA, and T. The domain of each variable is the set {red,green,blue}.The constraints require neighboring regions to have distinct colors; for example, the allowable combinations for WA and NT are the pairs

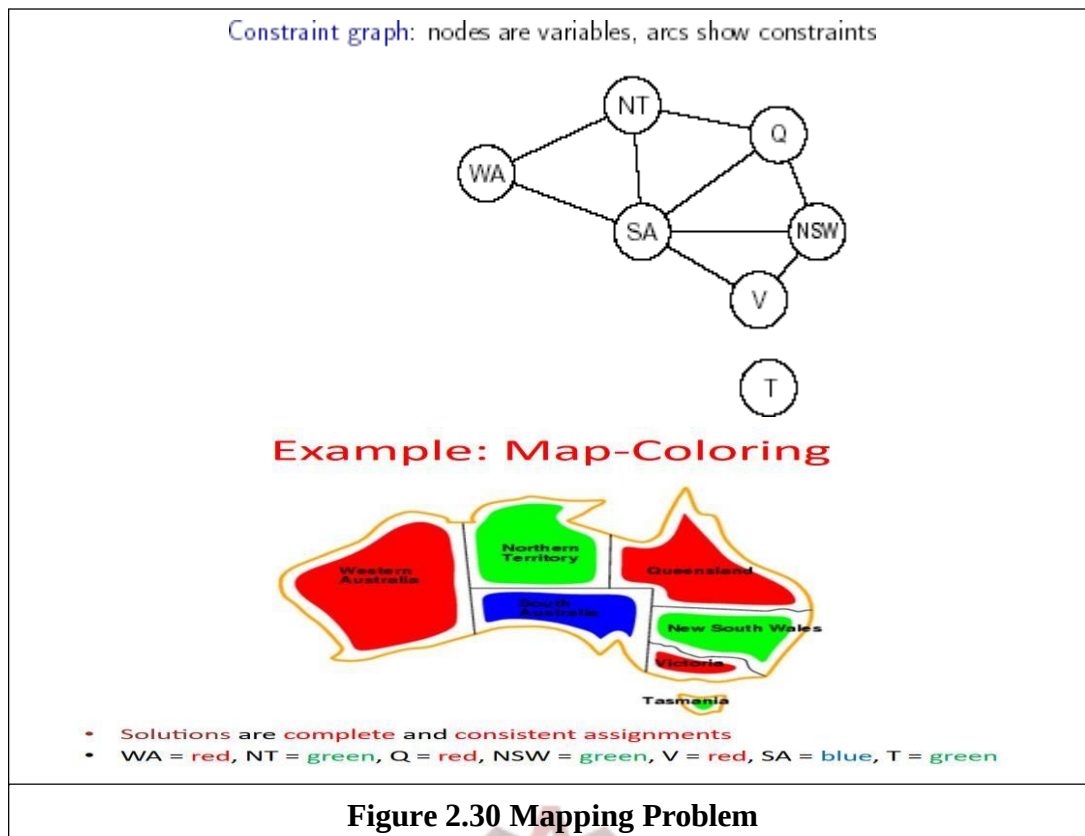
$\{(red,green),(red,blue),(green,red),(green,blue),(blue,red),(blue,green)\}$.

The constraint can also be represented more succinctly as the inequality $WA \neq NT$, provided the constraint satisfaction algorithm has some way to evaluate such expressions.) There are many possible solutions such as

$\{ WA = red, NT = green, Q = red, NSW = green, V = red, SA = blue, T = red \}$.

It is helpful to visualize a CSP as a constraint graph, as shown in Figure 2.29. The nodes of the graph corresponds to variables of the problem and the arcs correspond to constraints.





CSP can be viewed as a standard search problem as follows:

- **Initial state:** the empty assignment {}, in which all variables are unassigned.
- **Successor function:** a value can be assigned to any unassigned variable, provided that it does not conflict with previously assigned variables.
- **Goal test:** the current assignment is complete.
- **Path cost:** a constant cost (E.g., 1) for every step.

Every solution must be a complete assignment and therefore appears at depth n if there are n variables.

Depth first search algorithms are popular for CSPs

Varieties of CSPs

(i) Discrete variables Finite domains

The simplest kind of CSP involves variables that are **discrete** and have **finite domains**. Map coloring problems are of this kind. The 8-queens problem can also be viewed as finite-domain

CSP, where the variables $Q_1, Q_2, \dots, Q_8$ are the positions each queen in columns 1, ..., 8 and each variable has the domain $\{1, 2, 3, 4, 5, 6, 7, 8\}$. If the maximum domain size of any

variable in a CSP is d , then the number of possible complete assignments is $O(d^n)$ – that is, exponential in the number of variables. Finite domain CSPs include **Boolean CSPs**, whose variables can be either *true* or *false*. **Infinite domains**

Discrete variables can also have **infinite domains** – for example, the set of integers or the set of strings. With infinite domains, it is no longer possible to describe constraints by enumerating all allowed combination of values. Instead a constraint language of algebraic inequalities such as $\text{Startjob1} + 5 \leq \text{Startjob3}$.

(ii) CSPs with continuous domains

CSPs with continuous domains are very common in real world. For example in operation research field, the scheduling of experiments on the Hubble Telescope requires very precise timing of observations; the start and finish of each observation and manoeuvre are continuous-valued variables that must obey a variety of astronomical, precedence and power constraints. The best known category of continuous-domain CSPs is that of **linear programming** problems, where the constraints must be linear inequalities forming a *convex* region. Linear programming problems can be solved in time polynomial in the number of variables.

Varieties of constraints

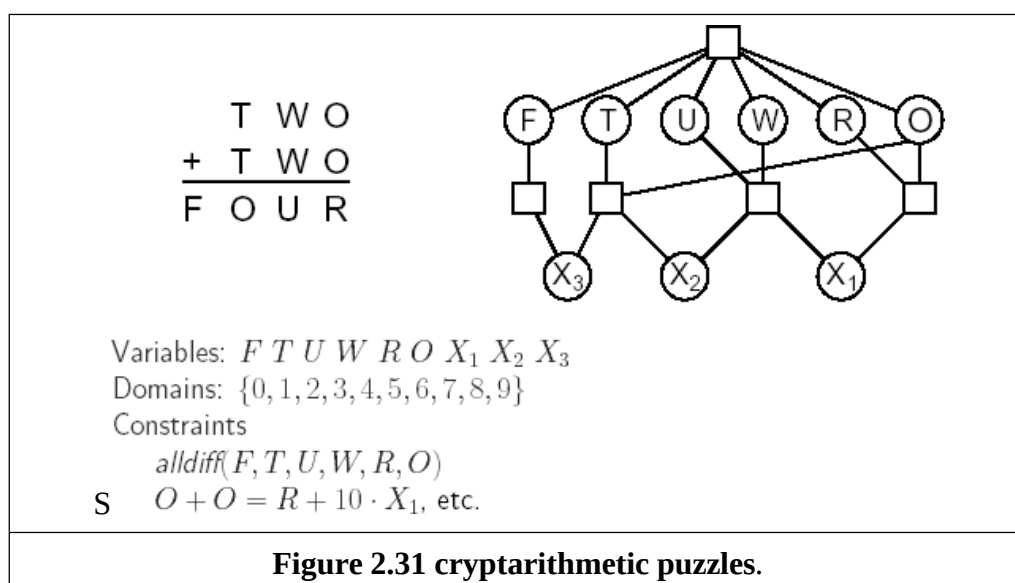
(i) **unary constraints** involve a single variable.

Example : SA # green

(ii) **Binary constraints** involve pairs of variables.

Example : SA # WA

(iii) **Higher order constraints** involve 3 or more variables. Example : cryptarithmic puzzles.

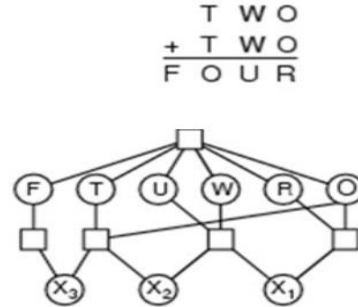


Example: Cryptarithmic

Each **letter** stands for a **distinct digit**; the aim is to find a substitution of digits for letters such that the resulting sum is arithmetically correct, with the added restriction that no leading zeroes are allowed

- **Variables:** $F, T, U, W, R, O, X_1, X_2, X_3$
- **Domains:** $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$
- **Constraints:** $Alldiff(F, T, U, W, R, O)$
- where X_1, X_2 , and X_3 are auxiliary variables representing the digit (0 or 1) carried over into the next column

- $O + O = R + 10 \cdot X_1$
- $X_1 + W + W = U + 10 \cdot X_2$
- $X_2 + T + T = F + 10 \cdot X_3$
- $X_3 = F, T \neq 0, F \neq 0$



The constraint hyper graph for the cryptarithmic problem, showing the *Alldiff* constraint as well as the column addition constraints

Figure 2.32 Cryptarithmic puzzles-Solution

- **Each puzzle may has one or many solutions or no solution**

$$\begin{array}{r} T W O \\ + T W O \\ \hline F O U R \end{array}$$

has 7 solutions:

1. $\begin{array}{r} 734 \\ +734 \\ \hline 1468 \end{array}$ $F=1 \ O=4 \ R=8 \ T=7 \ U=6 \ W=3$
2. $\begin{array}{r} 765 \\ +765 \\ \hline 1530 \end{array}$ $F=1 \ O=5 \ R=0 \ T=7 \ U=3 \ W=6$
3. $\begin{array}{r} 836 \\ +836 \\ \hline 1672 \end{array}$ $F=1 \ O=6 \ R=2 \ T=8 \ U=7 \ W=3$

4. $\begin{array}{r} 846 \\ +846 \\ \hline 1692 \end{array}$ $F=1 \ O=6 \ R=2 \ T=8 \ U=9 \ W=4$
5. $\begin{array}{r} 867 \\ +867 \\ \hline 1734 \end{array}$ $F=1 \ O=7 \ R=4 \ T=8 \ U=3 \ W=6$
6. $\begin{array}{r} 928 \\ +928 \\ \hline 1856 \end{array}$ $F=1 \ O=8 \ R=6 \ T=9 \ U=5 \ W=2$
7. $\begin{array}{r} 938 \\ +938 \\ \hline 1876 \end{array}$ $F=1 \ O=8 \ R=6 \ T=9 \ U=7 \ W=3$

Figure 2.33 Numerical Solution

Backtracking Search for CSPs

The term **backtracking search** is used for depth-first search that chooses values for one variable at a time and backtracks when a variable has no legal values left to assign. The algorithm is shown in figure

```

function BACKTRACKING-SEARCH(csp) returns solution/failure
  return RECURSIVE-BACKTRACKING({ }, csp)

function RECURSIVE-BACKTRACKING(assignment, csp) returns soln/failure
  if assignment is complete then return assignment
  var ← SELECT-UNASSIGNED-VARIABLE(VARIABLES[csp], assignment, csp)
  for each value in ORDER-DOMAIN-VALUES(var, assignment, csp) do
    if value is consistent with assignment given CONSTRAINTS[csp] then
      add { var = value } to assignment
      result ← RECURSIVE-BACKTRACKING(assignment, csp)
      if result ≠ failure then return result
      remove { var = value } from assignment
  return failure

```

Figure 2.34 A simple backtracking algorithm for constraint satisfaction problem. The algorithm is modeled on the recursive depth-first search

Backtracking example

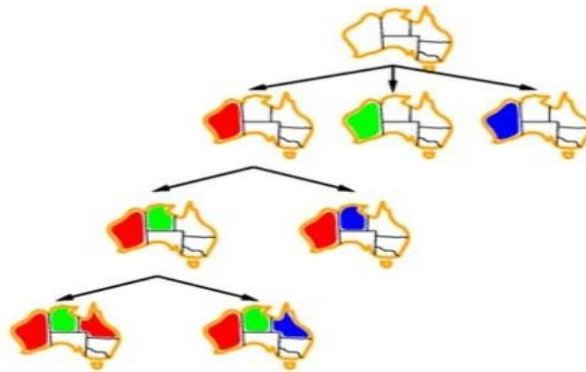


Figure 2.34 Part of the search tree generated by simple backtracking for the map-coloring problem

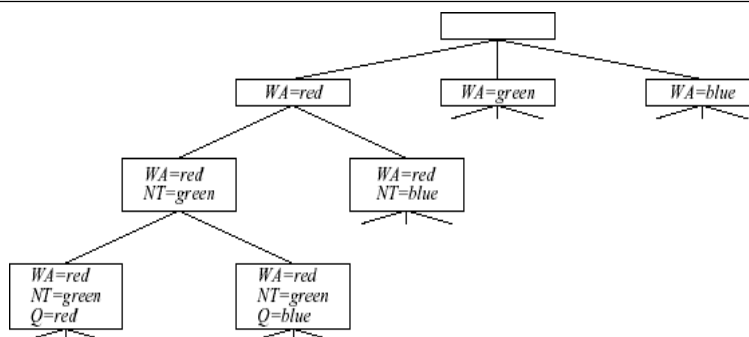


Figure 2.35 Part of search tree generated by simple backtracking for the map coloring problem.

Forward checking

One way to make better use of constraints during search is called forward checking. Whenever a variable X is assigned, the forward checking process looks at each unassigned variable Y that is connected to X by a constraint and deletes from Y 's domain any value that is inconsistent with the value chosen for X . Figure 5.6 shows the progress of a map-coloring search with forward checking.

	<i>WA</i>	<i>NT</i>	<i>Q</i>	<i>NSW</i>	<i>V</i>	<i>SA</i>	<i>T</i>
Initial domains	R G B	R G B	R G B	R G B	R G B	R G B	R G B
After <i>WA=red</i>	Ⓡ	G B	R G B	R G B	R G B	G B	R G B
After <i>Q=green</i>	Ⓡ	B	Ⓢ	R B	R G B	B	R G B
After <i>V=blue</i>	Ⓡ	B	Ⓢ	R	Ⓟ		R G B

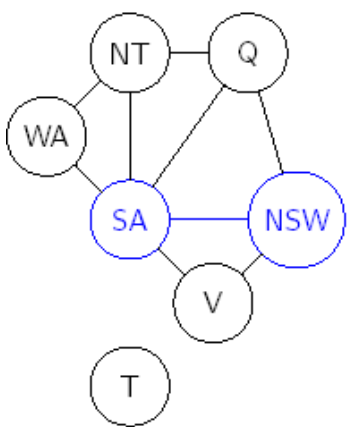
Figure 2.36 The progress of a map-coloring search with forward checking. *WA = red* is assigned first; then forward checking deletes *red* from the domains of the neighboring variables *NT* and *SA*. After *Q = green*, *green* is deleted from the domain of *NT*, *SA*, and *NSW*. After *V = blue*, *blue*, is deleted from the domains of *NSW* and *SA*, leaving *SA* with no legal values.

Constraint propagation

Although forward checking detects many inconsistencies, it does not detect all of them.

Constraint propagation is the general term for propagating the implications of a constraint on one variable onto other variables.

Arc Consistency



- One method of constraint propagation is to enforce **arc consistency**
 - Stronger than forward checking
 - Fast
- Arc refers to a *directed* arc in the constraint graph
- Consider two nodes in the constraint graph (e.g., *SA* and *NSW*)
 - An arc is **consistent** if
 - For every value x of *SA*
 - There is some value y of *NSW* that is consistent with x
- Examine arcs for consistency in *both* directions

Figure: Australian Territories

Figure 2.37 Arc Consistency

- Can define stronger forms of consistency

***k*-Consistency**

A CSP is ***k*-consistent** if, for **any** consistent assignment to $k - 1$ variables, there is a consistent assignment for the k -th variable

- **1-consistency (node consistency)**
 - Each variable by itself is consistent (has a non-empty domain)
- **2-consistency (arc consistency)**
- **3-consistency (path consistency)**
 - Any pair of adjacent variables can be extended to a third

Figure 2.38 Arc Consistency –CSP

k-Consistency

Local Search for CSPs

- Local search algorithms good for many CSPs
- Use complete-state formulation
 - Value assigned to every variable
 - Successor function changes one value at a time
- Have already seen this
 - Hill climbing for 8-queens problem (AIMA § 4.3)
- Choose values using **min-conflicts** heuristic
 - Value that results in the minimum number of conflicts with other variables

The Structure of Problems Problem Structure

- Consider ways in which the structure of the problem's constraint graph can help find solutions
- Real-world problems require decomposition into subproblems

Independent Subproblems

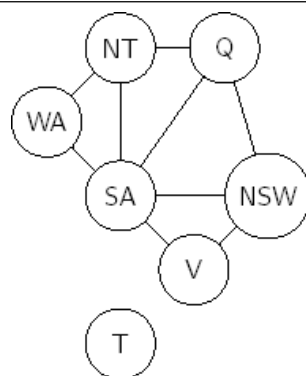
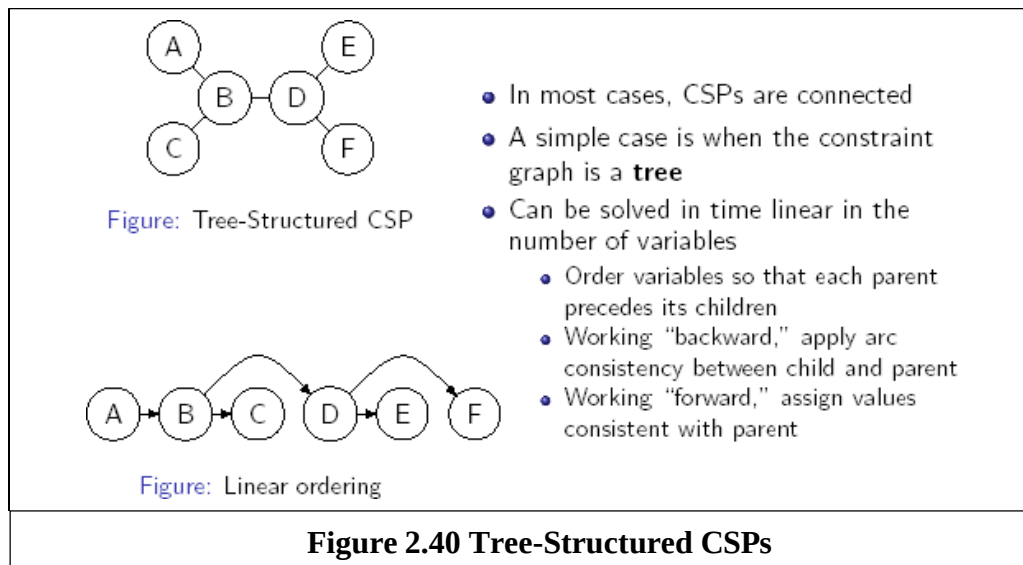


Figure: Australian Territories

- T is not connected
- Coloring T and coloring remaining nodes are **independent subproblems**
- Any solution for T combined with any solution for remaining nodes solves the problem
- Independent subproblems correspond to **connected components** of the constraint graph
- Sadly, such problems are rare

Figure 2.39 Independent Subproblems

Tree-Structured CSPs



2.30 ADVERSARIAL SEARCH

Competitive environments, in which the agent's goals are in conflict, give rise to **adversarial search** problems – often known as **games**.

Games

Mathematical **Game Theory**, a branch of economics, views any **multiagent environment** as a **game** provided that the impact of each agent on the other is "significant", regardless of whether the agents are cooperative or competitive. In, **AI**, "games" are deterministic, turn-taking, two-player, zero-sum games of perfect information. This means deterministic, fully observable environments in which there are two agents whose actions must alternate and in which the **utility values** at the end of the game are always equal and opposite. For example, if one player wins the game of chess(+1), the other player necessarily loses(-1). It is this opposition between the agents' utility functions that makes the situation **adversarial**.

Formal Definition of Game

We will consider games with two players, whom we will call **MAX** and **MIN**. MAX moves first, and then they take turns moving until the game is over. At the end of the game, points are awarded to the winning player and penalties are given to the loser. A **game** can be formally defined as a **search problem** with the following components:

- The **initial state**, which includes the board position and identifies the player to move.
- A **successor function**, which returns a list of (*move*, *state*) pairs, each indicating a legal move and the resulting state.

- A **terminal test**, which describes when the game is over. States where the game has ended are called **terminal states**.
- A **utility function** (also called an objective function or payoff function), which give a numeric value for the terminal states. In chess, the outcome is a win, loss, or draw, with values +1, -1, or 0. The payoffs in backgammon range from +192 to -192.

Game Tree

The **initial state** and **legal moves** for each side define the **game tree** for the game. Figure 2.18 shows the part of the game tree for tic-tac-toe (noughts and crosses). From the initial state, MAX has nine possible moves. Play alternates between MAX's placing an X and MIN's placing a 0 until we reach leaf nodes corresponding to the terminal states such that one player has three in a row or all the squares are filled. The number on each leaf node indicates the utility value of the terminal state from the point of view of MAX; high values are assumed to be good for MAX and bad for MIN. It is the MAX's job to use the search tree (particularly the utility of terminal states) to determine the best move.

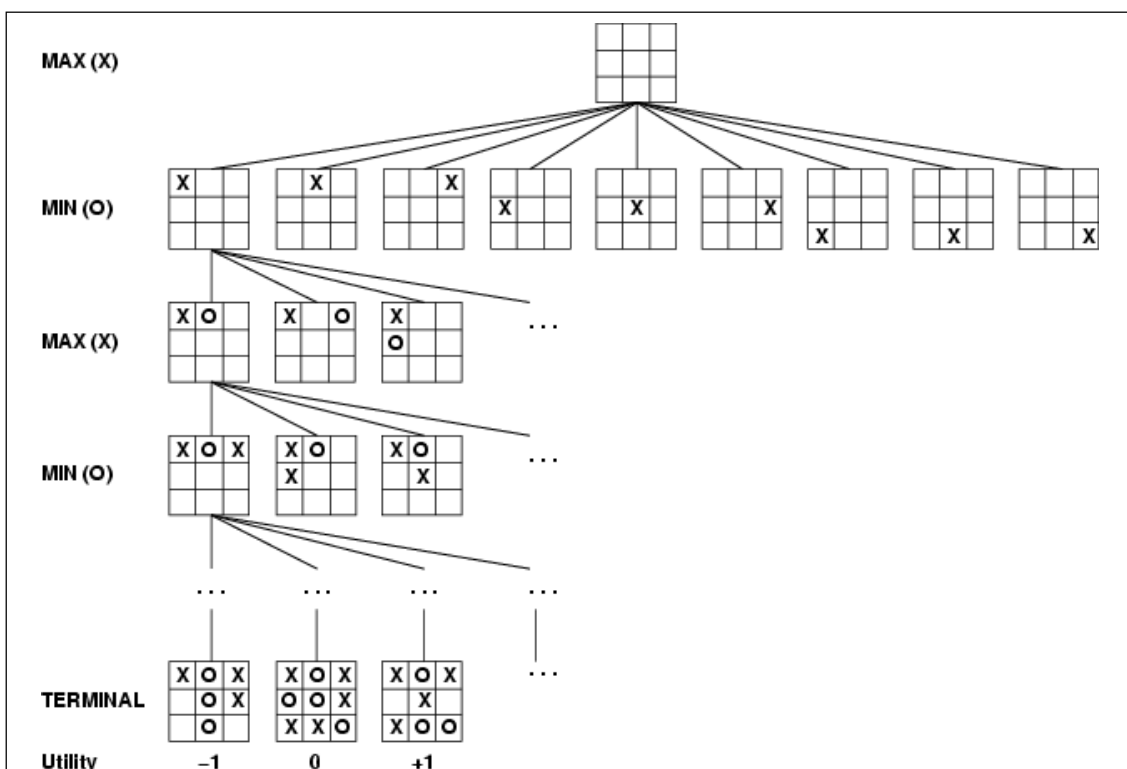
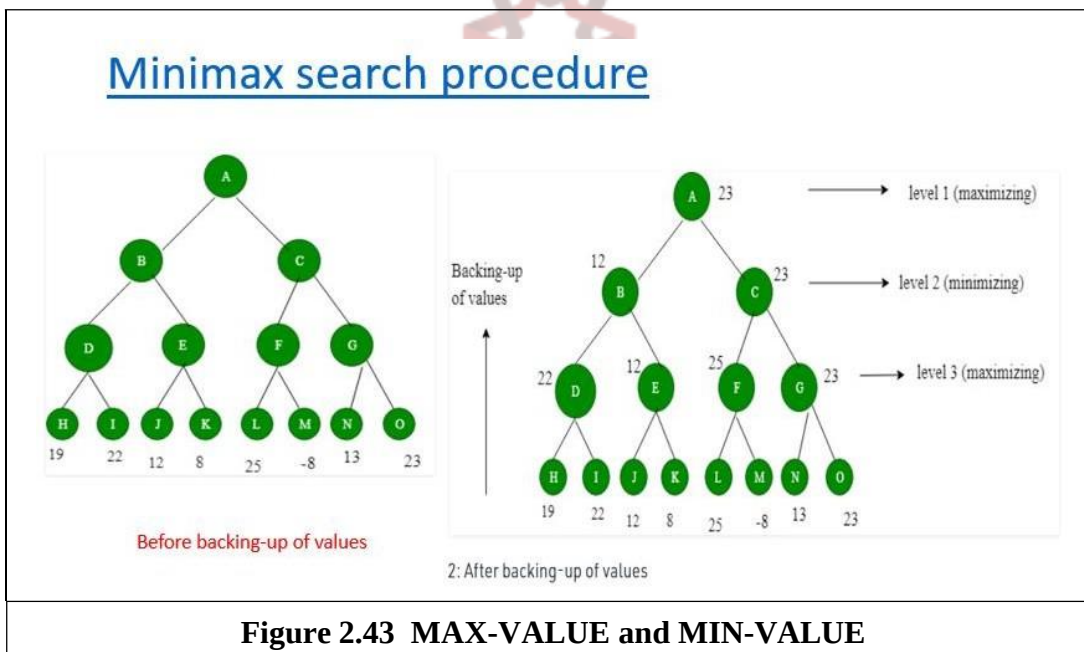
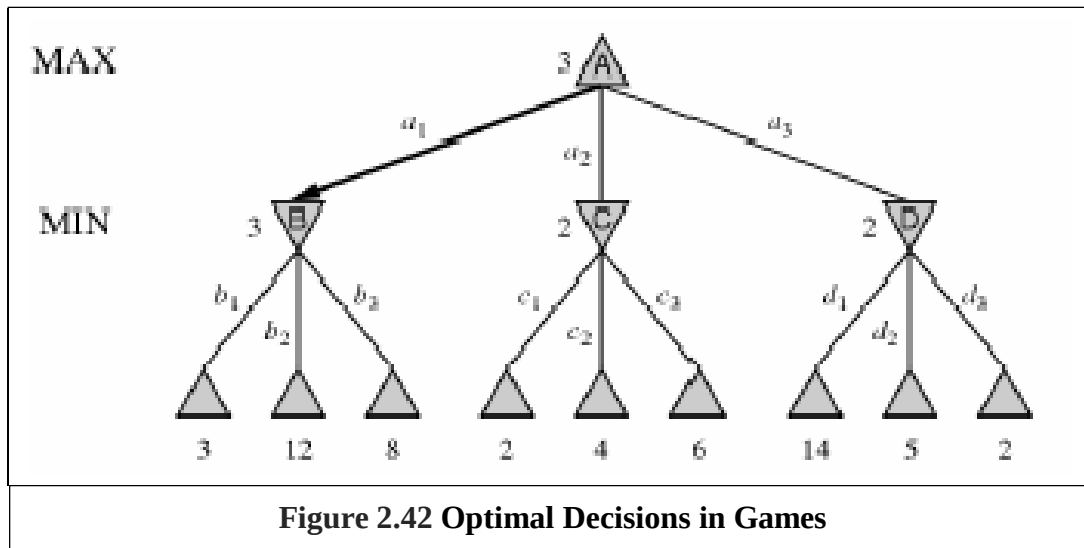


Figure 2.41 A partial search tree. The top node is the initial state, and MAX move first, placing an X in an empty square.

Optimal Decisions in Games

In normal search problem, the **optimal solution** would be a sequence of move leading to a **goal state** – a terminal state that is a win. In a game, on the other hand, MIN has something

to say about it, MAX therefore must find a contingent **strategy**, which specifies MAX's move in the **initial state**, then MAX's moves in the states resulting from every possible response by MIN, then MAX's moves in the states resulting from every possible response by MIN those moves, and so on. An **optimal strategy** leads to outcomes at least as good as any other strategy when one is playing an infallible opponent.



Minimax Search: Algorithm

```
function MINIMAX-DECISION(state) returns an action
inputs: state, current state in game
```

```
   $v \leftarrow \text{MAX-VALUE}(\text{state})$ 
  return the action in SUCCESSORS(state) with value  $v$ 
```

```
function MAX-VALUE(state) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
   $v \leftarrow -\infty$ 
  for  $a, s$  in SUCCESSORS(state) do
     $v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(s))$ 
  return  $v$ 
```

For MAX Node

```
function MIN-VALUE(state) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
   $v \leftarrow \infty$ 
  for  $a, s$  in SUCCESSORS(state) do
     $v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(s))$ 
  return  $v$ 
```

For MIN Node

Figure 2.44 An algorithm for calculating minimax decisions. It returns the action corresponding to the best possible move, that is, the move that leads to the outcome with the best utility, under the assumption that the opponent plays to minimize utility. The functions MAX-VALUE and MIN-VALUE go through the whole game tree, all the way to the leaves, to determine the backed-up value of a state.

The minimax Algorithm

The minimax algorithm computes the minimax decision from the current state. It uses a simple recursive computation of the minimax values of each successor state, directly implementing the defining equations. The recursion proceeds all the way down to the leaves of the tree, and then the minimax values are **backed up** through the tree as the recursion unwinds. For example in Figure 2.19, the algorithm first recurses down to the three bottom left nodes, and uses the utility function on them to discover that their values are 3, 12, and 8 respectively. Then it takes the minimum of these values, 3, and returns it as the backed-up value of node B. A similar process gives the backed up values of 2 for C and 2 for D. Finally, we take the maximum of 3, 2, and 2 to get the backed-up value of 3 at the root node. The minimax algorithm performs a complete depth-first exploration of the game tree. If the maximum depth of the tree is m , and there are b legal moves at each point, then the time complexity of the minimax algorithm is $O(b^m)$. The space complexity is $O(bm)$ for an algorithm that generates successors at once.

Alpha-Beta Pruning

The problem with minimax search is that the number of game states it has to examine is **exponential** in the number of moves. Unfortunately, we can't eliminate the exponent, but

we can effectively cut it in half. By performing **pruning**, we can eliminate large part of the tree from consideration. We can apply the technique known as **alpha beta pruning**, when applied to a minimax tree, it returns the same move as **minimax** would, but **prunes away** branches that cannot possibly influence the final decision.

Alpha Beta pruning gets its name from the following two parameters that describe bounds on the backed-up values that appear anywhere along the path:

- α : the value of the best (i.e., highest-value) choice we have found so far at any choice point along the path of MAX.
- β : the value of best (i.e., lowest-value) choice we have found so far at any choice point along the path of MIN.

Alpha Beta search updates the values of α and β as it goes along and prunes the remaining branches at a node (i.e., terminates the recursive call) as soon as the value of the current node is known to be worse than the current α and β value for MAX and MIN, respectively. The complete algorithm is given in Figure. The effectiveness of alpha-beta pruning is highly dependent on the order in which the successors are examined. It might be worthwhile to try to examine first the successors that are likely to be the best. In such case, it turns out that alpha-beta needs to examine only $O(b^{d/2})$ nodes to pick the best move, instead of $O(b^d)$ for minimax. This means that the effective branching factor becomes $\sqrt{b}$ instead of b – for chess, 6 instead of 35. Put another way alpha-beta can look ahead roughly twice as far as minimax in the same amount of time.

```

function ALPHA-BETA-SEARCH(state) returns an action
  inputs: state, current state in game
   $v \leftarrow \text{MAX-VALUE}(\text{state}, -\infty, +\infty)$ 
  return the action in SUCCESSORS(state) with value  $v$ 



---


function MAX-VALUE(state,  $\alpha$ ,  $\beta$ ) returns a utility value
  inputs: state, current state in game
            $\alpha$ , the value of the best alternative for MAX along the path to state
            $\beta$ , the value of the best alternative for MIN along the path to state

  if TERMINAL-TEST(state) then return UTILITY(state)
   $v \leftarrow -\infty$ 
  for  $a, s$  in SUCCESSORS(state) do
     $v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(s, \alpha, \beta))$ 
    if  $v \geq \beta$  then return  $v$ 
     $\alpha \leftarrow \text{MAX}(\alpha, v)$ 
  return  $v$ 

```

```

function MIN-VALUE(state,  $\alpha$ ,  $\beta$ ) returns a utility value
  inputs: state, current state in game
            $\alpha$ , the value of the best alternative for MAX along the path to state
            $\beta$ , the value of the best alternative for MIN along the path to state

  if TERMINAL-TEST(state) then return UTILITY(state)
   $v \leftarrow +\infty$ 
  for a, s in SUCCESSORS(state) do
     $v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(s, \alpha, \beta))$ 
    if  $v \leq \alpha$  then return  $v$ 
     $\beta \leftarrow \text{MIN}(\beta, v)$ 
  return  $v$ 

```

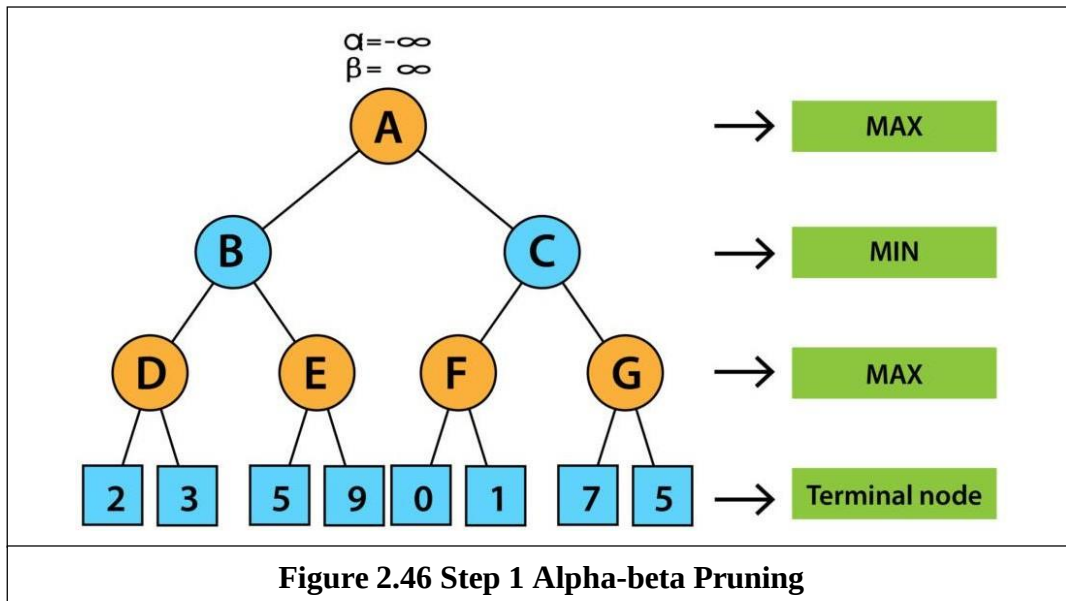
Figure 2.45 The alpha beta search algorithm. These routines are the same as the minimax routines in figure 2.20, except for the two lines in each of MIN-VALUE and MAX-VALUE that maintain α and β

Key points in Alpha-beta Pruning

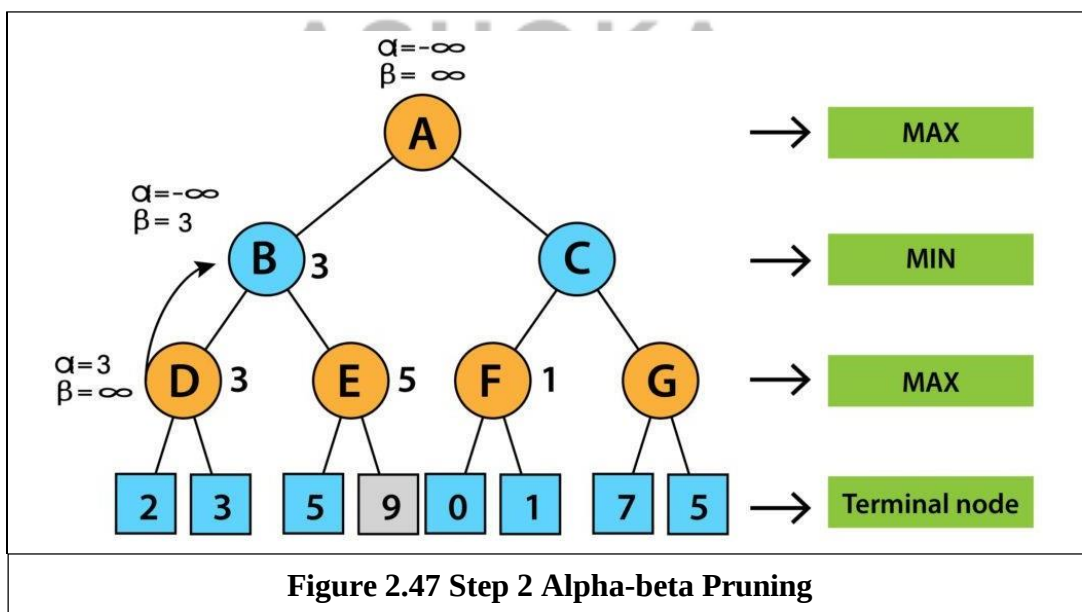
- Alpha: Alpha is the best choice or the highest value that we have found at any instance along the path of Maximizer. The initial value for alpha is $-\infty$.
- Beta: Beta is the best choice or the lowest value that we have found at any instance along the path of Minimizer. The initial value for alpha is $+\infty$.
- The condition for Alpha-beta Pruning is that $\alpha \geq \beta$.
- Each node has to keep track of its alpha and beta values. Alpha can be updated only when it's MAX's turn and, similarly, beta can be updated only when it's MIN's chance.
- MAX will update only alpha values and MIN player will update only beta values.
- The node values will be passed to upper nodes instead of values of alpha and beta during go into reverse of tree.
- Alpha and Beta values only be passed to child nodes.

Working of Alpha-beta Pruning

1. We will first start with the initial move. We will initially define the alpha and beta values as the worst case i.e. $\alpha = -\infty$ and $\beta = +\infty$. We will prune the node only when alpha becomes greater than or equal to beta.

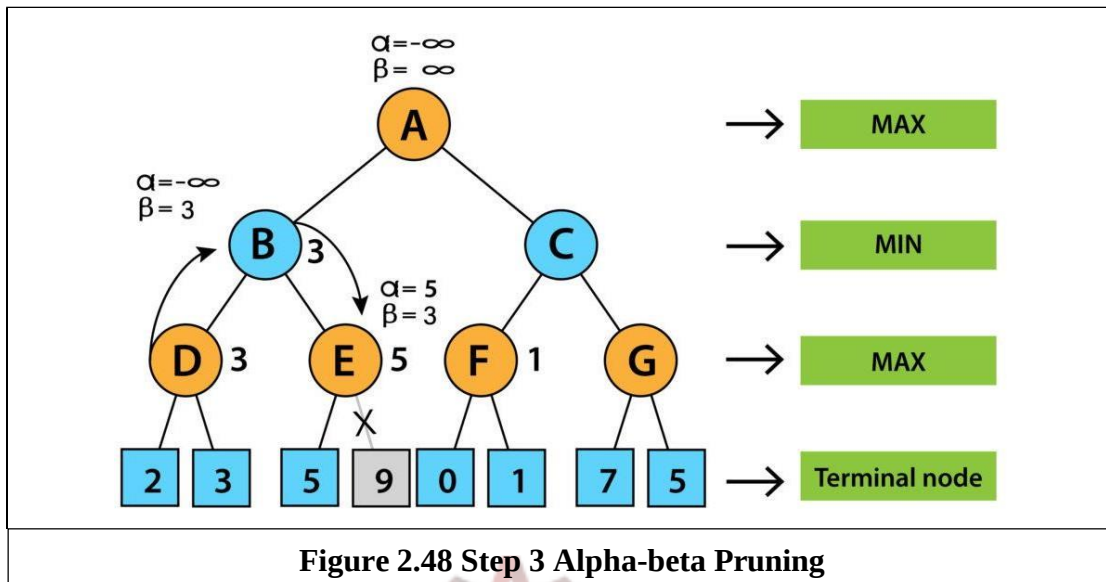


2. Since the initial value of alpha is less than beta so we didn't prune it. Now it's turn for MAX. So, at node D, value of alpha will be calculated. The value of alpha at node D will be max (2, 3). So, value of alpha at node D will be 3.
3. Now the next move will be on node B and its turn for MIN now. So, at node B, the value of alpha beta will be min (3, ∞). So, at node B values will be $\alpha = -\infty$ and beta will be 3.

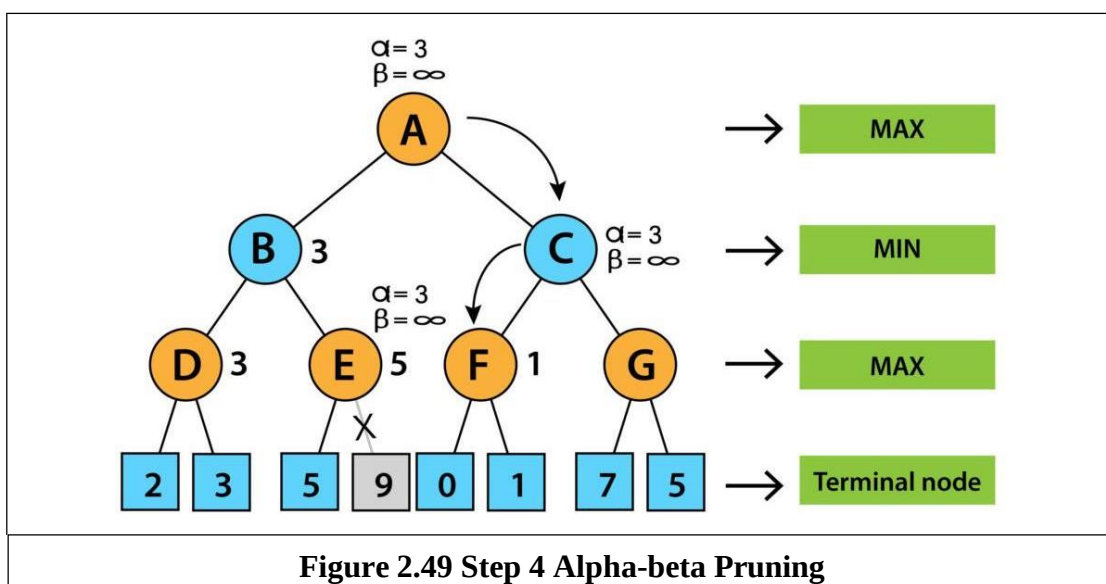


In the next step, algorithms traverse the next successor of Node B which is node E, and the values of $\alpha = -\infty$, and $\beta = 3$ will also be passed.

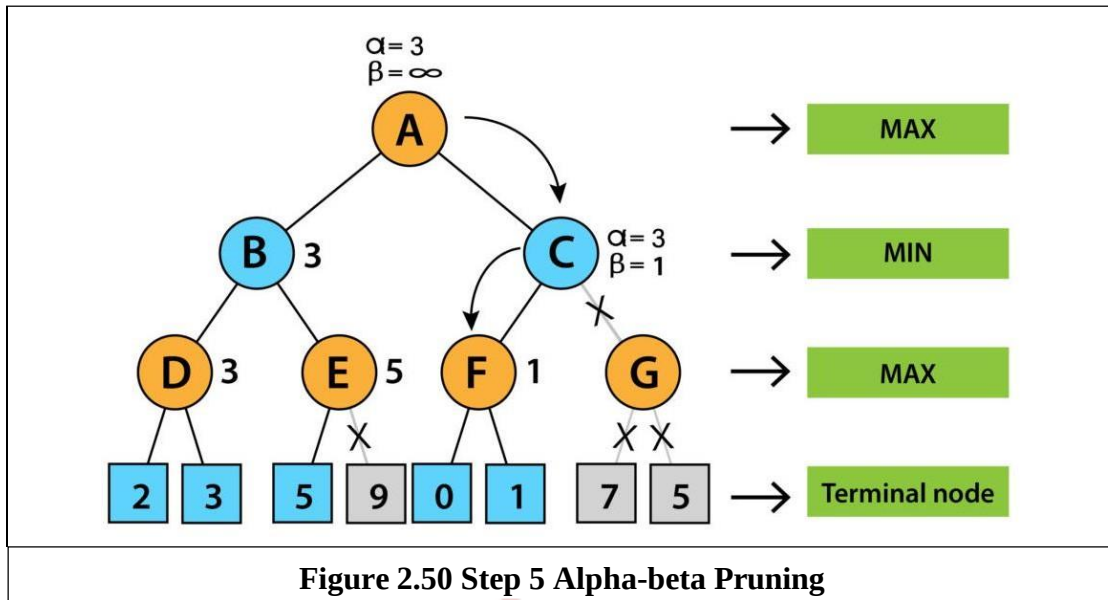
4. Now it's turn for MAX. So, at node E we will look for MAX. The current value of alpha at E is $-\infty$ and it will be compared with 5. So, $\text{MAX}(-\infty, 5)$ will be 5. So, at node E, $\alpha = 5$, $\beta = 5$. Now as we can see that alpha is greater than beta which is satisfying the pruning condition so we can prune the right successor of node E and algorithm will not be traversed and the value at node E will be 5.



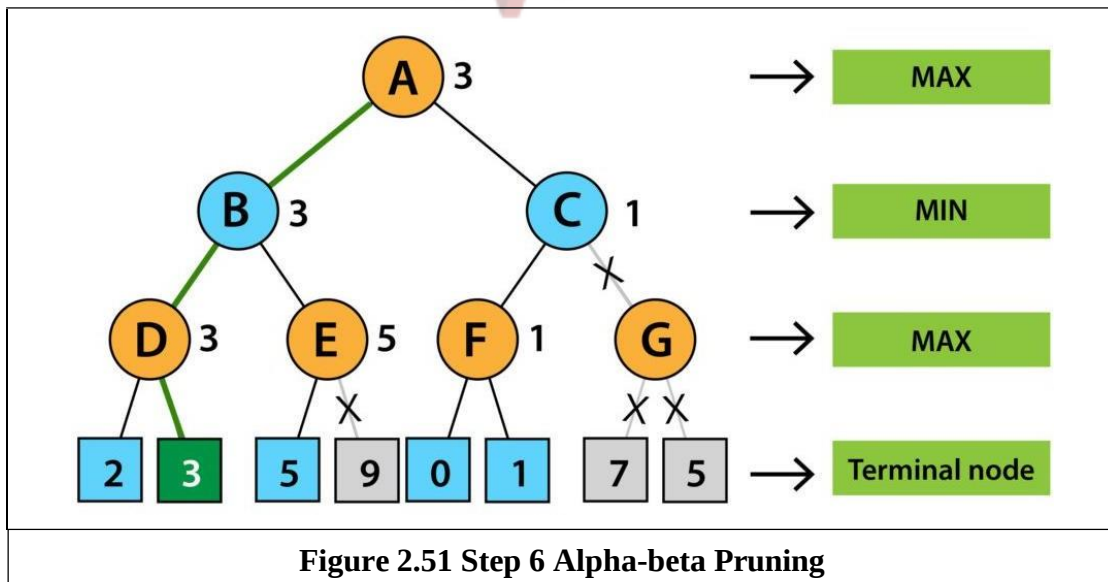
6. In the next step the algorithm again comes to node A from node B. At node A alpha will be changed to maximum value as $\text{MAX}(-\infty, 3)$. So now the value of alpha and beta at node A will be $(3, +\infty)$ respectively and will be transferred to node C. These same values will be transferred to node F.
7. At node F the value of alpha will be compared to the left branch which is 0. So, $\text{MAX}(0, 3)$ will be 3 and then compared with the right child which is 1, and $\text{MAX}(3, 1) = 3$ still α remains 3, but the node value of F will become 1.



8. Now node F will return the node value 1 to C and will compare to beta value at C. Now its turn for MIN. So, $\text{MIN}(+\infty, 1)$ will be 1. Now at node C, $\alpha = 3$, and $\beta = 1$ and alpha is greater than beta which again satisfies the pruning condition. So, the next successor of node C i.e. G will be pruned and the algorithm didn't compute the entire subtree G.



Now, C will return the node value to A and the best value of A will be $\text{MAX}(1, 3)$ will be 3.



The above represented tree is the final tree which is showing the nodes which are computed and the nodes which are not computed. So, for this example the optimal value of the maximizer will be 3.

UNIT-III

Representation of Knowledge

Representation of Knowledge: Knowledge representation issues, predicate logic- logic programming, semantic nets- frames and inheritance, constraint propagation, representing knowledge using rules, rules based deduction systems, reasoning under uncertainty, review of probability, Bayes' probabilistic interferences and Dempster-Shafer theory.

1. Important Issues in Knowledge Representation

When representing knowledge in AI systems, the following key issues arise:

a. Representational Adequacy

- **Definition:** The ability of the knowledge representation system to represent all the knowledge that is relevant for solving a problem.
- **Challenge:** AI systems need to capture the nuances of the real world, which includes different types of knowledge like factual knowledge, procedural knowledge, and knowledge about the relationships between concepts.

b. Inferential Adequacy

- **Definition:** The ability of the representation system to allow inferences or new knowledge derivation based on existing knowledge.
- **Challenge:** AI systems should not only store knowledge but also make logical deductions or conclusions from it. For instance, the system should infer that "if a person is a mother, then they must be a parent."

c. Inferential Efficiency

- **Definition:** The ability of the system to make inferences efficiently.
- **Challenge:** Even if a system can represent knowledge and make inferences, doing so efficiently in terms of time and resources is critical. Some representation methods may lead to slow or inefficient reasoning processes, particularly when dealing with large knowledge bases.

d. Acquisitional Efficiency

- **Definition:** The ease with which new knowledge can be added to the system.
- **Challenge:** AI systems are dynamic, and knowledge is continuously evolving. A good representation system allows easy updating and modification of knowledge without disrupting the existing structure or requiring excessive computational overhead.

2. Types of Knowledge to Represent

AI systems deal with various forms of knowledge, and each requires specific representation techniques:

a. Declarative Knowledge

- **Definition:** Knowledge about facts, such as "Paris is the capital of France."



- **Example Representations:**
 - **Propositional logic:** Uses true/false statements.
 - **Predicate logic:** More expressive, representing relationships (e.g., *Capital(Paris, France)*).

b. Procedural Knowledge

- **Definition:** Knowledge about how to perform tasks or operations, such as the steps to solve a math problem.
- **Challenge:** Procedural knowledge is harder to represent because it often involves dynamic processes, such as sequences of actions or decision-making steps.

c. Meta-Knowledge

- **Definition:** Knowledge about knowledge itself, such as understanding the limits of one's knowledge or knowing how to use specific problem-solving strategies.
- **Challenge:** Representing this reflective type of knowledge requires advanced frameworks that can model reasoning about knowledge and beliefs.

3. Common Methods of Knowledge Representation

The book discusses several popular methods for representing knowledge in AI systems:

a. Logical Representation (Predicate Logic)

- **Description:** Based on formal logic, logical representation uses symbols and logical operators to represent facts and relationships between entities.
- **Strengths:** Allows clear and unambiguous representation of knowledge, and supports rigorous inference mechanisms.
- **Weaknesses:** Can be computationally expensive, and not all types of knowledge (e.g., common sense reasoning) can be easily represented.

b. Semantic Networks

- **Description:** A graphical representation of knowledge where concepts are represented as nodes and relationships between them as edges (e.g., *Dog is-a Animal*).
- **Strengths:** Intuitive, visual, and efficient for representing hierarchical relationships.
- **Weaknesses:** Can be inefficient when handling complex relationships and large networks.

c. Frames

- **Description:** Frames are data structures that represent stereotypical situations or entities. A frame has slots (attributes) and slot values (details or characteristics).
- **Example:** A "house" frame might include slots for *Location*, *Size*, *Owner*, etc.
- **Strengths:** Effective for representing structured knowledge about entities or situations.
- **Weaknesses:** Difficult to represent dynamic processes and changing states.

d. Production Rules

- **Description:** If-then rules that specify actions to be taken when certain conditions are met.
- **Example:** *If (it is raining), then (carry an umbrella).*
- **Strengths:** Easy to implement and understand, good for procedural knowledge.
- **Weaknesses:** Can lead to inefficiency and difficulty in managing large sets of rules.

e. Scripts

- **Description:** Predefined sequences of actions or events that describe typical scenarios.
- **Example:** A "restaurant script" might include actions like being seated, ordering food, eating, and paying the bill.
- **Strengths:** Good for representing knowledge about routine activities.
- **Weaknesses:** Limited in handling flexible or unusual situations.

4. Challenges in Knowledge Representation

Several challenges arise when designing and implementing knowledge representation systems:

a. Ambiguity

- Natural language and human understanding of concepts are often ambiguous. AI systems need clear and precise representations to avoid confusion.

b. Completeness

- It's often impossible to represent all knowledge about a domain comprehensively. Representation systems must balance between being sufficiently expressive and practical.

c. Scalability

- Knowledge bases grow over time. The representation method must be able to scale with the volume of knowledge while maintaining inferential efficiency.

d. Commonsense Knowledge

- Representing everyday commonsense knowledge is notoriously difficult because it often involves understanding complex contextual and background information.

e. Contextual Knowledge

- Knowledge may be context-dependent. AI systems need to represent and reason about the context in which a particular piece of knowledge is valid.

5. Trade-offs in Knowledge Representation

- **Expressiveness vs. Efficiency:** More expressive representations (e.g., first-order logic) allow for more complex knowledge but often result in computationally expensive inference procedures. Simpler representations (e.g., propositional logic) are faster but less capable of expressing detailed knowledge.
- **Explicit vs. Implicit Knowledge:** Some systems store knowledge explicitly, while others infer it when needed. The trade-off lies in whether the system should store all details or generate them dynamically, balancing memory and computation efficiency.

Predicate logic- logic programming

Predicate Logic

Predicate logic is an extension of propositional logic, which allows reasoning about objects and their relationships. It provides a more expressive framework than propositional logic by introducing the use of **quantifiers**, **variables**, and **relations**.

1. Elements of Predicate Logic:

- **Terms:** Represent objects or entities in the domain. They can be constants (e.g., John, Apple) or variables (e.g., x , y).
- **Predicates:** Represent relationships between objects or properties of objects. Predicates take one or more terms as arguments (e.g., Likes(John, Apple) means John likes an apple).
- **Quantifiers:**
 - **Universal quantifier ($\forall$):** Indicates that a statement applies to all elements in the domain. Example: $\forall x P(x)$ means "P(x) is true for all x."
 - **Existential quantifier ($\exists$):** Indicates that a statement applies to at least one element in the domain. Example: $\exists x P(x)$ means "There exists an x such that P(x) is true."

2. Syntax of Predicate Logic:

- **Atomic Sentences:** Represent basic facts about objects. Example: Loves(John, Mary).
- **Logical Connectives:** Combine atomic sentences into complex expressions. These include AND ($\wedge$), OR ($\vee$), NOT ($\neg$), IMPLIES ($\rightarrow$), and BICONDITIONAL ($\leftrightarrow$).

3. Example:

Consider a domain where you are trying to describe the relationships between people:

- **Predicate Example:** Parent(John, Mary) means "John is a parent of Mary."
- **Quantifier Example:**
 - $\forall x \text{ Parent}(x, \text{Mary})$ means "For all x, x is a parent of Mary."
 - $\exists x \text{ Parent}(x, \text{Mary})$ means "There exists someone who is a parent of Mary."

4. Inference in Predicate Logic:

- **Modus Ponens:** If $A \rightarrow B$ and A are both true, then B must be true.

- **Unification:** Matching variables and constants between different predicates to derive new facts. For instance, if you have Loves(x, y) and Loves(John, y), unification will bind xxx to John.
-

Logic Programming

Logic programming is a paradigm based on formal logic, particularly predicate logic. It focuses on using logical statements to represent knowledge and derive conclusions.

1. Declarative Nature:

- Logic programming is **declarative**, meaning that the programmer specifies **what** the program should accomplish, rather than **how** to accomplish it. The underlying system (e.g., Prolog) determines how to solve the problem.

2. Prolog:

Prolog (Programming in Logic) is a popular language used in logic programming, grounded in first-order predicate logic.

Example of Prolog Syntax:

Facts: Basic facts are written as atomic predicates. For example:

parent(john, mary).

parent(mary, susan).

These are facts stating that John is a parent of Mary, and Mary is a parent of Susan.

Rules: Logical rules are written as implications. For example:

grandparent(X, Y) :- parent(X, Z), parent(Z, Y).

This rule states that X is a grandparent of Y if X is a parent of Z, and Z is a parent of Y.

Queries: You can ask questions about the knowledge base:

?- grandparent(john, susan).

1. Prolog will respond `true` if it can find a way to satisfy the query, based on the facts and rules provided.

Unification and Backtracking:

- **Unification:** The process of matching terms, predicates, and variables during logic execution. Prolog automatically unifies variables and constants when searching for solutions.
- **Backtracking:** If a solution path does not work, Prolog systematically tries other possibilities. This is key to how Prolog finds solutions to queries.

Example of Logic Programming in Prolog:

Let's take a scenario where you want to encode family relationships.

- **Facts:**

parent(amy, bob).

parent(bob, carol).

parent(amy, dave).

- **Rules:**

grandparent(X, Y) :- parent(X, Z), parent(Z, Y).

- **Query:**

?- grandparent(amy, carol).

Prolog would evaluate this by checking whether there exists an intermediate ZZZ (a child of Amy) who is also a parent of Carol, and would return true.

Applications of Predicate Logic and Logic Programming:

- **Knowledge Representation and Reasoning:** Used to represent knowledge in AI systems and reason about that knowledge to draw conclusions.
- **Expert Systems:** Logic programming is used to encode rules and facts in expert systems, which reason about specific domains, like medical diagnosis or legal reasoning.
- **Natural Language Processing (NLP):** Logic programming helps in parsing sentences, interpreting meanings, and constructing structured representations of natural language.
- **Problem Solving:** Automated theorem proving, constraint satisfaction, and puzzles can be tackled using logic programming.

Semantic nets- frames and inheritance

Semantic Networks (Semantic Nets)

Semantic Networks are a knowledge representation scheme used to encode information in a graph structure. In this structure, **nodes** represent objects, concepts, or entities, and **edges** represent relationships or associations between them.

Key Features:

- **Nodes:** Represent objects, concepts, or attributes.
- **Links/Edges:** Represent the relationships between these nodes.
- **Directed Graphs:** Relationships in semantic nets are often directional, meaning they specify the direction of the relationship.

Example:

Consider a network that represents the concept of a **dog**:

- **Dog** might be connected to **Animal** through an “is-a” link (indicating that a dog is a type of animal).
- **Dog** might have links to other nodes like **Barks** (indicating that dogs bark) and **Has-fur** (indicating that dogs have fur).

[Dog] ----> [Animal] (is-a relationship)

[Dog] ----> [Barks] (property of dog)

[Dog] ----> [Has-fur] (property of dog)

Frames

Frames are another form of structured knowledge representation, similar to records or objects in programming. A frame consists of a collection of **slots** (attributes) and **values** (the data or information for each slot).

Each frame can represent a situation, concept, or object in the world, and it includes:

- **Slots** (attributes or properties)
- **Facets** (constraints, conditions, or additional information about the slots)

Key Features:

- **Slots:** Represent the attributes or properties of an object or concept.
- **Values:** Contain the data or information for that slot.
- **Inheritance:** Frames can inherit properties from other frames. For example, a frame representing a specific dog might inherit attributes from a more general frame like “Dog,” which in turn might inherit from a more general frame like “Animal.”

Example of a Frame:

Let’s consider a frame for a **Dog**:

Frame: Dog

Slot: Type-of-animal Value: Mammal

Slot: Sound Value: Barks

Slot: Movement Value: Walks, Runs

Slot: Fur Value: Yes

Now, a specific dog like Max might inherit from the “Dog” frame:

Frame: Max (inherits from Dog)

Slot: Name Value: Max

Slot: Color Value: Brown

Here, Max automatically inherits that it is a mammal, that it barks, and that it has fur from the general Dog frame.

Inheritance in Semantic Nets and Frames

Inheritance allows objects or concepts to inherit properties from more general categories. This mechanism reduces redundancy in representation and helps in maintaining consistency.

Two Types of Inheritance:

1. **Direct Inheritance:** An object directly inherits properties from its parent (superclass or frame above).
2. **Multiple Inheritance:** An object can inherit properties from multiple parent objects.

Example of Inheritance in a Semantic Net:

Consider a hierarchy where **Bird** is a type of **Animal** and **Penguin** is a type of **Bird**.

[Animal] ---> [Bird] ---> [Penguin]

(flies) (does not fly)

- All **Birds** inherit the property of "flies" from the **Animal** category.
- However, the **Penguin** overrides this property by specifying "does not fly."

Example of Inheritance in Frames:

Frame: Animal

Slot: Movement Value: Walks

Slot: Sound Value: Varies

Frame: Bird (inherits from Animal)

Slot: Movement Value: Flies

Slot: Sound Value: Chirps

Frame: Penguin (inherits from Bird)

Slot: Movement Value: Swims (overrides)

Here, the **Penguin** frame inherits from **Bird**, which in turn inherits from **Animal**, but **Penguin** overrides the "Movement" slot to specify that it swims instead of flying.

Strengths of Semantic Nets and Frames:

- **Compact Representation:** Inheritance reduces redundancy and organizes information hierarchically.
- **Expressiveness:** Semantic nets and frames are capable of representing rich relationships and properties.
- **Intuitiveness:** These structures often align well with human thinking, making them easier to understand and design.

Challenges:

- **Ambiguity and Conflicts:** Inheritance may sometimes cause conflicts (e.g., what happens when a property is inherited but redefined in a specific subclass?).
- **Complexity:** In large knowledge bases, managing inheritance and updates can become complex.

Constraint propagation

Constraint Propagation is a concept in artificial intelligence (AI) used to solve problems by reducing the possible values for variables based on constraints. This method is especially useful in constraint satisfaction problems (CSPs), where variables are assigned values that must satisfy certain constraints. The technique systematically narrows down the set of possible solutions by "propagating" constraints across variables to eliminate inconsistent or invalid choices.

What is Constraint Propagation?

In a **constraint satisfaction problem (CSP)**, you have:

1. A set of variables, $X_1, X_2, \dots, X_n$
2. Each variable X_i has a domain of possible values D_i
3. A set of constraints C , where each constraint restricts the values that some subset of variables can simultaneously take.

Constraint propagation refers to the process of **inferring and enforcing constraints** to reduce the number of potential values in the domain of variables. It does this by applying the constraints iteratively, gradually reducing the domains until:

- All variables are assigned values (solution found)
- No solution exists (inconsistencies are detected)

How Constraint Propagation Works:

1. **Initial Constraints:** The system begins with a set of constraints that are applied to all the variables.
2. **Domain Reduction:** For each variable, based on the constraints, invalid values are removed from the domain.
3. **Propagation:** The effects of the constraint on one variable propagate to other related variables, further reducing their domains.
4. **Consistency Checking:** The algorithm continuously checks if the remaining values in the domains can satisfy the constraints. If not, further propagation is done until either a solution is found or an inconsistency is detected.

Types of Constraint Propagation:

1. **Node Consistency:** A variable is **node consistent** if all the values in its domain satisfy the unary constraints (constraints that only involve one variable). This is a simple form of constraint propagation.

Example: If X is a variable with domain $\{1, 2, 3\}$ and a unary constraint $X > 1$, constraint propagation will remove 1 from X 's domain, leaving $\{2, 3\}$.

2. **Arc Consistency:** A variable X is **arc consistent** with another variable Y if, for every value of X , there is some value of Y that satisfies the binary constraint between X and Y . Arc consistency is the most common form of constraint propagation in CSPs.

Example: Suppose we have two variables:

- X with domain $\{1, 2, 3\}$
- Y with domain $\{2, 3\}$
- Constraint: $X > Y$

After applying arc consistency:

- For $X=1$, no valid Y satisfies $X>Y$, so remove 1 from X 's domain.
- For $X=2$, only $Y=2$ satisfies $X>Y$, so $Y=2$ remains valid.
- $X=3$ is compatible with both $Y=2$ and $Y=3$.

The updated domains are:

- $X = \{2,3\}$
 - $Y = \{2\}$ (since $Y=3$ doesn't work with any X)
3. **Path Consistency:** A set of variables is **path consistent** if, for every pair of variables X and Y , there exists a third variable Z that ensures consistency along the path formed by X, Y, Z . This form of propagation is more global in scope, as it checks constraints across multiple variables.

Example: Suppose we have three variables:

- X with domain $\{1,2\}$
- Y with domain $\{2,3\}$
- Z with domain $\{1,3\}$

Constraints:

- $X < Y$
- $Y < Z$

If $X=1$, $Y=2$, and $Z=3$, the system is path consistent because $X < Y < Z$ holds along the path. Path consistency ensures that such relationships across multiple variables hold, which may not be captured by arc consistency alone.

4. **K-Consistency:** A CSP is **k-consistent** if, given any set of $k-1$ variables, a consistent value can be assigned to a k -th variable. Higher-order consistencies like 3-consistency (also known as **strong consistency**) are important for solving more complex problems.

Example of Constraint Propagation: Sudoku Puzzle

Sudoku can be solved using constraint propagation. In Sudoku:

- Variables: Each empty cell in the grid
- Domains: The values $\{1,2,\dots,9\}$
- Constraints: No number can repeat in any row, column, or 3×3 sub-grid

Steps:

1. **Initial Constraints:** For every cell, values that are already present in the same row, column, or sub-grid are removed from the domain.
2. **Domain Reduction:** If a cell has only one remaining value, that value is assigned to the cell.

3. **Propagation:** The effect of assigning a value propagates to other cells in the same row, column, or sub-grid, further reducing their possible domains.
4. **Consistency Checking:** The algorithm continues until all cells are filled (solution) or no valid values remain (inconsistency).

Representing knowledge using rules

1. What are Rules in AI?

Rules are a formal way of representing knowledge. A rule consists of a **condition** (also known as the "if" part or premise) and a **consequence** (also known as the "then" part or conclusion). In general, rules take the following form:

IF <condition> THEN <action/consequence>

These rules are also referred to as production rules, and they can be used in systems like expert systems or rule-based reasoning engines to apply logical steps to infer new knowledge or make decisions.

2. Syntax of a Rule

- **Antecedent (Premise):** The condition or set of conditions that must be satisfied for the rule to be applied. This is the "IF" part.
- **Consequent (Conclusion):** The action or outcome that occurs when the conditions are met. This is the "THEN" part.

For example:

IF the sky is cloudy AND it is humid THEN it will rain.

Here:

- **Antecedent:** "The sky is cloudy AND it is humid"
- **Consequent:** "It will rain"

3. Types of Knowledge Represented Using Rules

- **Declarative Knowledge:** This represents facts or information about the world.
 - Example

IF an object is metal THEN it conducts electricity.

- **Procedural Knowledge:** This represents how to perform tasks or actions.

Example

IF you want to drive a car THEN you must start the engine.

4. Chaining: Forward and Backward

Rules are often used in reasoning systems through a process called **chaining**. There are two main approaches:

- **Forward Chaining:** Starts with available data and applies rules to infer new information until the goal is reached. It is data-driven reasoning.
 - Example

Rule 1: IF it is raining THEN the ground will be wet.

Fact: It is raining.

Conclusion: The ground will be wet (inferred).

- **Backward Chaining:** Starts with a goal and works backward to see if the known facts support it. It is goal-driven reasoning.

Example

Goal: Prove that the ground is wet.

Rule 1: IF it is raining THEN the ground will be wet.

Check Fact: Is it raining?

5. Representing Knowledge in Expert Systems

Expert systems use a large collection of rules to represent knowledge in a particular domain (e.g., medical diagnosis, financial advice). Each rule encodes expert-level knowledge that can be used to make decisions based on input data.

Example: In a medical diagnosis system, a rule might look like:

IF the patient has a fever AND the patient has a cough THEN the patient might have the flu.

Here, the system checks the patient's symptoms (fever and cough) and uses the rule to infer that the patient might have the flu.

6. Conflict Resolution

In a system with many rules, conflicts can arise when multiple rules are applicable. Conflict resolution strategies are used to decide which rule to apply.

Common strategies include:

- **Specificity:** Apply the more specific rule first.
- **Recency:** Use the most recent data or fact.
- **Priority:** Some rules are given higher priority than others.

7. Example of a Rule-Based System:

Let's consider a **traffic light controller**:

1. **Rule 1:** IF the light is green THEN allow cars to pass.
2. **Rule 2:** IF the light is red THEN stop the cars.
3. **Rule 3:** IF the light is yellow THEN cars should prepare to stop.

Here, the knowledge of how a traffic light system works is encoded using simple production rules. The controller can manage the traffic flow by applying the appropriate rule based on the color of the light.

8. Example of a Diagnostic Rule:

A **car diagnostic system** might include rules like:

IF the car does not start AND the battery is dead THEN replace the battery.

In this case:

- **Antecedent:** "The car does not start AND the battery is dead"
- **Consequent:** "Replace the battery"

Such a system can help a mechanic diagnose problems by using predefined rules based on observed symptoms.

9. Limitations of Rule-Based Systems

While rule-based systems are useful for many AI applications, they do have limitations:

- **Scalability:** As the number of rules grows, managing and maintaining the system becomes difficult.
- **Complexity:** Some real-world problems are too complex to represent with simple if-then rules.
- **Ambiguity and Uncertainty:** Handling uncertainty or ambiguous situations is challenging in pure rule-based systems, which often assume binary logic.

10. Combining Rules with Other Representations

Rule-based systems are often combined with other knowledge representation techniques, such as:

- **Frames:** To represent hierarchical knowledge or structured information.
- **Probabilistic Reasoning:** To handle uncertainty (e.g., Bayesian Networks).
- **Fuzzy Logic:** To manage imprecise or vague rules.

Rule-Based Deduction Systems: Overview

A **rule-based system** is composed of three main components:

1. **Knowledge Base (KB):** A collection of facts and rules.
2. **Inference Engine:** The mechanism that applies logical rules to the facts in the KB to infer new facts or solve problems.
3. **Working Memory:** Where the inferred facts or results are stored temporarily during problem-solving.

1. Knowledge Representation in Rule-Based Systems

- **Facts:** The statements or assertions about the world.
 - Example: Sunny(Today) (It is sunny today).
- **Rules:** Logical implications that define how facts relate to each other, often in the form of **if-then** statements.
 - Example: If Sunny(Today), then GoToPark(Today).
 - This means: *If it is sunny today, then I will go to the park.*

2. Inference in Rule-Based Systems

There are two primary inference methods:

1. **Forward Chaining (Data-Driven Inference):**
 - Starts with known facts and applies rules to infer new facts until a goal is reached.
 - Example:
 - Facts: Sunny(Today), Warm(Today)
 - Rule: If Sunny(Today) and Warm(Today), then GoToPark(Today)
 - Inference: Since both facts are true, we deduce that GoToPark(Today) is true.
2. **Backward Chaining (Goal-Driven Inference):**
 - Starts with a goal (query) and works backward to determine if facts support that goal.
 - Example:
 - Query: GoToPark(Today)?
 - Rule: If Sunny(Today) and Warm(Today), then GoToPark(Today)
 - System checks if Sunny(Today) and Warm(Today) are true. If so, the goal is achieved.

3. Example of a Rule-Based System

Let's create a simple expert system for determining what action to take based on weather conditions:

- **Fact 1:** Sunny(Today)

- **Fact 2:** Warm(Today)
- **Fact 3:** NoClouds(Today)

Rules:

- If Sunny(Today) and Warm(Today), then GoToPark(Today)
- If NoClouds(Today), then GoForPicnic(Today)

Forward Chaining Example:

- Starting with facts Sunny(Today) and Warm(Today), apply the first rule and infer GoToPark(Today).
- Then, using the fact NoClouds(Today), infer GoForPicnic(Today).

Backward Chaining Example:

- Suppose the system is queried with GoForPicnic(Today)?
- The system will look for a rule that concludes GoForPicnic(Today), which is If NoClouds(Today), then GoForPicnic(Today).
- It then checks if NoClouds(Today) is true in the fact base, and if so, returns "Yes."

4. Conflict Resolution in Rule-Based Systems

When multiple rules can be applied at the same time, the system uses **conflict resolution strategies** to decide which rule to apply:

- **Specificity:** The most specific rule is applied first.
- **Recency:** The most recently added facts are used to trigger rules.
- **Priority:** Rules can be assigned priorities, and the system applies the highest-priority rule first.

5. Use Cases of Rule-Based Systems

- **Expert Systems in Medicine:** Rule-based systems diagnose diseases based on symptoms using a database of medical rules.
 - Example Rule: If Fever and Cough, then Flu.
- **Legal Reasoning Systems:** Lawyers use rule-based systems to interpret and apply laws to cases.
 - Example Rule: If Age < 18, then Minor.
- **Game AI:** A rule-based system determines the next move in a game based on the current game state.
 - Example Rule: If OpponentMoves(Aggressive), then CounterWith(Defense).

6. Strengths and Limitations

- **Strengths:**

- Easy to understand and implement.
- Clear reasoning process that can be explained to users.
- Suitable for well-defined domains where rules are explicit and finite.
- **Limitations:**
 - **Scalability:** As the number of rules increases, so does the complexity.
 - **Maintenance:** Updating and maintaining a large rule set can be difficult.
 - **Inflexibility:** Rules must be explicitly programmed, making it difficult to handle ambiguous or unexpected scenarios.

1. Introduction to Reasoning Under Uncertainty

In many real-world situations, systems do not have access to complete information or may face ambiguous or noisy data. Unlike classical logic, which deals with certainty, reasoning under uncertainty allows AI systems to operate in complex environments where information is incomplete, ambiguous, or contradictory.

AI systems must weigh evidence, make educated guesses, and provide conclusions even when certainty is not guaranteed. Probabilistic models and logic-based methods are often used to handle these situations.

2. Common Approaches to Reasoning Under Uncertainty

A. Probabilistic Reasoning

One of the most popular ways to deal with uncertainty is through probabilities. Probabilistic reasoning involves quantifying uncertainty in terms of probability values and updating beliefs based on evidence.

- **Bayesian Networks (Belief Networks):**
 - These networks represent a set of variables and their conditional dependencies using a directed acyclic graph.
 - Probabilistic inferences can be made by updating beliefs as new evidence becomes available using Bayes' Theorem.

Example:

- Consider a medical diagnosis system that has to determine whether a patient has a particular disease based on symptoms. If the patient reports having a headache and fever, the system can calculate the probability of having a specific illness (like the flu) based on the likelihood of those symptoms being associated with the disease.

- **Bayes' Theorem:**

$$P(H|E) = \frac{P(E|H)P(H)}{P(E)}$$

Where:

- $P(H|E)$ is the posterior probability (the probability of hypothesis H being true given the evidence E),
 - $P(E|H)$ is the likelihood (probability of evidence given the hypothesis),
 - $P(H)$ is the prior probability (initial belief in the hypothesis),
 - $P(E)$ is the probability of the evidence.
- **Example:**
 - Imagine a weather forecasting system that estimates the likelihood of rain. The system uses historical weather data (prior probability) and current weather conditions (evidence) to compute the updated probability of rain tomorrow.

B. Fuzzy Logic

Fuzzy logic is an extension of classical logic designed to handle the concept of partial truth. Instead of assigning a strict true/false value, fuzzy logic assigns a degree of truth (between 0 and 1).

- **Fuzzy Sets:**
 - In fuzzy logic, elements can belong to a set to varying degrees. For example, a room might be "warm" to varying degrees rather than strictly "warm" or "not warm."

Example:

- A temperature control system can adjust heating based on fuzzy rules like "If the room is somewhat cold, increase the heater slightly." The system uses fuzzy terms such as "somewhat cold" to describe intermediate states between "cold" and "warm."

C. Rule-Based Reasoning with Certainty Factors

In some expert systems, certainty factors are used to quantify how certain the system is about a given rule or piece of information. Certainty factors are numerical values between -1 and 1, where -1 represents total disbelief, 0 represents no knowledge, and 1 represents complete certainty.

- **Example:**

- In an expert system for medical diagnosis, a rule might state: "If the patient has a sore throat, then there is a 0.8 certainty that the patient has a cold." The system combines various pieces of evidence (with associated certainty factors) to arrive at a final diagnosis.

D. Dempster-Shafer Theory

This theory is a generalization of Bayesian probability, which allows for reasoning with belief and doubt. It assigns belief masses to different propositions, which indicate how much belief is assigned to them based on the evidence. Dempster-Shafer theory allows for the representation of uncertainty by leaving some probability unassigned to any specific event.

- **Example:**

- In a fault detection system, sensors provide incomplete data about potential machine failures. The system uses Dempster-Shafer theory to combine these uncertain and incomplete sources of evidence to assess the likelihood of various fault types.

E. Markov Decision Processes (MDPs)

MDPs are used for decision-making in situations where outcomes are partly random and partly under the control of a decision maker. They are a mathematical framework for modeling decision-making where the outcomes are uncertain.

- **Components of MDP:**

1. **States:** Different possible situations (e.g., a robot's position).
2. **Actions:** Choices available to the agent (e.g., move left, right).
3. **Transition Probabilities:** Probability of moving from one state to another due to an action.
4. **Rewards:** Numerical values representing the desirability of certain states.

Example:

- A self-driving car must decide at each intersection whether to turn left, right, or go straight. The decision is made under uncertainty (e.g., unpredictable traffic), with the goal of reaching the destination while minimizing travel time and avoiding accidents.

3. Applications of Reasoning Under Uncertainty

- **Medical Diagnosis:**

- Systems like **MYCIN** use probabilistic reasoning to diagnose diseases based on symptoms and patient history. The system provides diagnoses and treatment suggestions, taking uncertainty in symptoms and test results into account.

- **Speech Recognition:**

- Systems that recognize human speech must deal with uncertainty due to varying accents, background noise, and incomplete speech data. Probabilistic models (e.g.,

Hidden Markov Models) are used to infer the most likely words or sentences given the input sounds.

- **Autonomous Vehicles:**
 - Self-driving cars must make decisions in uncertain environments, such as dealing with unpredictable behavior from other drivers or poor weather conditions. Markov decision processes or Bayesian inference can be used to model these uncertainties and make safe driving decisions.

4. Challenges in Reasoning Under Uncertainty

- **Computational Complexity:**
 - Probabilistic reasoning often requires calculating probabilities over large spaces of possibilities, which can be computationally expensive.
- **Ambiguity in Data:**
 - In some cases, data may be ambiguous or conflicting, requiring systems to make decisions with incomplete or noisy information.
- **Balancing Exploration and Exploitation:**
 - Decision-making under uncertainty involves balancing between exploring new actions (to gather more information) and exploiting known actions (to maximize reward or minimize loss).



Dempster-Shafer Theory

Dempster-Shafer Overview: The Dempster-Shafer theory (DST) is a mathematical theory of evidence that allows for reasoning with uncertainty and combining evidence from different sources. It provides a framework for representing and combining beliefs without requiring precise probabilities.

Key Concepts:

- **Basic Probability Assignment (BPA):** Assigns a probability to each subset of the possible outcomes, representing the degree of belief in that subset.
- **Belief Function (Bel):** Represents the total belief supported by the evidence.
- **Plausibility Function (Pl):** Measures the extent to which evidence does not contradict the hypothesis.

Combination Rule: Dempster's rule of combination is used to combine evidence from different sources:

$$m(A) = \sum_{B \cap C = A} m_1(B) \cdot m_2(C) \cdot (1 - K)$$

Where:

- m_1 and m_2 are the basic probability assignments from two sources.
- K is a conflict term, representing the degree of conflict between the sources.

Example: Weather Prediction Suppose two weather stations provide predictions about whether it will rain (R) or not ($\neg R$). The basic probability assignments might be:

- Station 1: $m_1(R) = 0.6, m_1(\neg R) = 0.4$
- Station 2: $m_2(R) = 0.7, m_2(\neg R) = 0.3$

To combine their predictions:

1. Apply Dempster's rule of combination:

- Calculate conflict K (if any). Here, there is no direct conflict since the events are mutually exclusive.

2. Combine the beliefs:

$$m(R) = m_1(R) \cdot m_2(R) = 0.6 \cdot 0.7 = 0.42$$

$$m(\neg R) = m_1(\neg R) \cdot m_2(\neg R) = 0.4 \cdot 0.3 = 0.12$$

3. Normalize:

$$\text{Total} = m(R) + m(\neg R) = 0.42 + 0.12 = 0.54$$

$$m(R)_{\text{normalized}} = \frac{0.42}{0.54} \approx 0.778$$

$$m(\neg R)_{\text{normalized}} = \frac{0.12}{0.54} \approx 0.222$$

Interpretation: The combined belief in rain is approximately 77.8%, while the belief in no rain is 22.2%.



UNIT-IV

Logic Concepts

Logic concepts: First order logic. Inference in first order logic, propositional vs. first order inference, unification & lifts forward chaining, Backward chaining, Resolution, Learning from observation Inductive learning, Decision trees, Explanation based learning, Statistical Learning methods, Reinforcement Learning.

First-Order Logic (FOL)

- **Definition:** First-order logic (also called predicate logic) is an extension of propositional logic that allows for more expressive representations using objects, relations, and quantifiers. Unlike propositional logic, which only deals with true or false propositions, FOL enables reasoning about individual elements within a domain.
- **Components:**
 - **Objects:** Represent entities in the domain (e.g., *Alice*, *a cat*, *2*).
 - **Predicates:** Describe properties of objects or relationships between them (e.g., *Loves(Alice, Bob)*, *IsCat(Tom)*).
 - **Quantifiers:**
 - **Universal Quantifier ($\forall$):** Specifies that a statement applies to all elements (e.g., $\forall x \text{ Loves}(x, \text{IceCream})$ means everyone loves ice cream).
 - **Existential Quantifier ($\exists$):** Specifies that a statement applies to at least one element (e.g., $\exists x \text{ Loves}(x, \text{IceCream})$ means there exists someone who loves ice cream).
 - **Functions:** Map objects to other objects (e.g., *Father(John)* may return *Bob* if Bob is John's father).
- **Syntax and Semantics:** FOL expressions have a structured syntax:
 - **Atomic Sentences:** Simple propositions, like *Loves(Alice, Bob)*.
 - **Complex Sentences:** Formed using logical connectives like $\wedge$ (and), $\vee$ (or), $\rightarrow$ (implies), and $\neg$ (not).
 - **Interpretation:** Assigns meanings to symbols, giving truth values to statements.
- **Example in AI:**
 - Statement: "All humans are mortal, and Socrates is a human."
 - Representation:
 - $\forall x (\text{Human}(x) \rightarrow \text{Mortal}(x))$
 - $\text{Human}(\text{Socrates})$

2. Inference in First-Order Logic

Inference in FOL involves deriving new statements from known information using inference rules. Common techniques include:

- **Unification:** A process that finds substitutions to make two predicates identical, allowing FOL to apply rules by matching arguments.
- **Modus Ponens:** If $P \rightarrow Q$ and P are both true, then Q is inferred. This rule is often used in chain reasoning with quantified statements.
- **Generalized Modus Ponens:** Extends modus ponens for first-order logic, where variables are unified and instantiated when applying the rule.
- **Resolution:** A fundamental inference rule for FOL, used in proofs by refutation. Resolution

involves:



1. Converting statements to **Conjunctive Normal Form (CNF)**.
 2. Applying unification to clauses and combining them until a contradiction or solution is found.
- **Forward and Backward Chaining:**
 - **Forward Chaining:** Begins with known facts and applies inference rules to generate new facts until the goal is reached.
 - **Backward Chaining:** Starts with the goal and works backward to find supporting facts, useful in goal-directed reasoning (e.g., rule-based systems).
 - **Examples:**
 - **Example 1:** Proving “Socrates is mortal”:
 - Given knowledge:
 1. $\forall x (\text{Human}(x) \rightarrow \text{Mortal}(x))$ (All humans are mortal)
 2. $\text{Human}(\text{Socrates})$ (Socrates is a human)
 - Applying modus ponens:
 - Since $\text{Human}(\text{Socrates}) \rightarrow \text{Mortal}(\text{Socrates})$ and $\text{Human}(\text{Socrates})$ are both true, we conclude $\text{Mortal}(\text{Socrates})$.
 - **Example 2:** Finding someone who likes both pizza and ice cream:
 - Given knowledge:
 0. $\forall x (\text{Likes}(x, \text{Pizza}) \wedge \text{Likes}(x, \text{IceCream}) \rightarrow \text{Happy}(x))$ (Everyone who likes both pizza and ice cream is happy)
 1. $\text{Likes}(\text{John}, \text{Pizza})$
 2. $\text{Likes}(\text{John}, \text{IceCream})$
 - Inference:
 - From statements 2 and 3, we deduce $\text{Likes}(\text{John}, \text{Pizza}) \wedge \text{Likes}(\text{John}, \text{IceCream})$.
 - Applying generalized modus ponens, we infer $\text{Happy}(\text{John})$.

Propositional Inference

Propositional inference operates on **propositional logic**, where statements (or propositions) are represented as simple, indivisible entities that are either true or false.

Key Characteristics:

- **Atomic Sentences:** Propositional logic uses atomic sentences (propositions) like P, Q, or R with no internal structure, representing fixed statements.
- **Simple Structure:** Propositional logic lacks variables, quantifiers, or functions; each proposition is an independent fact.
- **Inference Rules:** Common inference rules include **Modus Ponens**, **Modus Tollens**, and **Resolution**. Inference relies solely on logical connectives like AND, OR, and NOT.

Example of Propositional Inference:

Consider the following knowledge base (KB):

1. If it rains, the grass will be wet: $\text{Rain} \rightarrow \text{WetGrass}$

2. It is raining: Rain

Using **Modus Ponens**:

1. Since Rain is true and $\text{Rain} \rightarrow \text{WetGrass}$, we can infer that WetGrass is also true.

Pros and Cons:

- **Pros:** Propositional inference is computationally simpler and works well for problems with straightforward, unstructured facts.
- **Cons:** It cannot handle more complex relationships or statements involving objects, variables, or quantifiers, making it limited in expressiveness.

First-Order Inference

First-order inference operates on **first-order logic (FOL)**, which allows reasoning about relationships between objects and includes variables and quantifiers.

Key Characteristics:

- **Structured Sentences:** FOL includes predicates, functions, and constants, allowing statements with internal structures, such as $\text{Loves}(\text{John}, \text{Mary})$.
- **Quantifiers:** First-order logic supports universal ($\forall$) and existential ($\exists$) quantifiers, allowing statements like “All humans are mortal.”
- **Rules of Inference:** First-order inference applies rules like **Universal Elimination**, **Existential Instantiation**, and **Resolution** to handle variable substitution and relationships.

Example of First-Order Inference:

Consider the following knowledge base:

1. All humans are mortal: $\forall x (\text{Human}(x) \rightarrow \text{Mortal}(x))$
2. Socrates is a human: $\text{Human}(\text{Socrates})$

Using **Universal Instantiation** and **Modus Ponens**:

1. Substitute Socrates for x in $\text{Human}(x) \rightarrow \text{Mortal}(x)$, yielding $\text{Human}(\text{Socrates}) \rightarrow \text{Mortal}(\text{Socrates})$.
2. Since $\text{Human}(\text{Socrates})$ is true, we infer $\text{Mortal}(\text{Socrates})$ is also true.

Pros and Cons:

- **Pros:** FOL is more expressive, able to represent complex relationships and universally quantified statements.
- **Cons:** First-order inference is computationally more demanding than propositional inference and can be slower for large KBs.

Comparison: Propositional vs. First-Order Inference

Aspect	Propositional Inference	First-Order Inference
Structure	Operates on atomic, indivisible propositions	Works with structured sentences (predicates)
Expressiveness	Limited, only allows true/false propositions	High, allows variables, functions, and objects
Quantifiers	Not supported	Supports universal and existential quantifiers
Efficiency	Computationally efficient for simple problems	More computationally intensive for complex KBs
Application Suitability	Works well for simpler, less structured data	Better suited for complex, real-world scenarios

Unification in Artificial Intelligence

Unification is a process in logic programming and artificial intelligence that involves making two logical expressions identical by finding a suitable substitution for their variables. It plays a crucial role in automated reasoning, natural language processing, and knowledge representation.

Key Concepts of Unification

- Terms:** A term can be a constant, variable, or a complex expression made up of function symbols applied to other terms. For example:
 - Constants: a, b
 - Variables: X, Y
 - Functions: $f(X, b), g(a)$
- Substitutions:** A substitution is a mapping of variables to terms. For instance, if we have the substitution $\{X/a, Y/b\}$, it means that the variable X is replaced with a and Y is replaced with b .
- Unification Process:** To unify two terms, the unification algorithm attempts to find a substitution that makes them identical. If a unifying substitution exists, the two terms can be considered equivalent.

Example of Unification

Consider the two terms:

- Term 1: $f(X, g(Y))$
- Term 2: $f(a, g(b))$

To unify these terms, we look for substitutions:

1. The outer function f is identical in both terms, so we proceed to unify the inner terms:
 - X must be unified with a . Thus, we have the substitution $\{X/a\}$.
 - $g(Y)$ must be unified with $g(b)$. This requires Y to be unified with b . Thus, we add $\{Y/b\}$.

Combining these, we get the final unifying substitution:

- $\{X/a, Y/b\}$

Using this substitution, both terms become identical:

- $f(a, g(b))$

Lifts in Artificial Intelligence

Lifts are a concept often used in the context of representing relationships between objects in structured representations such as logic and knowledge bases. The term "lift" can also refer to how certain operations or transformations can be applied to elements in a more abstract way.

Key Concepts of Lifts

1. **Functional Lifting:** This refers to the process of applying a function to a collection of values or data structures rather than individual elements. It often involves higher-order functions or operations that can take a function as an argument and return a new function that operates on a different type of data structure.
2. **Contextual Lifting:** In knowledge representation, lifts can help represent relationships in a more abstract way, allowing for reasoning about concepts without needing to specify every detail.

Example of Lifting

Suppose we have a function that takes a number and doubles it:

- $\text{double}(x) = 2 * x$

Using functional lifting, we can create a new function that applies `double` to every element in a list:

- Original list: [1, 2, 3]
- Lifted function: map(double, [1, 2, 3])
- Result: [2, 4, 6]

Summary

In summary, unification is a powerful mechanism for making logical expressions identical by finding suitable substitutions, which is fundamental in logic programming and AI. Lifts, on the other hand, enhance the expressiveness of functions by applying them to collections or more abstract representations, facilitating operations over data structures and relationships in knowledge representation. Both concepts are integral to building intelligent systems capable of reasoning and processing complex information.

Forward Chaining and Backward Chaining in Artificial Intelligence

Forward Chaining and **Backward Chaining** are two fundamental inference techniques used in rule-based systems and expert systems for deriving conclusions from a set of premises. They are often used in the context of production systems or logic programming.

Forward Chaining

Definition: Forward chaining is a data-driven approach where inference starts from the known facts and applies rules to derive new facts until a goal is reached.

Process:

1. **Initialization:** Begin with a set of known facts (the initial knowledge base).
2. **Rule Application:** Examine the rules in the knowledge base. If the conditions (antecedents) of a rule are satisfied by the current facts, the rule is fired, and the conclusion (consequent) is added to the knowledge base as a new fact.
3. **Repeat:** Continue this process until no more rules can be applied or a specific goal is reached.

Example: Consider the following rules in a simple expert system for diagnosing illnesses:

1. **Rule 1:** If a patient has a fever, then they might have the flu.
2. **Rule 2:** If a patient has a sore throat, then they might have strep throat.
3. **Rule 3:** If a patient has the flu, they should rest.
4. **Rule 4:** If a patient has strep throat, they should see a doctor.

Known Facts:

- Patient A has a fever.
- Patient A has a sore throat.

Forward Chaining Process:

1. Start with known facts: Patient A has a fever, Patient A has a sore throat.
2. Apply **Rule 1**: Patient A might have the flu (new fact added).
3. Apply **Rule 2**: Patient A might have strep throat (new fact added).
4. Now you have:
 - Patient A might have the flu.
 - Patient A might have strep throat.
5. No further rules can be applied unless there are additional facts or goals.

Use Cases:

- Used in production systems, expert systems, and scenarios where all facts are known, and the goal is to derive conclusions based on those facts.
-

Backward Chaining

Definition: Backward chaining is a goal-driven approach that starts with a specific goal and works backward to determine if the facts support that goal.

Process:

1. **Initialization:** Begin with a specific goal (the conclusion you want to prove).
2. **Rule Examination:** Check the rules in the knowledge base to see if any rules can lead to the goal.
3. **Sub-goal Creation:** If a rule's conclusion matches the goal, the antecedents become sub-goals that must be satisfied.
4. **Recursion:** Repeat the process for the new sub-goals until you either find sufficient facts to support the goal or determine that the goal cannot be satisfied.

Example: Using the same rules as before, let's consider the goal of determining whether Patient A should see a doctor.

Goal: Patient A should see a doctor.

Backward Chaining Process:

1. Start with the goal: Patient A should see a doctor.
2. Check rules:
 - **Rule 4** states that if Patient A has strep throat, they should see a doctor.
3. Set a sub-goal: Does Patient A have strep throat?
4. Check rules:
 - **Rule 2** states that if a patient has a sore throat, they might have strep throat.
5. Set another sub-goal: Does Patient A have a sore throat?
6. Known fact: Patient A has a sore throat (goal satisfied).
7. Sub-goal satisfied: Patient A might have strep throat (based on **Rule 2**).

8. Thus, the original goal is satisfied: Patient A should see a doctor.

Use Cases:

- Used in expert systems, theorem proving, and scenarios where the goal is known, and the focus is to determine the necessary conditions or facts to achieve that goal.

Summary of Differences

Feature	Forward Chaining	Backward Chaining
Direction	Data-driven (facts to conclusions)	Goal-driven (conclusions to facts)
Start Point	Begins with known facts	Begins with the goal to be proven
Process Type	Generates all possible conclusions	Generates specific conclusions based on goals
Efficiency	Can be inefficient with irrelevant conclusions	More efficient for specific goals

Resolution is a fundamental concept in artificial intelligence and automated reasoning, particularly within the realm of propositional and predicate logic. It is a rule of inference used to derive conclusions from premises, playing a critical role in logic programming, theorem proving, and knowledge representation.

Overview of Resolution

1. **Definition:** Resolution is a proof technique that operates on clauses (disjunctions of literals). It allows us to infer new clauses from existing ones by resolving two clauses that contain complementary literals.
2. **Complementary Literals:** Two literals are complementary if one is the negation of the other. For example, if we have a clause $A \vee B \vee C$ and another clause $\neg A \vee D \vee E$, these are complementary literals.
3. **Process:**
 - Given two clauses C_1 and C_2 , if C_1 contains a literal L and C_2 contains $\neg L$, we can resolve C_1 and C_2 to create a new clause that contains all the literals of C_1 and C_2 except for L and $\neg L$.
 - The resulting clause is known as the **resolvent**.

Steps in Resolution

1. **Convert to Clausal Form:** Convert sentences into conjunctive normal form (CNF) if they are not already. In CNF, a formula is a conjunction of clauses, where each clause is a disjunction of literals.

2. **Apply Resolution Rule:** Identify pairs of clauses that contain complementary literals and apply the resolution rule to derive new clauses.
3. **Iterate:** Continue applying the resolution rule on the new clauses until you derive a contradiction (empty clause) or reach a desired conclusion.

Example of Resolution

Problem Statement

Let's demonstrate resolution with a simple example involving propositional logic.

1. **Given Clauses:**
 - Clause 1: $P \vee Q$ (If either P or Q is true)
 - Clause 2: $\neg P \vee R$ (If P is false, then R is true)
 - Clause 3: $\neg Q$ (If Q is false)
2. **Resolution Steps:**
 - **Step 1:** Resolve Clause 1 and Clause 3:
 - From $P \vee Q$ and $\neg Q$:
 - Resolvent: P
 - **Step 2:** Resolve the new clause P with Clause 2:
 - From P and $\neg P \vee R$:
 - Resolvent: R
3. **Conclusion:**
 - The derived clauses are P and R. If Q is false, R must be true.

Unifying Resolution in Predicate Logic

Resolution can also be applied in predicate logic, where it involves **unification**:

1. **Unification:** A process of making two predicates identical by substituting variables with terms. For example, unifying $P(x)$ with $P(a)$ results in $x=a$.
2. **Example:**
 - **Given Clauses:**
 - Clause 1: $\forall x(P(x) \rightarrow Q(x))$
 - Clause 2: $P(a)$
 - **Convert to CNF:**
 - Clause 1 becomes $\neg P(x) \vee Q(x)$
 - Clause 2 remains $P(a)$
 - **Resolution Steps:**
 - Resolve $\neg P(x) \vee Q(x)$ and $P(a)$:
 - Result: $Q(a)$ (since we substitute x with a)

Applications of Resolution

1. **Automated Theorem Proving:** Resolution is used in systems that automate the proving of mathematical theorems and logical assertions.

2. **Logic Programming:** Languages like Prolog use resolution as a fundamental inference mechanism to derive conclusions based on rules and facts.
3. **Knowledge Representation:** Resolution aids in reasoning about knowledge represented in logical form, enabling intelligent agents to draw conclusions from known facts.

Learning from Observation in Artificial Intelligence

Learning from observation is a fundamental concept in artificial intelligence (AI) that involves acquiring knowledge and improving performance through experience. This approach allows agents to learn patterns and make decisions based on the data they collect from their environment. Russell and Norvig discuss this concept extensively in their book "Artificial Intelligence: A Modern Approach."

Key Concepts

1. **Types of Learning:**
 - **Supervised Learning:** Involves learning a function that maps inputs to desired outputs based on a labeled dataset. The model learns from the examples provided during training.
 - **Unsupervised Learning:** Involves learning patterns or structures from unlabeled data without explicit instructions on what to predict.
 - **Reinforcement Learning:** Involves learning how to take actions in an environment to maximize some notion of cumulative reward through trial and error.
2. **Data Representation:**
 - Data can be represented in various forms, such as vectors, matrices, or more complex structures like graphs. Effective representation is crucial for the learning algorithm to extract meaningful features from the data.
3. **Generalization:**
 - The goal of learning from observation is not just to memorize the training data but to generalize from it so that the model can make accurate predictions on unseen data.
4. **Inductive Bias:**
 - Every learning algorithm incorporates some assumptions about the underlying data distribution, known as inductive bias. These biases influence the generalization capabilities of the model.
5. **Feature Selection:**
 - Selecting the right features (attributes) for learning is crucial. Irrelevant or redundant features can degrade the performance of learning algorithms.

Examples

1. **Supervised Learning Example: Image Classification**
 - **Problem:** Classifying images of animals (e.g., cats and dogs).
 - **Data:** A labeled dataset containing images of cats and dogs, each with corresponding labels (cat or dog).
 - **Learning Process:** A supervised learning algorithm, like a Convolutional Neural Network (CNN), learns to recognize features (e.g., shape, color, texture) associated with each class by adjusting its parameters based on the labeled examples.

- **Outcome:** After training, the model can classify new images of animals into the categories of cats or dogs with a certain accuracy.
- 2. **Unsupervised Learning Example: Clustering**
 - **Problem:** Segmenting customers based on purchasing behavior.
 - **Data:** A dataset containing customer transactions without any labels.
 - **Learning Process:** An unsupervised learning algorithm, like K-means clustering, groups customers into clusters based on similarities in their purchasing patterns.
 - **Outcome:** The algorithm identifies distinct customer segments, allowing the business to tailor marketing strategies for each group.
- 3. **Reinforcement Learning Example: Game Playing**
 - **Problem:** Training an AI agent to play chess.
 - **Data:** The agent learns by playing games against itself or human opponents.
 - **Learning Process:** Through reinforcement learning, the agent receives rewards for making good moves and penalties for bad moves. It uses this feedback to refine its strategy over many games.
 - **Outcome:** The agent improves its performance over time, eventually becoming proficient at playing chess, potentially even surpassing human experts.

Advantages of Learning from Observation

- **Adaptability:** Agents can adapt to changing environments by learning from new observations, improving their decision-making capabilities.
- **Scalability:** Learning from observation can handle large amounts of data, enabling models to learn complex patterns and make informed predictions.
- **Automation:** It reduces the need for manual programming of rules, allowing systems to learn autonomously and improve continuously.

Challenges

- **Data Quality:** Poor quality or biased data can lead to incorrect learning outcomes.
- **Overfitting:** A model may learn noise in the training data rather than the underlying pattern, resulting in poor generalization to new data.
- **Computational Resources:** Learning from large datasets can require significant computational power and time.

Inductive Learning in Artificial Intelligence

Inductive learning is a fundamental approach in artificial intelligence that involves inferring general rules or patterns from specific examples or observations. This method contrasts with deductive reasoning, which starts with general rules and applies them to specific cases. Inductive learning is especially relevant in machine learning, where models are trained on data to make predictions or classifications.

Key Concepts of Inductive Learning

1. Generalization:

- The primary goal of inductive learning is to generalize from specific instances to broader concepts. For example, if a learning algorithm is presented with several instances of cats (e.g., images of different cats), it should be able to learn the general characteristics of "cats" that can then be applied to recognize new, unseen cat images.

2. Hypothesis Formation:

- Inductive learning involves the formulation of hypotheses based on the input data. A hypothesis is a proposed explanation or general rule that the learning algorithm assumes to be true. For instance, after seeing multiple examples of birds, a hypothesis might be that all birds can fly (though this is not universally true).

3. Inductive Bias:

- Inductive bias refers to the set of assumptions that the learning algorithm makes to generalize from the training data. Different algorithms may have different biases, influencing their ability to learn from the same data. For example, decision trees might assume that the data can be segmented into distinct classes based on certain features, while neural networks might assume a more complex, nonlinear relationship.

4. Training and Testing:

- Inductive learning typically involves a training phase, where the model learns from a labeled dataset (training data), and a testing phase, where the model is evaluated on a separate dataset (test data) to assess its generalization performance.

Examples of Inductive Learning

1. Decision Trees:

- **Example:** Suppose we want to classify whether a fruit is an apple or an orange based on features like color and size.
 - **Training Data:**
 - (Red, Small) → Apple
 - (Orange, Medium) → Orange
 - (Red, Large) → Apple
 - (Orange, Small) → Orange
 - **Inductive Learning Process:** The decision tree algorithm analyzes the training data and forms rules based on the attributes (e.g., if the fruit is red and small, then it is an apple).
 - **Resulting Hypothesis:** If a new fruit is (Red, Small), the model predicts it is an **Apple**.

2. Neural Networks:

- **Example:** A neural network might be used to classify images of handwritten digits (0-9).
 - **Training Data:** The model is trained on thousands of labeled images of handwritten digits.
 - **Inductive Learning Process:** The network adjusts its weights based on the input images and the corresponding labels during training.

- **Resulting Hypothesis:** After training, if the model is presented with a new image of the digit "5," it can classify it correctly based on the patterns learned during training.

3. Rule-Based Learning:

- **Example:** In a medical diagnosis system, the algorithm learns rules based on symptoms and diagnoses.
 - **Training Data:**
 - (Fever, Cough) → Flu
 - (Fever, Rash) → Measles
 - **Inductive Learning Process:** The system formulates rules from the examples provided.
 - **Resulting Hypothesis:** The system might conclude that if a patient has a fever and a cough, they likely have the **Flu**.

Advantages and Disadvantages

- **Advantages:**
 - Can handle large amounts of data effectively.
 - Capable of discovering patterns and relationships in data.
 - Useful in situations where it is difficult to formulate explicit rules.
- **Disadvantages:**
 - May lead to overfitting, where the model learns noise instead of the underlying pattern.
 - Requires a sufficiently large and representative dataset to generalize well.
 - Can be sensitive to the inductive bias and assumptions made by the learning algorithm.

Decision Trees

Definition: Decision Trees are a type of supervised learning model used for classification and regression tasks. They represent decisions and their possible consequences as a tree-like structure, where each internal node represents a feature (attribute), each branch represents a decision rule, and each leaf node represents an outcome.

Characteristics:

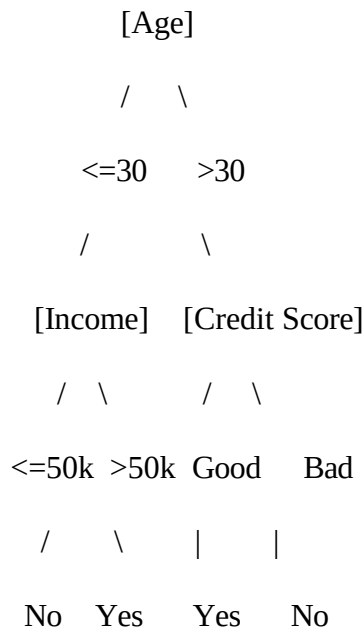
- **Hierarchical Structure:** Organized in a tree format with nodes and branches.
- **Interpretability:** Easy to understand and interpret, making them useful for decision-making processes.
- **Non-Parametric:** No assumptions about the underlying data distribution.

Construction:

- **Selecting Attributes:** Use algorithms like ID3, C4.5, or CART to select the best attribute for splitting the data based on metrics like Gini impurity or entropy.

- **Stopping Criteria:** The process stops when all samples belong to the same class, or further splitting does not improve the model significantly.

Example: Suppose we want to classify whether a person will buy a car based on attributes such as age, income, and credit score.



Explanation-Based Learning (EBL)

Definition: Explanation-Based Learning is a form of learning in which the learner uses background knowledge to understand why a particular instance is an example of a concept, allowing it to generalize from this explanation.

Characteristics:

- **Focus on Explanations:** EBL emphasizes understanding the underlying reasons behind examples rather than just memorizing them.
- **Generalization:** It enables the learner to generalize from a small number of examples by extracting relevant features and relationships.

Process:

1. **Identify an Example:** Select an example that fits the desired concept.
2. **Generate an Explanation:** Use existing knowledge to explain why the example is a member of the concept.
3. **Construct Generalizations:** From the explanation, construct rules that generalize beyond the specific example.

Example: If a learner is given the example of a bird that can fly, it might generalize that "all birds can fly." However, through EBL, it realizes that "birds with wings" is a more accurate generalization.

Statistical Learning Methods

Definition: Statistical Learning Methods involve using statistical approaches to model and understand data, allowing for predictions based on the identified patterns.

Characteristics:

- **Model-Based:** Constructs probabilistic models to understand the relationships within data.
- **Feature Selection:** Focuses on selecting relevant features that contribute to the predictive capability of the model.

Common Methods:

- **Linear Regression:** A method for predicting a continuous outcome based on one or more predictor variables.
- **Logistic Regression:** Used for binary classification problems to predict the probability of a binary outcome.
- **Bayesian Methods:** Incorporate prior knowledge into the learning process to make predictions.

Example: In a dataset predicting house prices, a statistical model might use features like size, location, and number of bedrooms to determine the price.

Reinforcement Learning (RL)

Definition: Reinforcement Learning is a type of machine learning where an agent learns to make decisions by performing actions in an environment to maximize cumulative rewards.

Characteristics:

- **Trial-and-Error Learning:** The agent learns through exploration and exploitation, adjusting its actions based on the rewards received.
- **Markov Decision Process:** Often modeled as a Markov Decision Process (MDP), where states, actions, and rewards are defined.

Components:

- **Agent:** The learner or decision-maker.
- **Environment:** Everything the agent interacts with.
- **Actions:** Choices made by the agent that affect its state.
- **Rewards:** Feedback from the environment based on the agent's actions.

Example: In a simple game like Tic-Tac-Toe, the agent learns to play by trying different moves, receiving rewards (e.g., +1 for winning, -1 for losing), and adjusting its strategy over time to improve its chances of winning.



UNIT-V

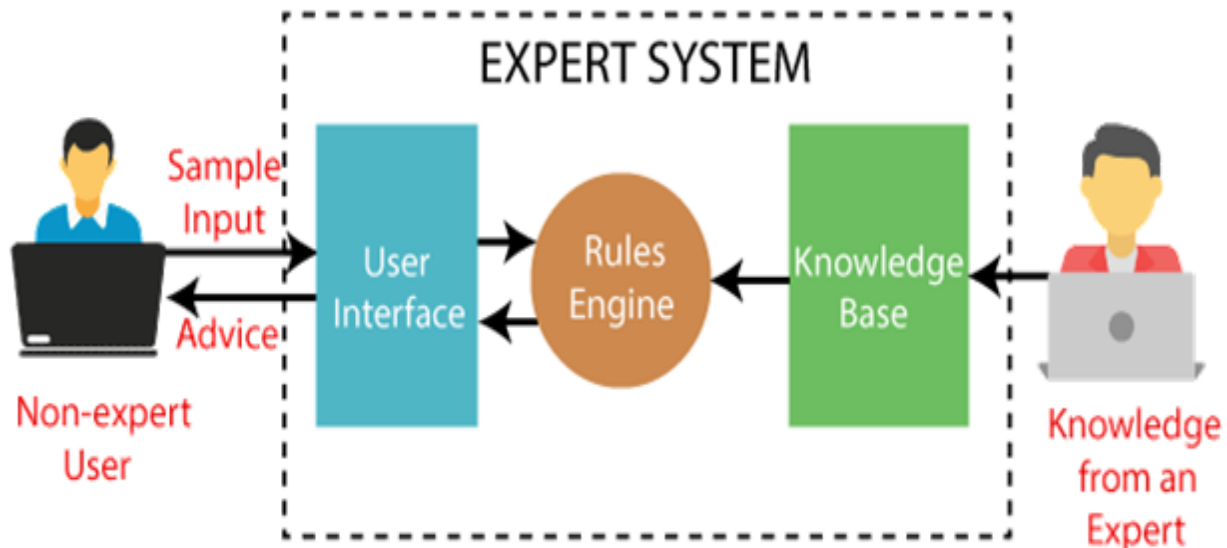
Expert Systems: Architecture of expert systems, Roles of expert systems – Knowledge Acquisition Meta knowledge Heuristics. Typical expert systems – MYCIN, DART, XCON: Expert systems shell, Hardware based AI-GPU, TPU, NPU.

Expert System:

An expert system is a computer program that is designed to solve complex problems and to provide decision-making ability like a human expert. It performs this by extracting knowledge from its knowledge base using the reasoning and inference rules according to the user queries.

The expert system is a part of AI, and the first ES was developed in the year 1970, which was the first successful approach of artificial intelligence. It solves the most complex issue as an expert by extracting the knowledge stored in its knowledge base. The system helps in decision making for complex problems using both facts and heuristics like a human expert. It is called so because it contains the expert knowledge of a specific domain and can solve any complex problem of that particular domain. These systems are designed for a specific domain, such as medicine, science, etc.

The performance of an expert system is based on the expert's knowledge stored in its knowledge base. The more knowledge stored in the KB, the more that system improves its performance. One of the common examples of an ES is a suggestion of spelling errors while typing in the Google search box.



Below are some popular examples of the Expert System:

DENDRAL: It was an artificial intelligence project that was made as a chemical analysis expert system. It was used in organic chemistry to detect unknown organic molecules with the help of their mass spectra and knowledge base of chemistry.

MYCIN: It was one of the earliest backward chaining expert systems that was designed to find the bacteria causing infections like bacteraemia and meningitis. It was also used for the recommendation of antibiotics and the diagnosis of blood clotting diseases.

PXDES: It is an expert system that is used to determine the type and level of lung cancer. To determine the disease, it takes a picture from the upper body, which looks like the shadow. This shadow identifies the type and degree of harm.

CaDeT: The CaDet expert system is a diagnostic support system that can detect cancer at early stages.

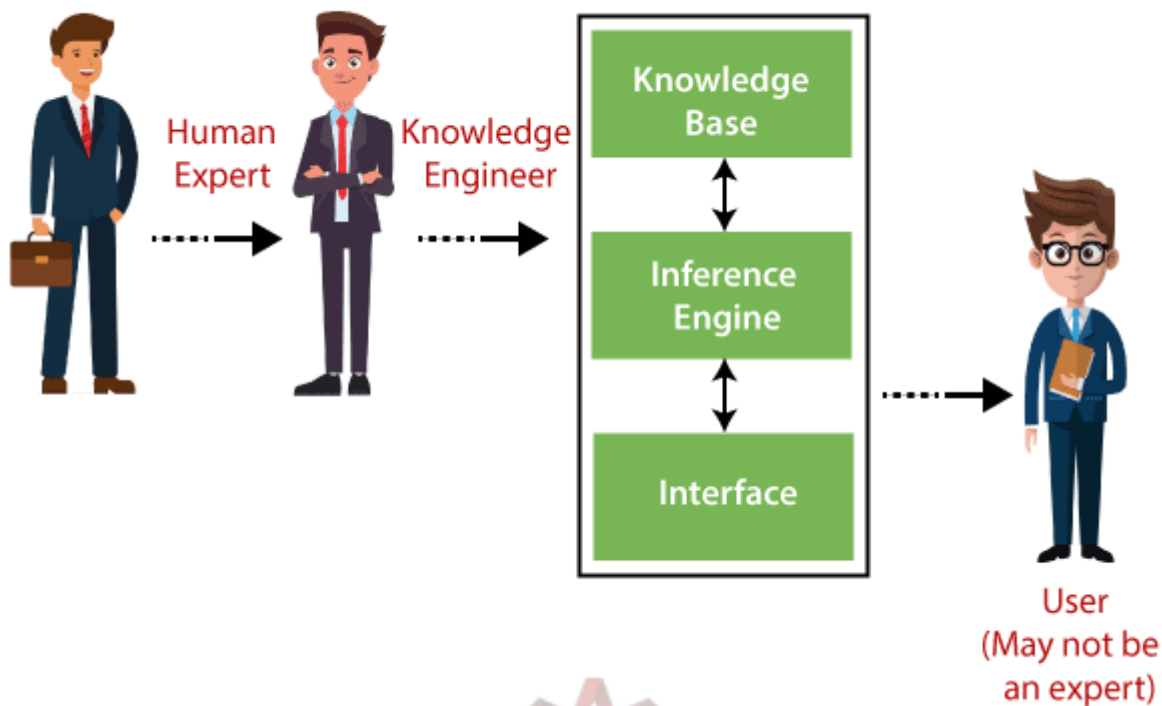
Characteristics of Expert System

- **High Performance:** The expert system provides high performance for solving any type of complex problem of a specific domain with high efficiency and accuracy.
- **Understandable:** It responds in a way that can be easily understandable by the user. It can take input in human language and provides the output in the same way.
- **Reliable:** It is much reliable for generating an efficient and accurate output.
- **Highly responsive:** ES provides the result for any complex query within a very short period of time.

Components of Expert System

An expert system mainly consists of three components:

- User Interface
- Inference Engine
- Knowledge Base



1. User Interface

With the help of a user interface, the expert system interacts with the user, takes queries as an input in a readable format, and passes it to the inference engine. After getting the response from the inference engine, it displays the output to the user. In other words, it is an interface that helps a non-expert user to communicate with the expert system to find a solution.

2. Inference Engine(Rules of Engine)

- The inference engine is known as the brain of the expert system as it is the main processing unit of the system. It applies inference rules to the knowledge base to derive a conclusion or deduce new information. It helps in deriving an error-free solution of queries asked by the user.
- With the help of an inference engine, the system extracts the knowledge from the knowledge base.
- There are two types of inference engine:
- Deterministic Inference engine: The conclusions drawn from this type of inference engine are assumed to be true. It is based on facts and rules.
- Probabilistic Inference engine: This type of inference engine contains uncertainty in conclusions, and based on the probability.

Inference engine uses the below modes to derive the solutions:

- Forward Chaining: It starts from the known facts and rules, and applies the inference rules to add their conclusion to the known facts.
- Backward Chaining: It is a backward reasoning method that starts from the goal and works backward to prove the known facts.

3. Knowledge Base

- The knowledgebase is a type of storage that stores knowledge acquired from the different experts of the particular domain. It is considered as big storage of knowledge. The more the knowledge base, the more precise will be the Expert System.
- It is similar to a database that contains information and rules of a particular domain or subject.
- One can also view the knowledge base as collections of objects and their attributes. Such as a Lion is an object and its attributes are it is a mammal, it is not a domestic animal, etc.

Components of Knowledge Base

- **Factual Knowledge:** The knowledge which is based on facts and accepted by knowledge engineers comes under factual knowledge.
- **Heuristic Knowledge:** This knowledge is based on practice, the ability to guess, evaluation, and experiences.

Knowledge Representation: It is used to formalize the knowledge stored in the knowledge base using the If-else rules.

Knowledge Acquisitions: It is the process of extracting, organizing, and structuring the domain knowledge, specifying the rules to acquire the knowledge from various experts, and store that knowledge into the knowledge base.

Development of Expert System

Here, we will explain the working of an expert system by taking an example of MYCIN ES. Below are some steps to build an MYCIN:

- Firstly, ES should be fed with expert knowledge. In the case of MYCIN, human experts specialized in the medical field of bacterial infection, provide information about the causes, symptoms, and other knowledge in that domain.
- The KB of the MYCIN is updated successfully. In order to test it, the doctor provides a new problem to it. The problem is to identify the presence of the bacteria by inputting the details of a patient, including the symptoms, current condition, and medical history.
- The ES will need a questionnaire to be filled by the patient to know the general information about the patient, such as gender, age, etc.
- Now the system has collected all the information, so it will find the solution for the problem by applying if-then rules using the inference engine and using the facts stored within the KB.
- In the end, it will provide a response to the patient by using the user interface.

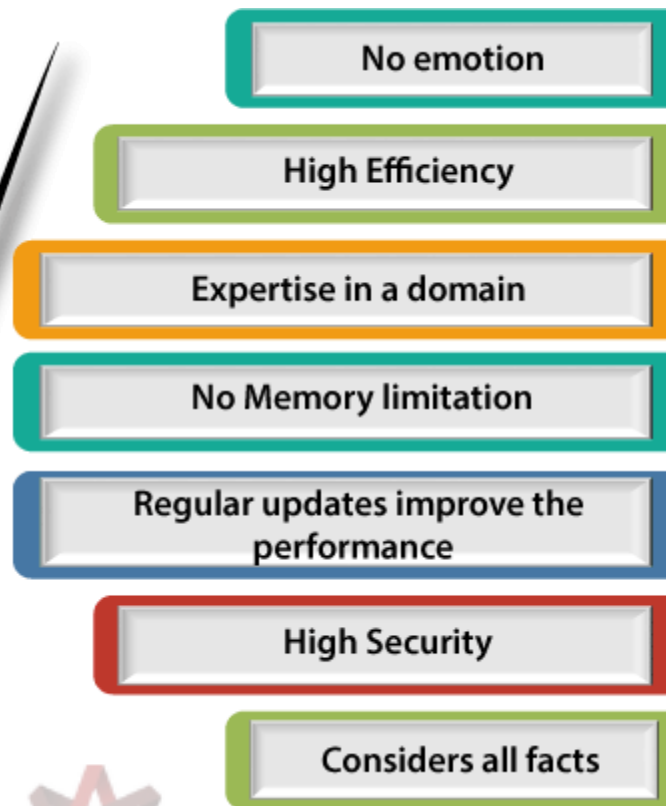
Participants in the development of Expert System

There are three primary participants in the building of Expert System:

- **Expert:** The success of an ES much depends on the knowledge provided by human experts. These experts are those persons who are specialized in that specific domain.
- **Knowledge Engineer:** Knowledge engineer is the person who gathers the knowledge from the domain experts and then codifies that knowledge to the system according to the formalism.
- **End-User:** This is a particular person or a group of people who may not be experts, and working on the expert system needs the solution or advice for his queries, which are complex.

Why Expert System?

Why Expert System



Below are the points that are describing the need of the ES:

- **No memory Limitations:** It can store as much data as required and can memorize it at the time of its application. But for human experts, there are some limitations to memorize all things at every time.
- **High Efficiency:** If the knowledge base is updated with the correct knowledge, then it provides a highly efficient output, which may not be possible for a human.
- **Expertise in a domain:** There are lots of human experts in each domain, and they all have different skills, different experiences, and different skills, so it is not easy to get a final output for the query. But if we put the knowledge gained from human experts into the expert system, then it provides an efficient output by mixing all the facts and knowledge
- **Not affected by emotions:** These systems are not affected by human emotions such as fatigue, anger, depression, anxiety, etc.. Hence the performance remains constant.
- **High security:** These systems provide high security to resolve any query.
- **Considers all the facts:** To respond to any query, it checks and considers all the available facts and provides the result accordingly. But it is possible that a human expert may not consider some facts due to any reason.
- **Regular updates improve the performance:** If there is an issue in the result provided by the expert systems, we can improve the performance of the system by updating the knowledge base.

Capabilities of the Expert System

Below are some capabilities of an Expert System:

- Advising: It is capable of advising the human being for the query of any domain from the particular ES.
- Provide decision-making capabilities: It provides the capability of decision making in any domain, such as for making any financial decision, decisions in medical science, etc.
- Demonstrate a device: It is capable of demonstrating any new products such as its features, specifications, how to use that product, etc.
- Problem-solving: It has problem-solving capabilities.
- Explaining a problem: It is also capable of providing a detailed description of an input problem.
- Interpreting the input: It is capable of interpreting the input given by the user.
- Predicting results: It can be used for the prediction of a result.
- Diagnosis: An ES designed for the medical field is capable of diagnosing a disease without using multiple components as it already contains various inbuilt medical tools.

Advantages of Expert System

- These systems are highly reproducible.
- They can be used for risky places where the human presence is not safe.
- Error possibilities are less if the KB contains correct knowledge.
- The performance of these systems remains steady as it is not affected by emotions, tension, or fatigue.
- They provide a very high speed to respond to a particular query.

Limitations of Expert System

- The response of the expert system may get wrong if the knowledge base contains the wrong information.
- Like a human being, it cannot produce a creative output for different scenarios.
- Its maintenance and development costs are very high.
- Knowledge acquisition for designing is much difficult.
- For each domain, we require a specific ES, which is one of the big limitations.
- It cannot learn from itself and hence requires manual updates.

Applications of Expert System

- In designing and manufacturing domain
It can be broadly used for designing and manufacturing physical devices such as camera lenses and automobiles.
- In the knowledge domain
These systems are primarily used for publishing the relevant knowledge to the users. The two popular ES used for this domain is an advisor and a tax advisor.
- In the finance domain
In the finance industries, it is used to detect any type of possible fraud, suspicious activity, and advise bankers that if they should provide loans for business or not.
- In the diagnosis and troubleshooting of devices

In medical diagnosis, the ES system is used, and it was the first area where these systems were used.

- **Planning and Scheduling**

The expert systems can also be used for planning and scheduling some particular tasks for achieving the goal of that task.

Typical expert systems – MYCIN, DART, XCON

1. MYCIN (Medical Diagnosis System)

Overview:

- Developed at Stanford University in the 1970s.
- Focused on diagnosing infectious blood diseases and suggesting suitable antibiotics.
- Based on backward chaining and rule-based reasoning.

Key Features:

1. Knowledge Base:

- Contained about 600 rules related to infectious diseases.
- Rules were in the form of *if-then* statements.

IF: The gram stain of the organism is gram-positive

AND: The morphology is coccus

THEN: There is suggestive evidence (0.7) that the organism is Streptococcus.

2. Inference Engine:

- Used to match symptoms and test results with rules in the knowledge base.
- Derived diagnoses and treatment recommendations.

3. Explanation Facility:

- Explained its reasoning to doctors.
- Example: "The system suggests this antibiotic because the organism is Streptococcus and is resistant to penicillin."

Advantages:

- Could achieve diagnostic accuracy comparable to expert physicians.
- Reduced dependency on human experts for routine diagnoses.

Limitations:

- Required manual input of symptoms and lab results.
- Focused on a narrow domain (infectious diseases).

2. DART (Dynamic Analysis and Replanning Tool)

Overview:

- Developed for the U.S. Department of Defense during the Gulf War.
- Assisted in logistics and transportation planning for military operations.

Key Features:

- 1. Knowledge Base:**
 - Contained data about transportation networks, resources, and schedules.
 - Integrated real-time updates on changes in resources or routes.
- 2. Inference Engine:**
 - Optimized planning by analyzing different scenarios and providing recommendations.
 - Example: Identified the best routes and transportation modes for deploying troops and equipment efficiently.
- 3. User Interface:**
 - Allowed planners to input constraints (e.g., time, resources) and receive actionable plans.

Advantages:

- Enabled rapid and accurate logistical planning during high-stakes situations.
- Saved significant time and resources during the Gulf War.

Limitations:

- Complex system requiring continuous updates to the knowledge base.
- Domain-specific and not easily adaptable to civilian logistics.

3. XCON (Expert Configuration System)

Overview:

- Developed by Digital Equipment Corporation (DEC) in the 1980s.
- Automated the configuration of VAX computer systems based on customer orders.

Key Features:

1. Knowledge Base:

Encoded rules about VAX system components and their compatibility.

IF: A customer orders a specific CPU model

THEN: Include compatible power supply and memory modules.

2. Inference Engine:

- Ensured that every configuration was valid and met customer specifications.
- Automated error checking and reduced the need for manual intervention.

3. Explanation Facility:

- Justified the selection of components in a configuration.
- Example: *"This memory module is selected because it is compatible with the CPU model ordered."*

Advantages:

- Significantly reduced the time and cost of system configuration.
- Minimized errors and improved customer satisfaction.

Limitations:

- Required frequent updates as new hardware components were introduced.
- Limited to DEC's products and not adaptable to other hardware manufacturers.



Comparison of Expert Systems

Feature	MYCIN	DART	XCON
Domain	Medical Diagnosis	Military Logistics	Computer System Configuration
Type	Rule-Based System	Planning and Optimization System	Rule-Based System
Key Functionality	Diagnose diseases, suggest antibiotics	Plan logistics, optimize transportation	Configure computer systems
User Interaction	Input patient symptoms, receive treatment recommendations	Input constraints, receive optimized plans	Input customer orders, receive configurations
Primary Advantage	Expert-level diagnosis	Rapid, efficient planning	Accurate, error-free configurations
Primary Limitation	Limited to infectious diseases	Specific to military operations	Limited to DEC products

